



Hochschule für Angewandte Wissenschaften Hamburg
Hamburg University of Applied Sciences

DEPARTMENT INFORMATION

Bachelorarbeit

Berufliche Kompetenzen in Online-Stellenausschreibungen -
Entwicklung einer Methode zur automatischen Identifizierung von
Kompetenzen mit der Programmiersprache Python

vorgelegt von

Kathrin Wardatzky

Studiengang Bibliotheks- und Informationsmanagement

erste Prüferin: Prof. Dr. Susanne Glissmann
zweite Prüferin: Dr.-Ing. Maika Büschenfeldt

Hamburg, November 2016

Abstract

Diese Arbeit befasst sich mit der Frage, wie mit der Programmiersprache Python automatisch berufliche Kompetenzen in Online-Stellenbeschreibungen identifiziert werden können.

Nach der Beschreibung der theoretischen Grundlagen zu beruflichen Kompetenzen, Stellenanzeigenanalysen und der Programmiersprache Python, wurden bestehende Vorgehensmodelle und Verfahrensweisen aus den Bereichen Data, Text und Job Mining bezüglich der abgedeckten Phasen und genutzten Methoden verglichen. Auf Grundlage dessen wurde die Prozessphasen *Konzeptualisierung*, *Datensammlung*, *Datenaufbereitung* und *Datenanalyse* gebildet und zunächst das Vorgehen in den einzelnen Phasen beschrieben. Dabei wurde ein besonderer Fokus auf die Struktur und Besonderheiten des Datentyps Stellenausschreibungen gelegt. Begleitend wurden Beispiele aus der Analyse von 1036 Stellenausschreibungen aus dem Berufsfeld *Bibliotheks- und Informationsmanagement*, die von drei Stellenbörsen im Zeitraum von drei Monaten gesammelt und mit Hilfe eines Kategoriensystems ausgewertet wurden, angefügt. Die dabei entstandenen Skripte wurden in der interaktiven Entwicklungsumgebung Jupyter Notebook erstellt und mit erklärendem Text angereichert. Dadurch sind nur wenige Anpassungen nötig, um die Skripte für andere Berufsfelder wiederzuverwerten. Durch die Kombination aus erläuterndem Text und interaktiven Codeelementen sind die Skripte auch von Personen mit wenig Programmiererfahrung nutzbar.

Schlagworte: Stellenanzeigenanalyse – Python – Text Mining – lexikonbasiertes Verfahren – Jupyter Notebook – methodisches Vorgehen – berufliche Kompetenzen

Inhaltsverzeichnis

Abstract	II
Inhaltsverzeichnis	III
Abbildungsverzeichnis	V
Tabellenverzeichnis	VI
Listings	VII
Abkürzungsverzeichnis	VIII
1. Einleitung	1
1.1. Stand der Forschung	2
1.2. Zielsetzung der Arbeit	2
1.3. Aufbau der Arbeit	3
2. Theoretische Grundlagen	5
2.1. Berufliche Kompetenzen	5
2.2. Stellenanzeigenanalyse	6
2.3. Programmiersprache Python	7
2.3.1. Distribution Anaconda	8
2.3.2. Python Bibliotheken	9
3. Verfahrensweisen und Vorgehensmodelle	12
3.1. Data Mining Vorgehensmodelle	13
3.2. Text Mining Vorgehensmodelle und Verfahren	15
3.3. Job Mining Verfahren	19
3.4. Fazit	21
4. Phasen der Methode	24
4.1. Phase I: Konzeptualisierung	24
4.1.1. Forschungsfrage	25
4.1.2. Eingrenzung der potenziellen Datenmenge	26
4.1.3. Eigenschaften der Datenquellen	28
4.1.4. Anwendungsbeispiel	33
4.1.5. Zusammenfassung	35
4.2. Phase II: Datensammlung	36
4.2.1. Web Scraping	36
4.2.2. Dublettenkontrolle	42
4.2.3. Anwendungsbeispiel	44
4.2.4. Zusammenfassung	48
4.3. Phase III: Datenaufbereitung	49

4.3.1. Extrahieren der Stellenbeschreibung und Erstellen eines Korpus	50
4.3.2. Verarbeitungsverfahren.....	52
4.3.3. Anwendungsbeispiel	56
4.3.4. Zusammenfassung	60
4.4. Phase IV: Datenanalyse	61
4.4.1. Kategoriensystem	62
4.4.2. Worthäufigkeiten	63
4.4.3. Anwendungsbeispiel	64
4.4.4. Zusammenfassung	65
5. Qualitätskontrolle	67
5.1. Qualität der Suchbegriffe.....	67
5.2. Evaluation des Kategoriensystems.....	68
5.3. Feedbackschleifen	69
6. Zusammenfassung und Ausblick	70
Literaturverzeichnis.....	72
Anhang	i
Anhang 1: Beigabe (CD)	i
Anhang 2: Skripte zur Phase II – Datensammlung	ii
Anhang 2.1.: Data Collection - Stepstone	ii
Anhang 2.2.: Data Collection – Zeit Online	vii
Anhang 2.3.: Data Collection – Xing	xii
Anhang 3: Skripte zur Phase III – Datenaufbereitung	xviii
Anhang 3.1. Job Ad Extraction - Stepstone.....	xviii
Anhang 3.2. Job Ad Extraction – Zeit.....	xxi
Anhang 3.3. Job Ad Extraction – Xing.....	xxiv
Anhang 3.4. Data Preparation.....	xxvii
Anhang 4: Skript zur Phase IV – Datenanalyse	xxxiii
Anhang 5: Kategoriensystem	xxxvii
Anhang 6: Stoppwortliste.....	xl

Abbildungsverzeichnis

Abbildung 1: Screenshot “The Zen of Python”, Tim Peters	8
Abbildung 2: CRISP-DM Modell (CC-BY-SA, Jensen 2012).....	13
Abbildung 3: Job-Mining-Prozess im Überblick (nach Bensberg, Buscher 2016, S. 3) 19	
Abbildung 4: Input und Output Phase I	25
Abbildung 5: Aufbau einer Stellenanzeige mit Strukturelementen	30
Abbildung 6: Aufbau einer Stellenanzeige ohne Strukturelemente	32
Abbildung 7: Konzeptualisierungsphase	35
Abbildung 8: Input und Output Phase II – Datensammlung	36
Abbildung 9: Robots.txt Datei von Stepstone	38
Abbildung 10: Zusammenfassung Phase II: Datensammlung	49
Abbildung 11: Input und Output Phase III - Datenaufbereitung	50
Abbildung 12: Zusammenfassung Phase III - Datenaufbereitung.....	61
Abbildung 13: Input und Output Phase IV - Datenanalyse	61
Abbildung 14: Zusammenfassung Phase IV – Datenanalyse	66
Abbildung 15: Gesamtdarstellung der entwickelten Methode	70

Tabellenverzeichnis

Tabelle 1: Genutzte Python Bibliotheken aus der Anaconda Distribution	10
Tabelle 2: Phasenabdeckung CRISP-DM	15
Tabelle 3: Phasenabdeckung Analyseverfahren von Backs et al.	16
Tabelle 4: Phasenabdeckung generisches Vorgehensmodell von Schieber und Hilbert	18
Tabelle 5: Phasenabdeckung Job Mining Analyse nach Bensberg und Buscher (2016)	20
Tabelle 6: Phasenabdeckung Yang et al.....	21
Tabelle 7: Zusammenfassung Phasenabdeckung der Modelle/Verfahren.....	22
Tabelle 8: Vergleich Stellenbörsen.....	29
Tabelle 9: Berufsbezeichnungen Bibliotheks- und Informationsmanagement.....	34
Tabelle 10: Suchbegriffe aller genutzten Stellenbörsen	44
Tabelle 11: Beispiel POS-Tagset (nach Backs et al. 2014)	54
Tabelle 12: Auszug des Kategoriensystems	63
Tabelle 13: Finaler Output des Beispiels in Phase IV.....	65
Tabelle 14: Vergleich Treffermenge - Precision Zeit und Stepstone (Suchergebnisse vom 01.11.2016)	68

Listings

Listing 1: HTML-Struktur des Beispiels aus Abbildung 5 (vereinfachte Darstellung)....	31
Listing 2: HTML-Struktur des Beispiels aus Abbildung 6 (vereinfachte Darstellung)....	33
Listing 3: Funktion, um Google Spreadsheet als Data Frame zu importieren	40
Listing 4: Funktion zum Scrapen der Suchübersicht	40
Listing 5: Funktion zum Speichern der HTML-Dateien	40
Listing 6: Extrahieren der URL zur Stellenausschreibung in der Suchübersicht ZeitOnline	41
Listing 7: Extrahieren der URL zur Stellenausschreibung in der Suchübersicht von Stepstone	41
Listing 8: Scrapen und Speichern der Stellenbeschreibungen von Stepstone	42
Listing 9: Erstellen einer Liste von Suchbegriffen aus einer Tabellenspalte	45
Listing 10: Scrapen und Speichern der Suchübersichtsseiten.....	45
Listing 11: Extraktion der URL und der Metadaten zu den Stellenausschreibungen....	47
Listing 12: Scrapen und Speichern der Stellenausschreibung.....	48
Listing 13: Tagger import	53
Listing 14: Extraktion der Analyseeinheiten.....	56
Listing 15: Extrahieren und Hinzufügen von Metadaten	57
Listing 16: Korpus erstellen.....	58
Listing 17: Entfernen von Tags und Extrahieren von Listenelementen	58
Listing 18: Wortsegmentierung und POS-Tagging der Listenelemente	59
Listing 19: Speichern als Excel-Datei.....	59
Listing 20: Standard Stoppwortentfernung	59
Listing 21: Individuelle Stoppwortentfernung	60
Listing 22: Stemming	60
Listing 23: Wortfrequenzanalyse Adjektive.....	64
Listing 24: Laden des Kategoriensystems und Stemming der Synonym-Spalte	64
Listing 25: Matching von Synonymen.....	65
Listing 26: Anzeigen der Kompetenzgruppen zu den gefundenen Fähigkeiten	65

Abkürzungsverzeichnis

BIBB	Bundesinstitut für Berufsbildung
BGH	Bundesgerichtshof
CRISP-DM	Cross Industry Standard Process for Data Mining
CSS	Cascading Style Sheets
CSV	Comma Separated Value
DOM	Document Object Model
HAW	Hochschule für Angewandte Wissenschaft
HfTL	Hochschule für Telekommunikation Leipzig
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
KIdB	Klassifikation der Berufe
KWIC	Key Word in Context
MIT	Massachusetts Institute of Technology
NLP	Natural Language Processing
NLTK	Natural Language Toolkit
POS	Part of Speech
Regex	Regular Expressions
URL	Uniform Resource Locator

1. Einleitung

Was muss man für Kompetenzen mitbringen, um gute Chancen auf dem Arbeitsmarkt zu haben? Welchen Schwerpunkt sollte man im Studium wählen, um bestmöglich auf die Berufswelt vorbereitet zu sein? Diese Fragen hat sich vermutlich jeder, der sich beruflich orientieren wollte, schon gestellt. Auch Bildungseinrichtungen sollten sich einen Überblick verschaffen, welche Anforderungen in der Berufswelt gefordert sind, um auf etwaige Veränderungen reagieren zu können und ihren Absolventen einen guten Einstieg zu ermöglichen. Wie kann man auf diese Fragen verlässliche Antworten finden? Eine Möglichkeit bietet eine Stellenanzeigenanalyse, da Stellenausschreibungen den aktuellen Bedarf und die Nachfrage der Kompetenzen auf dem Arbeitsmarkt widerspiegeln (Sailer 2009, S. 36).

Mit dem Aufkommen des Internets verschob sich der Publikationsort für Stellenbeschreibungen zunehmend von Printmedien in den online Bereich. Carnevale et al. schätzen, dass in 2013 etwa 60 bis 70 Prozent der Stellenausschreibungen in den USA online veröffentlicht wurden (2014, S. 11). Dadurch ist die Menge an zugänglichen Stellenausschreibungen enorm angestiegen, so dass es unmöglich wäre, manuell genügend Stellenausschreibungen zu sichten, um sich ein genaues Bild der geforderten Kompetenzen zu machen. Gleichzeitig bietet die Digitalisierung die Möglichkeit eine große Menge an Stellenanzeigen mit automatischen Verfahren zu analysieren und die daraus gewonnenen Erkenntnisse zu nutzen.

Das Forschungsprojekt JobMining@HfTL der Hochschule für Telekommunikation Leipzig (HfTL), unter der Leitung von Prof. Dr. Frank Bensberg, nutzt zum Beispiel eine Text Mining Methode, um Stellenbeschreibungen hinsichtlich der geforderten Kompetenzen im ICT-Sektor mit der IBM Analytics Software zu analysieren. Die Ergebnisse fließen in die Curricula-Planung entsprechender Studiengänge an der HfTL ein (Hochschule für Telekommunikation Leipzig 2016).

Aber ist es auch möglich, mit frei zugänglichen Programmen automatisch die Kompetenzen in online Stellenausschreibungen zu identifizieren? Durch das Aufkommen verschiedener E-Learning Plattformen, wie zum Beispiel CodeCademy oder das auf Data Science spezifizierte DataCamp, ist es auch für unerfahrene Menschen möglich, Grundlagen der Programmierung zu lernen. Zusätzlich richten sich Publikationen, wie zum Beispiel die von Sweigart, an Personen, die Routineaufgaben im Alltag durch Programmieren erleichtern möchten (Sweigart 2015, S. 2). Daher liegt die

Vermutung nahe, dass auch Menschen mit wenig Erfahrung im Programmieren und im Umgang mit großen Datenmengen die Analyse von Stellenbeschreibungen für ihre Zwecke nutzen können.

1.1. Stand der Forschung

Das Instrument der Stellenanzeigenanalyse wird bereits seit den 1980er Jahre von Forschern mit unterschiedlichen Methoden und Zielsetzungen genutzt, um Aussagen und Prognosen über den Arbeitsmarkt treffen zu können (Sailer 2009, S. 35). Demzufolge gibt es sowohl zum methodischen Vorgehen als auch zur Analyse von verschiedenen Berufsfeldern anhand von Stellenbeschreibungen bereits eine große Menge an Literatur, beispielsweise von Ahmed (2005), Backs et al. (2014) sowie von Bensberg und Buscher (2016).

Auch die Methode Text Mining zur Durchführung der Stellenausschreibungsanalyse ist bereits ausgiebig in der Fachliteratur behandelt worden (zum Beispiel Gao und Eldin 2014; Debortoli et al. 2014; Yang et al. 2016). Bei den Vorgehensmodellen und Verfahrensweisen wird allerdings selten ein Fokus auf die spezifische Umsetzung gelegt und konkrete Beispiele, wie etwa exemplarischer Code, sind nicht zu finden. Das lässt sich damit erklären, dass sich sämtliche recherchierte Vorgehensmodelle und Verfahrensweisen an ein Fachpublikum richten und die jeweilige Methode Flexibilität in der Umsetzung bieten möchte. Harper bestätigt diesen Mangel an Anleitung in der Fachliteratur, insbesondere in den Bereichen Datensammlung und -analyse (2012, S. 32).

Eine Analyse von sowohl anwendungsneutralen als auch auf Stellenausschreibungen spezifizierte Vorgehensmodellen und -verfahren wird in Kapitel 3 vorgenommen, weshalb diese hier nicht weiter ausgeführt wird.

1.2. Zielsetzung der Arbeit

In dieser Arbeit entsteht auf Basis von bestehenden Vorgehensmodellen im Data und Text Mining eine spezifische Methode, die auf die Besonderheiten des Datentyps „Stellenausschreibung“ und der Programmiersprache Python angepasst ist. Im Unterschied zu den bereits bestehenden Methoden sollen Personen mit wenig Erfahrung im Programmieren und im Umgang mit Daten das beschriebene Vorgehen

nachvollziehen und anwenden können, weshalb konkrete Beispiele und die dazu entwickelten Skripte zur Verfügung gestellt werden. Diese Methode soll zunächst in dem Studiengangübergreifenden Projekt *Digitale Profile*, das im Wintersemester 2016/17 am Department Information der HAW Hamburg stattfindet, genutzt werden. Zudem werden einzelne Elemente in zukünftigen Seminaren und Projekten genutzt, so dass die Methode eine generelle Grundlage für Datenprojekte am Department bildet. Dadurch entsteht der Anspruch, dass das beschriebene Vorgehen und der für diese Arbeit entwickelte Code auch für im Programmieren unerfahrene Studierende, gegebenenfalls nach einer kurzen Einführung in die Programmiersprache, nachvollzogen werden kann. Des Weiteren soll auch das kollaborative Arbeiten mit dieser Methode ermöglicht werden. Vorausgesetzt werden lediglich die Kenntnisse, die am Department in den IT-Pflichtseminaren gelehrt werden. Insbesondere sind das Kenntnisse der HTML-Struktur und, unabhängig von der Programmiersprache, Grundlagen der Programmierung, wie zum Beispiel Schleifen, Funktionen, Operatoren und bedingte Anweisungen.

Zusammenfassend ergibt sich aus den oben genannten Kriterien folgende Forschungsfrage, an der sich diese Arbeit orientiert: Wie geht man vor, wenn man auf möglichst einfache Art und Weise mit der Programmiersprache Python Kompetenzen in Stellenausschreibungen identifizieren möchte?

1.3. Aufbau der Arbeit

Die Arbeit beginnt mit theoretischen Grundlagen und Definitionen, die zum Verständnis der Methode notwendig sind und begründen, warum Abgrenzungen und Entscheidungen getroffen wurden. Die Grundlagen beruhen auf Literatur, die im Hinblick auf die Zielsetzung der Arbeit recherchiert und analysiert wurde. Im Einzelnen beinhaltet das Kapitel sowohl Hintergründe und Definitionen zu den technischen Komponenten der Forschungsfrage, wie Text Mining oder der Programmiersprache Python, als auch zu den nicht-technischen Komponenten, wie zum Beispiel der Methode der Stellenbeschreibungsanalyse und der Abgrenzung von beruflichen Kompetenzen.

Im weiteren Verlauf werden fünf ausgewählte Vorgehensmodelle und -verfahren aus dem Data und Text Mining vorgestellt und hinsichtlich der Phasenabdeckung und dem beschriebenen methodischen Vorgehen verglichen. Zielsetzung dieses Kapitels ist die Extraktion von Elementen, die zum Erreichen der Zielsetzung dieser Arbeit beitragen.

Kapitel 4 beschreibt die einzelnen Phasen, die nötig sind, um mit Python eine Stellenbeschreibungsanalyse durchzuführen. Unterstützt wird dieses von Beispielen aus einer für diese Arbeit exemplarisch durchgeführten Stellenausschreibungsanalyse des Berufsfelds *Bibliotheks- und Informationsmanagement*. Dabei wird auf die spezifischen Herausforderungen, die durch die Daten oder die Programmiersprache gegeben sind, eingegangen. Der Programmcode wurde in der Entwicklungsumgebung Jupyter Notebook mit Python 3.5 erstellt. Für die Beispiele wurden 1036 Stellenausschreibungen aus drei unterschiedlichen Stellenbörsen im Zeitraum vom 31.07. bis zum 25.10.2016 gesammelt und ausgewertet.

Kapitel 5 zeigt im Anschluss Möglichkeiten auf, die Qualität der durchgeführten Analyse zu messen und zu verbessern, bevor die Methode abschließend in Kapitel 6 noch einmal zusammengefasst und ein Ausblick auf mögliche Anwendungen der Analyseergebnisse gegeben wird.

2. Theoretische Grundlagen

Im folgenden Abschnitt werden die theoretischen Hintergründe zu der entwickelten Methode beschrieben und begründet, warum sich die beschriebenen Methoden und Elemente für den spezifischen Anwendungsfall eignen.

2.1. Berufliche Kompetenzen

Im Bildungskontext wird der Kompetenzbegriff weitgehend als „Verbindung von Wissen und Können in der Bewältigung von Handlungsanforderungen“ definiert (Bundesinstitut für Berufsbildung 2016). Im beruflichen Kontext wird daher oft von Handlungskompetenz gesprochen. Es existieren viele Kompetenzmodelle, was die einheitliche Zuordnung einzelner Fähigkeiten zu einem Kompetenzbereich erschwert. Das Gabler Wirtschaftslexikon unterteilt den Begriff Handlungskompetenz in Fach-, Methoden- und Sozialkompetenz (Springer Gabler Verlag 2016d). Der Europäische Qualifikationsrahmen für Lebenslanges Lernen schließt in seiner Definition von Kompetenz auch die persönliche Kompetenz ein (Europäische Kommission Bildung und Kultur 2008). Das European Dictionary of Skills and Competences, das die Grundlage für das in Kapitel 4.4.1. beschriebene Kompetenzwörterbuch bildet, beruht auf der Definition des Kompetenzbegriffes des Europäischen Qualifikationsrahmens für Lebenslanges Lernen, unterteilt den Kompetenzbegriff allerdings lediglich in persönliche, fachliche und so genannte Kernkompetenzen, die sowohl soziale als auch methodische Kompetenzen umfassen (3s Unternehmensberatung 2016).

Für die in dieser Arbeit entwickelte Methode wurde der Kompetenzbegriffes in Fach-, Sozial-, Methoden- und persönlicher Kompetenz unterteilt, da diese Unterkategorien in der Regel auch in Anforderungsprofilen von Stellenausschreibungen vorkommen (Wagner 2016). Im Folgenden werden die Unterbegriffe genannt und definiert:

- **Fachkompetenz:** Rein fachbezogene Fähigkeiten und Kenntnissen, die zum Beispiel im Rahmen einer Ausbildung oder eines Studiums erworben wurden (Springer Gabler Verlag 2016a). Die dazugehörigen Fähigkeiten können sich dabei, je nach Beruf, unterscheiden. Für einen Dolmetscher oder Übersetzer sind Sprachkenntnisse zum Beispiel eindeutige Fachkompetenzen, in anderen

Berufsfeldern würden diese Kenntnisse der Sozialkompetenz hinzugezählt werden.

- **Sozialkompetenz:** Aus der Sozialisation beziehungsweise dem sozialen Lernen entstandene Fähigkeiten, die eine gute Kommunikation und Zusammenarbeit mit anderen Menschen ermöglichen (Springer Gabler Verlag 2016b). Hierzu zählen zum Beispiel Teamfähigkeit oder Kommunikationsfähigkeiten.
- **Methodenkompetenz:** Fähigkeiten zur Anwendung von Strategien, Verfahrensweisen und Methoden, zum Beispiel zur Problemlösung (Springer Gabler Verlag 2016c). Weiterhin gehören Fähigkeiten, wie strukturiertes oder analytisches Denken zu der Methodenkompetenz.
- **Persönliche Kompetenz:** Eigenschaften, die die Persönlichkeit, innere Haltung und Einstellung ausdrücken, wie zum Beispiel Belastbarkeit oder selbstbewusstes Auftreten (Wagner 2016a).

Wie bereits im Punkt „Fachkompetenz“ angedeutet, sind die Grenzen zwischen den Kompetenzgruppen teilweise schwammig und können sich je nach Berufsfeld unterscheiden. Daher ist es wichtig, für den eigenen, spezifischen Anwendungsfall genau festzulegen, wie die Kompetenzgruppen abgegrenzt werden, da dieses das Erstellen des Kategoriensystems (siehe Kapitel 4.4.1.) erleichtert. Sind mehrere Personen an der Erstellung des Kategoriensystems beteiligt, ist es sogar unabdingbar, ein Regelwerk aufzustellen, das die Kompetenzgruppen genau abgrenzt und regelt, welche Fähigkeiten zu welcher Kompetenzgruppe zugeordnet werden.

2.2. Stellenanzeigenanalyse

Sailer (2009, S. 39) definiert die Stellenanzeigenanalyse als „die auf empirischer Basis gewonnenen Informationen aus Stellenanzeigen über derzeitige und künftige berufliche, personenbezogene und sozialkommunikative Ansprüche der Unternehmen bzw. Organisationen an spezifische Bewerbergruppen.“ Stellenanzeigen bieten demzufolge Informationen über die aktuell geforderten Kompetenzen an den Bewerber. Da Unternehmen und Organisationen ihre Stellenausschreibungen im Allgemeinen zukunftsorientiert gestalten und die Kompetenzen beschreiben, die sie langfristig

benötigen, ist es ebenfalls möglich, mit einer Stellenanzeigenanalyse Aussagen über zukünftige Anforderungen einzelner Berufsfelder zu treffen (Mehra 2015, S. 2; Sailer 2009, S. 38).

Zusätzlich zum Prognosecharakter nennen sowohl Sailer (2009, S. 39) als auch Mehra (2015, S. 2) den Vorteil, dass bei einer Stellenanzeigenanalyse keine Daten erhoben werden müssen, sondern bereits bestehende Daten gesammelt und ausgewertet werden. Dadurch entstehe keine Abhängigkeit zu Dritten, wie zum Beispiel Interviewpartner. Der nicht-reaktive Charakter der Stellenbeschreibungsanalyse verhindert ebenso eine Beeinflussung von Unternehmen auf die Analyseergebnisse. Die allgemeine und freie Zugänglichkeit der Stellenausschreibungen bedeutet zudem, dass für die Durchführung der Methode keine Kosten anfallen.

Als Nachteile der Stellenanzeigenanalyse führt Mehra (ebd.) die Tatsache an, dass es durch die hohe Anzahl an publizierten Stellenanzeigen in verschiedenen Medien lediglich möglich sei eine Stichprobe aller Anzeigen zu analysieren. Eine eventuelle Über- bzw. Unterrepräsentierung einiger Unternehmenstypen oder Branchen auf bestimmten Medien erschwere es zudem diese Stichprobe repräsentativ auszuwählen.

Sowohl Mehra (ebd.) als auch Sailer (2009, S. 39) führen zudem den hohen Arbeitsaufwand zur Aufbereitung der Daten als Nachteil an. Jedoch schließen beide das Fazit, dass die Vorteile der Stellenanzeigenanalyse diesen Aufwand rechtfertigen.

Aufgrund der genannten Nachteile, wurde im Kapitel 4.1. ein besonderer Fokus auf die Überlegungen gelegt, die vor dem eigentlichen Start des Datenprojekts stattfinden sollten.

2.3. Programmiersprache Python

Python ist eine objekt-orientierte Mehrzweckprogrammiersprache, die 1991 von Guido van Rossum entwickelt wurde. Neben den vielseitigen Einsatzmöglichkeiten zeichnet sich die Programmiersprache durch eine einfache und gut lesbare Syntax aus (Python Software Foundation 2016). Der Stellenwert, den die Einfachheit und gute Lesbarkeit bei der Entwicklung von Python einnimmt, zeigt sich ebenfalls dadurch, dass „The Zen of Python“ von Tim Peters erscheint, sobald man `import this` in der Python Konsole ausführt (Abbildung 1).

```
In [1]: import this
```

```
The Zen of Python, by Tim Peters
```

```
Beautiful is better than ugly.  
Explicit is better than implicit.  
Simple is better than complex.  
Complex is better than complicated.  
Flat is better than nested.  
Sparse is better than dense.  
Readability counts.  
Special cases aren't special enough to break the rules.  
Although practicality beats purity.  
Errors should never pass silently.  
Unless explicitly silenced.  
In the face of ambiguity, refuse the temptation to guess.  
There should be one-- and preferably only one --obvious way to do it.  
Although that way may not be obvious at first unless you're Dutch.  
Now is better than never.  
Although never is often better than *right* now.  
If the implementation is hard to explain, it's a bad idea.  
If the implementation is easy to explain, it may be a good idea.  
Namespaces are one honking great idea -- let's do more of those!
```

Abbildung 1: Screenshot "The Zen of Python", Tim Peters

Durch die Simplizität der Programmiersprache, ermöglicht Python einen guten Einstieg in das Programmieren. Am Massachusetts Institute of Technology (MIT) wird Python im Modul *Electrical Engineering and Computer Science* als Einstiegssprache in die Programmierung verwendet (MIT 2016).

Für den Bereich des Natural Language Processings (NLP) bietet Python eine Vielzahl an Bibliotheken und Entwicklungsumgebungen, von denen diejenigen, die für diese Arbeit verwendet wurden, im Folgenden vorgestellt werden.

2.3.1. Distribution Anaconda

Anaconda ist eine Distribution, die von Continuum Analytics speziell für Datenanalyse mit Python entwickelt wurde. Dabei handelt es sich um ein Paket bestehend aus der Python-Anwendung, verschiedenen Entwicklungsumgebungen, einem Paketmanager und den wichtigsten Bibliotheken für den Bereich Data Science (Continuum Analytics Inc. 2016). Nach der Installation von Anaconda können diese sofort genutzt werden. Bei der Entwicklung der anwendungsspezifischen Methode wurde bei dem Code in dieser Arbeit daher darauf geachtet, möglichst ausschließlich Bibliotheken aus der Anaconda Distribution zu verwenden, damit dieser direkt angewandt werden kann, ohne zuvor

externe Pakete installieren und kompilieren zu müssen. Größtenteils ist dieses auch gelungen, auf Ausnahmen wird gesondert hingewiesen.

Des Weiteren enthält Anaconda die interaktive, webbasierte Applikation Jupyter Notebook vom Project Jupyter, die für diese Arbeit als Entwicklungsumgebung genutzt wurde. Das Projekt Jupyter hatte seinen Ursprung in dem IPython Projekt, eine interaktive Entwicklungsumgebung, die ausschließlich für die Programmiersprache Python konzipiert war. Im Jahr 2014 folgte die Änderung zu Project Jupyter, da die Plattform zunehmend andere Data Science fokussierte Programmiersprachen, wie zum Beispiel R oder Julia, mit einbezog (Berkeley Institute for Data Science 2016).

Jupyter Notebook vereint eine browserbasierte Webapplikation, in der Dokumente erstellt werden können, die unter anderem erklärenden Text, interaktiv veränderbaren Code und dessen Output beinhalten. Zum anderen kann das Dokument gleichzeitig als Dokumentation mit sämtlichem Output und Grafiken dienen und auf einfache Art in verschiedenen Formaten, wie zum Beispiel HTML oder PDF, veröffentlicht werden (Project Jupyter 2015). Die Kombination aus Interaktivität, Nutzerfreundlichkeit und Visualisierungen unter der Einbeziehung von Daten und Code wird teilweise als mögliche Brücke zwischen der wissenschaftlichen Community und der breiten Öffentlichkeit gesehen (Abdalla 2015).

Aus diesem Grund wurde Jupyter Notebook ausgewählt, um die Skripte zu entwickeln, die dieser Arbeit zugrunde liegen und im Anhang zur Verfügung gestellt werden. Diese sollen eine Grundstruktur geben, mit denen im Programmieren unerfahrene Personen den Code einfach ausführen und gegebenenfalls individuell anpassen können. Damit die Notebook-Dateien auch in den englischsprachigen Seminaren des Departments Information verwendet werden können, wurde der erklärende Text im Markdown in Englisch verfasst.

2.3.2. Python Bibliotheken

Wie bereits im vorigen Kapitel erwähnt, bietet die Anaconda Distribution eine Vielzahl an Bibliotheken, die speziell für die Durchführung von Datenprojekten entwickelt wurden (Continuum Analytics Inc. 2016). Diese müssen zu Beginn eines jeden Python-Skripts zunächst importiert werden, bevor man sie nutzen kann.

Die in dieser Methode genutzten Bibliotheken der Anaconda Distribution werden in Tabelle 1 beschrieben.

Name der Bibliothek	Anwendungszweck
Beautiful Soup	Extraktion von Daten aus XML- oder HTML-Dateien. Mit einem Parser nach Wahl kann durch den HTML-beziehungsweise XML-Baum navigiert werden und Elemente können gesucht und verändert werden (Richardson 2015).
Pandas	Bietet umfangreiche Funktionen zur Datenanalyse und Datenstrukturierung (McKinney 2016). In dieser Arbeit wird Pandas hauptsächlich dazu verwendet, Daten in Data Frames zu strukturieren und auf Google Spreadsheets zuzugreifen beziehungsweise Daten in ein Spreadsheet zu exportieren.
Natural Language Toolkit (NLTK)	Bietet umfassende Funktionen, um mit Korpora zu arbeiten, Texte zu kategorisieren, linguistische Strukturen zu analysieren und natürlichsprachige Dokumente für die Analyse aufzubereiten. Bei der Entwicklung wurde unter anderem Wert auf eine intuitive Nutzung und Modularität gelegt, so dass der Nutzer ¹ einzelne Komponenten nutzen kann, ohne das komplette Toolkit verstehen zu müssen (Bird et al. 2009, S. XIV - XV). Vor der erstmaligen Verwendung müssen die einzelnen Module des Toolkits mit dem Befehl <code>nltk.download(„all“)</code> heruntergeladen werden.
Requests	Bibliothek zur einfachen Umsetzung von HTTP-Anfragen (Reitz 2016). Requests wird in dieser Arbeit für die Datensammlung durch Webscraping genutzt.

Tabelle 1: Genutzte Python Bibliotheken aus der Anaconda Distribution

Neben den Bibliotheken der Anaconda Distribution wurden zudem Python Standardbibliotheken genutzt, wie zum Beispiel *re* zum Auffinden von Mustern in den Daten mit Regular Expressions (Regex). Diese werden in den jeweiligen Notebooks zu Beginn kurz erläutert.

Einzig zum Durchführen des Part-of-Speech (POS) Taggings ist es notwendig, eine externe Bibliothek zu verwenden. Die Installation dieser wird in Kapitel 4.3.2. beschrieben.

¹ Aus Gründen der besseren Lesbarkeit wird auf die gleichzeitige Verwendung männlicher und weiblicher Sprachformen verzichtet. Sämtliche Personenbezeichnungen gelten gleichwohl für beiderlei Geschlecht.

Zusammenfassend kann man sagen, dass die Programmiersprache Python sich aufgrund ihrer einfach verständlichen Syntax und der flachen Lernkurve eignet, um von Programmierneinsteigern für die Durchführung eines Datenprojekts genutzt zu werden. Zusätzlich bietet Python mit der Anaconda Distribution den Werkzeugkasten, der benötigt wird, um das in dieser Arbeit beschriebene Projekt durchzuführen, ohne zuvor umfassende Installationen durchführen zu müssen.

3. Verfahrensweisen und Vorgehensmodelle

Um einen Anhaltspunkt zu erhalten, wie ein solches Datenprojekt durchgeführt werden sollte, wurden verschiedene Vorgehensmodelle und Verfahrensweisen zum Data, Text und Job Mining betrachtet.

Vorgehensmodelle werden in der Softwareentwicklung dazu genutzt, Entwicklungsprozesse in kleine, überschaubare Einheiten zu gliedern. Dadurch wird die Projektplanung und -umsetzung erleichtert und die Wahrscheinlichkeit einer erfolgreichen Durchführung erhöht (Filß 2005, S. 1).

Angelehnt an den Kriterien von Filß (2005) wurden die Vorgehensmodelle und Verfahrensweisen auf die von ihnen abgedeckten Phasen und deren Vorgehensweise untersucht und analysiert, welche Elemente sich für das Erreichen der spezifischen Zielsetzung dieser Arbeit eignen. Es wird den Fragen nachgegangen, wie der Text Mining Prozess durchgeführt wird und ob dazu eine bestimmte Software oder eine Programmiersprache wie Python oder R genutzt wird.

Es wird angenommen, dass anwendungsneutrale Modelle und Verfahren, um größtmögliche Flexibilität bei Anwendungsfall und Methode zu bieten, bei der genauen Beschreibung der Methode wenig spezifisch sind, dafür aber ein breiteres Feld an Analysemethoden abdecken als anwendungsspezifische Modelle. Daher wurden, neben den anwendungsspezifischen Modellen und Verfahren zur Analyse von Stellenbeschreibungsdaten, auch anwendungsneutrale in den Vergleich mit einbezogen. Zudem wurde auch das weiter gefasste Themenfeld Data Mining hinzugezogen.

Mit dem Vergleich der Prozessphasen soll ein Überblick gegeben werden, welche Phasen von allen Modellen und Verfahren abgedeckt werden und somit unabdingbar für einen erfolgreichen Analyseprozess sind, und welche Phasen nur von einzelnen Modellen abgedeckt werden. Diese Phasen werden noch einmal genauer betrachtet, um zu entscheiden, ob sie in die Methode mit einbezogen werden oder nicht.

In den folgenden Unterkapiteln wird zunächst die entsprechende Verfahrensweise definiert, bevor ausgewählte Modelle und Vorgehensweisen aus den Bereichen Data, Text und Job Mining vorgestellt und hingehend der oben genannten Kriterien untersucht werden. In den Kapiteln zum Data und Text Mining wurden dabei anwendungsneutrale Vorgehensmodelle ausgewählt. Die anwendungsspezifischen Modelle und Verfahren werden im Kapitel 3.3. analysiert.

3.1. Data Mining Vorgehensmodelle

Data Mining ist die weitestgehend automatische Extraktion von Zusammenhängen in Daten, durch deren Analyse neue Erkenntnisse gewonnen werden (Lackes 2016). Bei den Daten handelt es sich dabei um bereits strukturierte Daten, die aus einer Datenbank bezogen werden.

Obwohl die Stellenausschreibungen in dieser Arbeit in unstrukturierter Form vorliegen, wird dennoch ein Data Mining Modell vorgestellt, da auf dieses in der Literatur zum Text und Data Mining immer wieder verwiesen wird und viele Text Mining Vorgehensmodelle große strukturelle Ähnlichkeiten mit diesem aufweisen (Schieber, Hilbert 2014, S. 14).

Aufgrund der unterschiedlichen Datengrundlagen wird aber davon abgesehen, neben dem folgenden Modell, noch weitere Data Mining Modelle vorzustellen.

Der *Cross Industry Standard Process for Data Mining* (CRISP-DM) wurde bereits 1996 von DaimlerChrysler, SPSS und NCR als anwendungs- und industrienneutrales Modell für Data Mining Projekte entwickelt und war das erste Vorgehensmodell in diesem Bereich (Chapman et al. 2000, S. 1). Auch wenn sich das CRISP-DM Modell nicht auf den Text Mining, sondern auf den Data Mining Prozess und somit auf die Analyse von strukturierten Daten bezieht, greifen viele Text Mining Modelle auf die im CRISP-DM Modell definierten Phasen zurück und passen lediglich den Datenaufbereitungsprozess hinsichtlich der Datenstruktur an (Schieber, Hilbert 2014, S. 14). Zudem ist das CRISP-DM Modell gemäß einer Umfrage aus dem Jahr 2014 noch immer eines der meist genutzten Modelle für Datenprojekte (Piatetsky 2014).

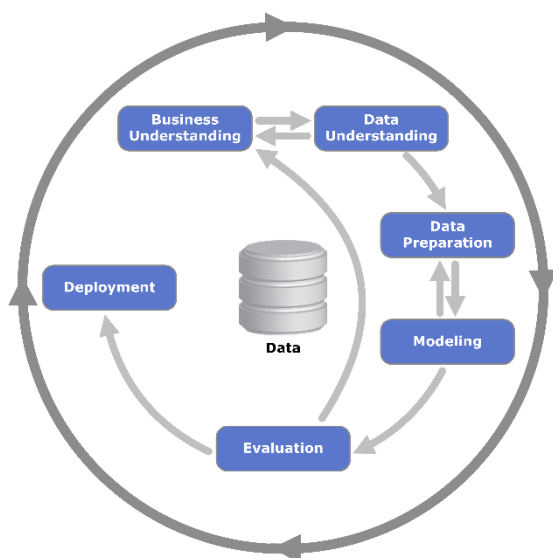


Abbildung 2: CRISP-DM Modell (CC-BY-SA, Jensen 2012)

Abbildung 2 zeigt die einzelnen Phasen des CRISP-DM Modells und deren Beziehung zueinander. Der Ablauf wird als Kreis abgebildet, um zu verdeutlichen, dass der Data Mining Prozess rekursiv ist. Aus den Analyseergebnissen werden neue Erkenntnisse gewonnen, die wiederum in ein weiteres Data Mining Verfahren übergehen können. Die rekursiven Pfeile verdeutlichen, dass es auf die Qualität des Ergebnisses der jeweiligen Phase ankommt, ob mit der nächsten Phase weitergemacht wird oder ob einzelne Schritte der vorigen Phase noch einmal angepasst und verändert werden müssen, um zu dem gewünschten Ergebnis zu kommen (Chapman et al. 2000, S. 10).

Der Ablauf eines Data Mining Prozesses gliedert sich in sechs Phasen, die in Tabelle 2 beschrieben werden.

Besonders im Hinblick auf die Zielsetzung dieser Arbeit, eine Methode zu entwickeln, die auch für Personen nachvollziehbar ist, die bisher wenig mit Daten gearbeitet haben, ist dieses Modell zu allgemein gefasst. Jedoch werden die einzelnen Phasen ausgiebig beschrieben, so dass ein guter Überblick über den generellen Ablauf eines Datenprojektes gegeben wird.

Phase	Schritte
<i>I</i>	Business Understanding: • Definition der Problemstellung • Projektplanung
<i>II</i>	Data Understanding: • Datensammlung • Datenverständnis
<i>III</i>	Data Preparation: • Aufbereitung der Rohdaten
<i>IV</i>	Modeling: • Durchführung des Analyseprozesses
<i>V</i>	Evaluation: • Überprüfung der Datenqualität
<i>VI</i>	Deployment: • Anwendung der Ergebnisse

Tabelle 2: Phasenabdeckung CRISP-DM

3.2. Text Mining Vorgehensmodelle und Verfahren

Text Mining ist der weitgehend automatisierte Prozess zur Gewinnung neuer Informationen und Erkenntnisse aus natürlicher Sprache (Hippner, Rentzmann 2006, S. 287). Dabei werden im Text Mining Verfahren und Techniken aus den Bereichen Data Mining, Natural Language Processing (NLP), Information Retrieval (IR) und Wissensmanagement angewandt, um die Daten zu analysieren (Feldman, Sanger 2007, S. 1). Teilweise wird Text Mining auch als Data Mining mit unstrukturierten Daten gesehen (Backs et al. 2014). Im Gegensatz zum Data Mining, das sich mit der Analyse strukturierter Daten, die sich zumeist in Datenbanken befinden, auseinandersetzt, befasst sich das Text Mining mit unstrukturierten Daten in Form von natürlicher Sprache. Merriam-Webster definiert natürliche Sprache unter anderem als „[...] language of ordinary speaking and writing“ (Merriam-Webster Incorporated, 2015). Aus linguistischer Sicht weist Sprache durch Syntax und Semantik zwar gewisse Strukturen auf, diese

müssen jedoch erst durch einen Datenaufbereitungsprozess in eine Form gebracht werden, die von einem automatischen Programm verarbeitet werden kann.

Für den Bereich Text Mining wurden zwei anwendungsneutrale Vorgehensmodelle beziehungsweise Analyseverfahren gewählt. Das begründet sich darin, dass die auf Stellenbeschreibungsanalyse spezifizierten Verfahren im Kapitel zu Job Mining abgehandelt werden. Des Weiteren würde es den Rahmen dieser Arbeit sprengen, auf andere Anwendungen spezifizierte Modelle und Verfahren dahingehend zu untersuchen, ob diese für die Zielsetzung der Arbeit verwertbar sind.

Die Fallstudie *Analyseverfahren im Text Mining* von Backs et al. (2014) gibt einen allgemeinen Überblick über das Vorgehen bei einer Text Mining-Analyse. Dabei liegt der Schwerpunkt auf dem Analyseprozess und dem Nutzen, der aus den einzelnen Analyseschritten gezogen werden kann.

Einleitend werden theoretischen Hintergründe und linguistische Grundlagen erörtert, bevor auf den eigentlichen Text Mining-Prozess eingegangen wird. Dieser Prozess wird anhand der einzelnen Methoden zur Datenaufbereitung und -analyse, wie zum Beispiel Segmentierung oder Clustering, beschrieben.

Phase	Schritte
<i>I</i>	• Forschungsfrage • Identifizieren von Datenquellen
<i>II</i>	• Part of Speech-Tagging • Satz- bzw. Wortsegmentierung • Speicherung
<i>III</i>	• Clusteranalyse • Musteranalyse • Hybride Methode
<i>IV</i>	Anwendung: • Web 2.0 • Competitive Intelligence • Wissenschaft und Forschung • Dokumentenverarbeitung

Tabelle 3: Phasenabdeckung Analyseverfahren von Backs et al.

Wie in Tabelle 3 ersichtlich wird, deckt das Verfahren von Backs et al. vier Phasen ab, die die Durchführung eines Text Mining Verfahrens erklären. Jedoch ist dieses Verfahren für den konkreten Anwendungsfall, den diese Arbeit abdecken möchte, nicht spezifisch genug. Die Vorüberlegungen, die in ein solches Projekt gehen, werden in Phase I nur sporadisch angedeutet und der Aspekt der Datensammlung wird gar nicht angesprochen. Dafür bietet das Analyseverfahren Einblick in unterschiedliche Analysemethoden, den nötigen theoretischen Hintergrund und potentielle Anwendungsgebiete für die Analyseergebnisse.

Das generische Vorgehensmodell von Schieber und Hilbert (2014), wurde ausgewählt, da es einer ausführlichen Analyse bestehender Vorgehensmodelle zu Grunde liegt.

Es beruht auf der Erkenntnis, dass es zum Entstehungszeitpunkt keine ausführlich beschriebenen, anwendungsneutralen Text Mining Vorgehensmodelle, die alle Phasen abdecken, gab (Schieber, Hilbert 2014, S. 23).

Nach der Auswertung der Literaturanalyse, unterteilten Schieber und Hilbert den Ablauf eines Text Mining Projektes von der Konzeptualisierung bis zur Anwendung der Ergebnisse in sechs Phasen (Tabelle 4), deren Ablauf ausführlich beschrieben wird. Des Weiteren werden in diesem Vorgehensmodell Feedbackschleifen eingebunden, um die Ergebnisse während des Prozessablaufs kontinuierlich zu überprüfen und zu verbessern (Schieber, Hilbert 2014, S. 45-46).

Phase	Schritte
I	• Analyseziele definieren • Anwendungsdomäne bestimmen
II	• Quellsysteme und -dokumente bestimmen • Eigenschaften der Dokumente feststellen • Dokumente in Arbeitsbereich übertragen
III	• Terme identifizieren • Linguistische Aufbereitung durchführen ○ Lexikalische Analyse ○ Syntaktische Analyse ○ Semantische Analyse • Technische Aufbereitung durchführen ○ Terme indexieren und gewichten ○ Terme reduzieren ○ Datenstruktur anpassen ○ Vorbereitende Analyse durchführen
IV	• Klassifikationsverfahren • Segmentierungsverfahren • Abhängigkeitsanalysen • Meinungsanalyseverfahren • Sonstige Verfahren
V	• Kennzahlen auswählen • Ergebnisse auswerten • Vorgehen überprüfen und nächste Schritte definieren
VI	• Maßnahmen ableiten und Ergebnisse anwenden

Tabelle 4: Phasenabdeckung generisches Vorgehensmodell von Schieber und Hilbert

Ähnlich wie das Verfahren von Backs et al. ist auch das Modell von Schieber und Hilbert zu allgemein, um direkt für den spezifischen Anwendungsfall genutzt werden zu können. Vor allem der methodische Bereich ist zu weit gefasst und es fehlen konkrete Beispiele, die für unerfahrene Programmierer als Leitfaden dienen können. Die spezifischen Anforderungen dieser Arbeit liegen auch nicht mehr im Anwendungsbereich, der von dem Modell von Schieber und Hilbert abgedeckt werden möchte. Die abgedeckten

Phasen bieten allerdings ein solides Grundgerüst, das auf den Anwendungsfall dieser Arbeit übertragen werden kann.

3.3. Job Mining Verfahren

Unter dem Begriff *Job Mining* wird die automatische Analyse von Stellenausschreibungen verstanden (Bensberg, Buscher 2016).

Da die vorigen Modelle zu allgemein gefasst waren, werden im Folgenden zwei konkrete Verfahren vorgestellt, die sich explizit auf die Analyse von Stellenbeschreibungen beziehen. Da es auf dieser Ebene kaum Literatur gibt, die den Fokus auf das methodische Vorgehen legt, wurden zudem auch Stellenausschreibungsanalysen in Betracht gezogen, bei denen die Vorgehensweise ausführlicher beschrieben wird. Bei der Auswahl der Verfahren wurde, aufgrund der Tatsache, dass sich technologische Verfahren schnell weiterentwickeln, darauf geachtet, dass diese in den letzten fünf Jahren publiziert wurden. Zudem muss sich ein signifikanter Anteil der Publikation der Vorgehensweise widmen, damit die genutzte Methode analysiert werden kann.

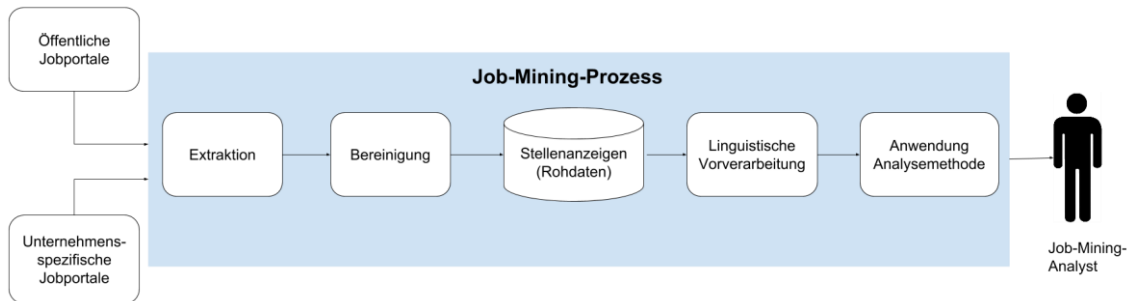


Abbildung 3: Job-Mining-Prozess im Überblick (nach Bensberg, Buscher 2016, S. 3)

Abbildung 3 beschreibt das typische Verfahren einer Job Mining Analyse, das Bensberg und Buscher in ihrem Aufsatz *Job Mining als Analyseinstrument für das Human-Resource-Management* (2016) beschrieben haben. Dieser Aufsatz wurde gewählt, da der Schwerpunkt auf der Methodik liegt und nicht auf der Anwendung der Analyseergebnisse. Tabelle 5 zeigt noch einmal die genaueren Schritte, die in den einzelnen Phasen aus Abbildung 3 beschrieben werden, auf. Dabei wird der Vorgehensprozess, beginnend bei der Datensammlung, in vier Phasen unterteilt.

Phase	Schritte
I	Identifikation von Datenquellen
II	Extraktion und Bereinigung: • Datensammlung • Bereinigung • Anreicherung mit Metadaten
III	Linguistische Vorverarbeitung und Analyse: • Vorverarbeitung ○ Tokenizing ○ Stemming ○ POS-Tagging ○ Kompositazerlegung • Analyse ○ Frequenzanalysen ○ Kookurrenzanalysen ○ Mustersuche (Nominalphrasen) ○ Clustering ○ Klassifikation
IV	Anwendung: • Kompetenzanalyse • Wettbewerbsanalyse

Tabelle 5: Phasenabdeckung Job Mining Analyse nach Bensberg und Buscher (2016)

Phase II (Extraktion und Bereinigung) und Phase III (Linguistische Vorverarbeitung der Analyse) werden dabei schwerpunktmäßig behandelt und die gängigen Schritte vorgestellt. Dabei werden jeweils die möglichen Methoden beschrieben ohne auf das genaue Vorgehen oder das Werkzeug zur Durchführung einzugehen. Das hat zum Vorteil, dass die Phasen auf verschiedene Methoden der Durchführung übertragbar sind, allerdings muss das individuelle Verfahren für jedes Werkzeug erst noch ermittelt werden.

Des Weiteren werden sowohl die konzeptionellen Überlegungen, die in so ein Projekt fließen, sowie auch die Qualitätskontrolle der Ergebnisse außen vor gelassen.

Yang et al. nutzten für ihre Analyse der Stellenausschreibungen der American Library Association einen inhaltsanalytischen Ansatz, der auf einem Codebuch beruht (2016, S. 4). Das Codebuch basiert wiederum auf den von der American Library Association festgelegten Kernkompetenzen im Bibliothekswesen und einem zuvor erstellten Codebuch zu beruflichen Kompetenzen (Yang et al. 2016, S. 6).

Phase	Schritte
<i>I</i>	Datensammlung und Auswahl
<i>II</i>	Vorverarbeitung: • Extraktion der Kontexteinheit • Unterteilen der Kontexteinheiten in Record Units • Record Units in Worteinheiten unterteilen • Stoppwortentfernung und Stemming
<i>III</i>	Erstellung Codebuch: • Initiale Erstellung von Codebuch und Classifier • Update auf Grundlage von Clustern im Text
<i>IV</i>	Datenkodierung: • Kodierung mit regelbasiertem Classifier • Extraktion neuer Cluster aus unannotiertem Text
<i>V</i>	Auswertung der Ergebnisse

Tabelle 6: Phasenabdeckung Yang et al.

Die Phasenabdeckung in Tabelle 6 macht deutlich, dass sich der methodische Ansatz von Yang et al. auf die Schritte von der Datensammlung bis hin zu der Kodierung der Analyseeinheiten bezieht. Allerdings liegt der Schwerpunkt der Publikation auf der Auswertung der Analyseergebnisse, so dass dieses Ergebnis zu erwarten war.

3.4. Fazit

Die untersuchten Modelle und Verfahren aus dem Data und Text Mining waren zu weit gefasst, um sie für den konkreten Anwendungsfall nutzen zu können. Die spezifischen Modelle und Verfahren aus dem Job Mining setzten den Schwerpunkt entweder auf die

Auswertung der Ergebnisse oder waren ebenfalls zu allgemein gefasst. Das methodische Vorgehen wurde zwar ebenfalls angesprochen, jedoch nicht in einer Tiefe, die für das Erreichen der Zielsetzung dieser Arbeit nötig wäre. Die Job Mining Verfahren, die einen größeren Schwerpunkt auf die Methodik legten, waren zwar spezifischer als die Data und Text Mining Modelle, jedoch ebenfalls nicht spezifisch genug. Bei fast keinem Modell oder Verfahren wurde die Planung des Projekts intensiver behandelt. Den Erfahrungen nach, die bei der Durchführung des Beispiels dieser Arbeit gemacht wurden, ist diese Phase aber besonders für in dem Bereich unerfahrene Personen ein elementarer Bestandteil, der nicht außer Acht gelassen werden sollte. Ebenso wurde in keinem der vorgestellten Modelle und Verfahren die genaue Durchführung der Methode behandelt. Das lässt sich mit der individuellen Zielsetzung und Zielgruppe der jeweiligen Publikationen erklären, die sich an ein Fachpublikum wenden und gleichzeitig das Werkzeug, das zur Durchführung genutzt wird, wie zum Beispiel die Programmiersprache Python oder R, offen halten wollen. Tabelle 7 fasst noch einmal die abgedeckten Phasen aller vorgestellten Modelle und Verfahren zusammen, wobei jede Phase in unterschiedlicher Intensität abgehandelt wurde.

Modell/ Verfahren	Vorüberle- gungen	Daten- sammlung	Datenver- arbeitung	Daten- analyse	Auswer- tung	Anwendung
CRISP-DM	X	X	X	X	X	X
Backs et al.	X		X	X		X
Schieber, Hilbert	X	X	X	X	X	X
Bensberg, Buscher	X	X	X	X		X
Yang et al.		X	X	X	X	

Tabelle 7: Zusammenfassung Phasenabdeckung der Modelle/Verfahren

Auch wenn von den betrachteten Modellen und Verfahren keines komplett für das Erreichen der Zielsetzung dieser Arbeit genutzt werden kann, konnten aus allen vorgestellten Modellen und Verfahren Elemente in den Prozessablauf der zu entwickelnden Methode einfließen. Die Unterteilung der Phasen wurde auf Grundlage

des CRISP-DM-Modells und des generischen Vorgehensmodells von Schieber und Hilbert vorgenommen, Backs et al. lieferten Input zu den linguistischen Grundlagen und den einzelnen Schritten in den jeweiligen Phasen, Bensberg und Buscher spannten den Bogen zu dem Datentyp „Stellenausschreibung“ und Yang et al. lieferten die Grundlage für den Analyseprozess durch ein Kategoriensystem.

Aufgrund des Vergleichs wurden folgende Phasen für die Methode dieser Arbeit definiert: Konzeptualisierung, Datensammlung, Datenaufbereitung und Datenanalyse. Die eigentlich noch anschließenden Phasen der Auswertung und Anwendung wurden nicht mit in die Methode integriert, da sich diese Arbeit lediglich mit dem Identifizieren der Kompetenzen befasst. Die Phasen werden im Einzelnen im nächsten Kapitel vorgestellt.

4. Phasen der Methode

Im Folgenden wird die auf Basis der im vorigen Kapitel beschriebenen theoretischen Grundlagen und der analysierten Vorgehensmodelle deduktiv entwickelte Methode zur Identifizierung von Kompetenzen in Stellenbeschreibungen vorgestellt.

Die einzelnen Schritte in den Phasen wurden am Beispiel von Stellenausschreibungen aus dem Berufsfeld *Bibliotheks- und Informationsmanagement* durchgeführt. Der dafür beispielhaft entwickelte Code wurde mit Python 3.5.2 und der Anaconda Distribution in der Version 4.0 mit der interaktiven Entwicklungsumgebung Jupyter Notebook erstellt.

Das jeweilige Vorgehen in den einzelnen Phasen und der dazugehörige Output hängt dabei von der eigenen, in Phase I definierten Zielsetzung ab.

Die Methode ist darauf ausgelegt, sich von zwei Seiten an die in den Stellenausschreibungen genannten Anforderungen anzunähern, zum einen durch das Entfernen von Wörtern, die keine Bedeutung transportieren, zum anderen durch das in einer Mischform aus induktivem und deduktivem Vorgehen entstandenen Kategoriensystem, mit dem die Fähigkeiten in der Stellenausschreibung identifiziert und einer Kompetenz zugeordnet werden kann.

Die Kapitel sind so aufgebaut, dass zu jeder Phase zunächst in einer Infobox gezeigt wird, welche Datenelemente Grundlage der jeweiligen Phase sind und welche Elemente am Ende der entsprechenden Phase herauskommen. Daraufhin folgt eine Erklärung und Beschreibung des theoretischen Hintergrunds und des Vorgehens in jedem zu der Phase gehörenden Schritt, bevor anschließend der technische Ablauf anhand von Beispielen aufgezeigt wird. Abschließend wird jede Phase in einer Grafik noch einmal zusammengefasst.

4.1. Phase I: Konzeptualisierung

Für eine erfolgreiche Stellenanzeigenanalyse muss der Prozessablauf geplant und konzeptioniert werden, bevor die ersten Daten gesammelt werden. Dazu sind verschiedene Vorüberlegungen und -arbeiten nötig, die in diesem Kapitel näher beleuchtet werden.

In den analysierten Vorgehensmodellen wurde das Datenverständnis als separate Phase betrachtet. In dieser Arbeit wird es in die Konzeptualisierung integriert, da es die einzelnen Facetten der Konzeptualisierung direkt beeinflusst.

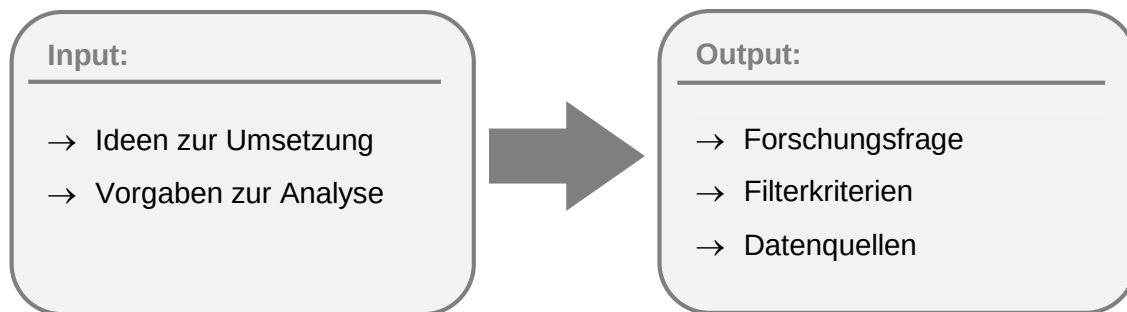


Abbildung 4: Input und Output Phase I

In dieser Phase werden eigene Ideen zusammen mit den Vorgaben zur Analyse zu einer Forschungsfrage zusammengefasst, die dem Projekt eine klare Zielsetzung gibt. Die Zielsetzung kann auch bereits durch die Aufgabenstellung oder durch einen Auftraggeber definiert sein. Anschließend wird anhand der Forschungsfrage der Rahmen des Projektes abgesteckt und Filterkriterien für die Daten festgelegt, sowie potenzielle Datenquellen evaluiert und ausgewählt (Abbildung 4).

4.1.1. Forschungsfrage

Um möglichst präzise bestimmen zu können, welche Daten benötigt und wie diese bearbeitet werden müssen, muss zunächst eine Forschungsfrage oder eine Hypothese definiert werden. Dadurch wird der Stellenanzeigenanalyse ein Rahmen und der Analyse ein Ziel gegeben. Je enger Forschungsfragen gefasst sind, desto präziser werden die Anforderungen an Datenquellen und Datenbeschaffenheit. Beschränkt sich die Forschungsfrage, zum Beispiel, auf eine bestimmte Region, muss sichergestellt werden, dass entweder die Suchergebnisse in der Datenquelle regional filterbar sind oder dass in den Stellenausschreibungen die Region angegeben ist, so dass die Stellenbeschreibungen in der gewünschten Region extrahiert werden können. Umgekehrt können die möglichen Filterkriterien auch die Forschungsfrage beeinflussen. Ist die Datenquelle bereits in der Zielsetzung vorgegeben, sind die Filterkriterien davon abhängig, welche Möglichkeiten die Datenquelle bietet.

Auch der Prozess der Datenaufbereitung ist abhängig von der Definition der Zielsetzung, da diese zum Beispiel festlegt, welche Elemente einer Stellenbeschreibung extrahiert und für den Analyseprozess aufbereitet werden. Umgekehrt steht man bei einer sehr weit gefassten Forschungsfrage vor der Herausforderung, dass eine große Menge potentieller Datenquellen zur Verfügung stünde, aus denen man geeignete Stellenbörsen herausfiltern müsste.

Die Forschungsfrage ist dabei kein starres Konstrukt. Stellt man etwa fest, dass die Forschungsfrage zu weit gefasst ist und die Datenmenge eingegrenzt werden sollte, kann diese dementsprechend angepasst werden. Falls bereits Daten gesammelt wurden, sollte man jedoch darauf achten, ob diese auch der angepassten Forschungsfrage entsprechen, ansonsten müsste man die Phase der Datensammlung erneut durchführen.

Die Forschungsfrage beeinflusst sämtliche Schritte der folgenden Phasen, da die Entscheidung, welche Schritte in der Datensammlungs-, Datenaufbereitungs- und Datenanalysephase durchgeführt werden, davon abhängen, welches Ziel mit der Analyse angestrebt wird.

4.1.2. Eingrenzung der potenziellen Datenmenge

Mit der Festlegung von Zielsetzung und Definition einer Forschungsfrage legt man den Grundstein für die Eingrenzung der Daten, die ausgewertet werden sollen. Aufgrund der großen Menge an online verfügbaren Stellenausschreibungen, ist es sinnvoll, genau zu bestimmen, nach welchen Kriterien man diese filtern möchte, um für das festgelegte Ziel die optimale Datengrundlage zu haben.

Diese Arbeit konzentriert sich ausschließlich auf Stellenausschreibungen, die auf öffentlich zugänglichen Stellenbörsen publiziert wurden. Dadurch werden Ausschreibungen, die im firmeninternen Intranet, ausschließlich auf der unternehmenseigenen Webseite oder über soziale Medien veröffentlicht werden, nicht mit einbezogen. Bei einer ähnlich eingegrenzten Analyse sollte man bedenken, dass nur ein Teil der potenziell verfügbaren Datenquellen abgedeckt wird und somit die Analyseergebnisse kein komplett umfassendes Bild abgeben.

Die Eingrenzung des Berufsfelds hat direkten Einfluss auf die Wahl der Datenquellen und den Datensammelungsprozess. Anhand des Berufsfeldes und der in dem Berufsfeld enthaltenen Berufsbezeichnungen werden die Suchbegriffe formuliert und auch die

Datenquellen ausgewählt. Für einige Berufsfelder gibt es spezifische Stellenbörsen wie zum Beispiel OpenBiblioJobs, eine Stellenbörse, auf der ausschließlich Stellenangebote von Bibliotheken, Archiven und Informationseinrichtungen veröffentlicht werden.

Aber wie grenzt man ein Berufsfeld sinnvoll ein? Einen Anhaltspunkt kann die Klassifikation der Berufe (KldB) der Bundesagentur für Arbeit geben. Die KldB gliedert offizielle Berufsbezeichnungen hierarchisch in systematische Gruppen (Bundesagentur der Arbeit 2014). Neben dem systematischen Verzeichnis der Berufsgruppen bietet die Bundesagentur für Arbeit zudem ein Berufs- und Tätigkeitsverzeichnis, das neben den aktuellen Berufs- und Tätigkeitsbezeichnungen auch Synonyme bietet, die zur Generierung von Suchbegriffen für das Sammeln der Stellenbeschreibungen in Phase II genutzt werden können. Beide Verzeichnisse sind als Excel-Datei frei zugänglich (Bundesagentur für Arbeit 2016). Des Weiteren können Alumnistudien der entsprechenden Studiengänge oder spezifische Literatur Aufschluss über das Berufsfeld geben.

Grenzt man die potentiellen Stellenbeschreibungen durch die Forschungsfrage regional ein, muss entweder die Möglichkeit bestehen, die Suchergebnisse auf den Stellenbörsen nach Regionen zu filtern oder die Daten müssen so beschaffen sein, dass der Standort des Arbeitgebers durchgehend identifiziert werden kann, so dass man bei der Datenbearbeitung die entsprechenden Stellenbeschreibungen in der gewünschten Region extrahieren kann. Wenn die geografische Einschränkung international ausgeweitet wird, muss bei der Auswahl der Datenquellen bedacht werden, dass die gängigen Stellenbörsen aus dem jeweiligen Land berücksichtigt und die Suchbegriffe in der jeweiligen Landessprache verfasst werden.

Weiterhin sollte überlegt werden, welche Art von Stellenausschreibungen in die Analyse einbezogen werden. Die meisten Stellenbörsen bieten sowohl Teil- und Vollzeitstellen als auch Angebote für studentische Hilfskräfte, Praktika und Abschlussarbeiten. Die Anforderungen für Praktika oder Abschlussarbeiten unterscheiden sich dabei von denen für eine Festanstellung. Dieses muss bedacht werden, wenn man Anzeigen für Praktika und Abschlussarbeiten mit in die Analyse einbezieht. Je nach Datenmenge und Anteil an Anzeigen für Praktika und Abschlussarbeiten kann dadurch das Ergebnis der Analyse beeinflusst werden.

4.1.3. Eigenschaften der Datenquellen

Bei der Auswahl der Datenquellen spielen die bereits genannten Kriterien eine entscheidende Rolle. Je mehr Kriterien festgelegt werden, die die Datenmenge eingrenzen, desto näher müssen die Stellenbörse betrachtet werden. Die großen Stellenbörsen, wie zum Beispiel Xing, bieten zumeist umfangreiche Filtermöglichkeiten. Andere Stellenbörsen, wie OpenBiblioJobs oder indeed.com, bieten nur wenige bis gar keine Möglichkeiten die Suchergebnisse zu filtern, so dass man, wenn diese Stellenbörsen in der Analyse einbezogen werden sollen, die jeweiligen Filterkriterien mühevoll aus den Stellenbeschreibungen auslesen müsste, um das Ergebnis nicht zu verfälschen.

Des Weiteren sollte man die Stellenbörsen dahingehend untersuchen, wie aufwändig es ist, die Stellenanzeigen zu extrahieren und zu speichern. Viele der großen Stellenbörsen hosten die Anzeigen direkt auf der eigenen Webseite und bieten diese zumeist im HTML-Format an. Andere Stellenbörsen mischen Anzeigen in HTML- und PDF-Format. Beide Formate müssen im Datenaufbereitungsprozess unterschiedlich behandelt werden. Wieder andere Stellenbörsen bieten nur einen kurzen Teaser und einen Link zu der vollständigen Stellenbeschreibung bei einem externen Anbieter oder direkt auf der Unternehmenshomepage. Für die Beispiele dieser Arbeit wurden lediglich Stellenbörsen verwendet, die Stellenanzeigen selber hosten und die Anzeigen in HTML-Format anbieten. Tabelle 8 bietet einen Überblick über eine Auswahl an in Frage kommenden Stellenbörsen.

Name	Format der Stellenanzeigen	Stellenanzeigen selber gehostet?	Anmerkungen
Stepstone	HTML	Ja	-
Zeit	HTML	Ja	geringe Treffermenge
openBiblioJobs	HTML und PDF	Nein	keine Suchmöglichkeiten
indeed.com	HTML	Nein	kaum Filtermöglichkeiten
Monster	HTML	Ja	viel Ballast bei den Ergebnissen
Xing	HTML	Ja	-

Tabelle 8: Vergleich Stellenbörsen

Es eignet sich zudem nicht jede Stellenbörse für jedes Berufsfeld. Die großen Stellenbörsen decken jeweils ein breites Spektrum an Berufen und Positionen ab, andere Stellenbörsen fokussieren sich dagegen auf bestimmte Branchen oder Positionen. Die Stellenbörse der Zeit veröffentlicht zum Beispiel tendenziell Stellenausschreibungen für akademische Positionen.

Für die Analyse von Stellenausschreibungen ist es essentiell, sich mit deren Struktur auseinander zu setzen.

Die meisten Stellenbörsen geben keine oder nur wenig strukturelle Elemente vor, so dass die Anbieter dahingehend ihre Stellenausschreibungen frei gestalten können. Nach Sichtung von Stellenausschreibungen auf unterschiedlichen Stellenbörsen sind Elemente, die von den meisten Ausschreibungen abgedeckt werden:

- 1) Titel der Ausschreibung inkl. Berufsbezeichnung
- 2) Beschreibung des Unternehmens
- 3) Auflistung des Tätigkeitfeldes
- 4) Auflistung der Anforderungen
- 5) Angebot des Unternehmens
- 6) Formalia (Bewerbungsschluss, Kontakt, gegebenenfalls Gleichstellungshinweis, etc.)

Abbildung 5 zeigt eine Stellenbeschreibung, die, mit Ausnahme einer Beschreibung des Unternehmens / der Einrichtung, sämtliche Merkmale abdeckt. Die Beschreibung des Tätigkeitsfeldes sowie die Anforderungen, die vom Bewerber erwartet werden, sind in Listen strukturiert.

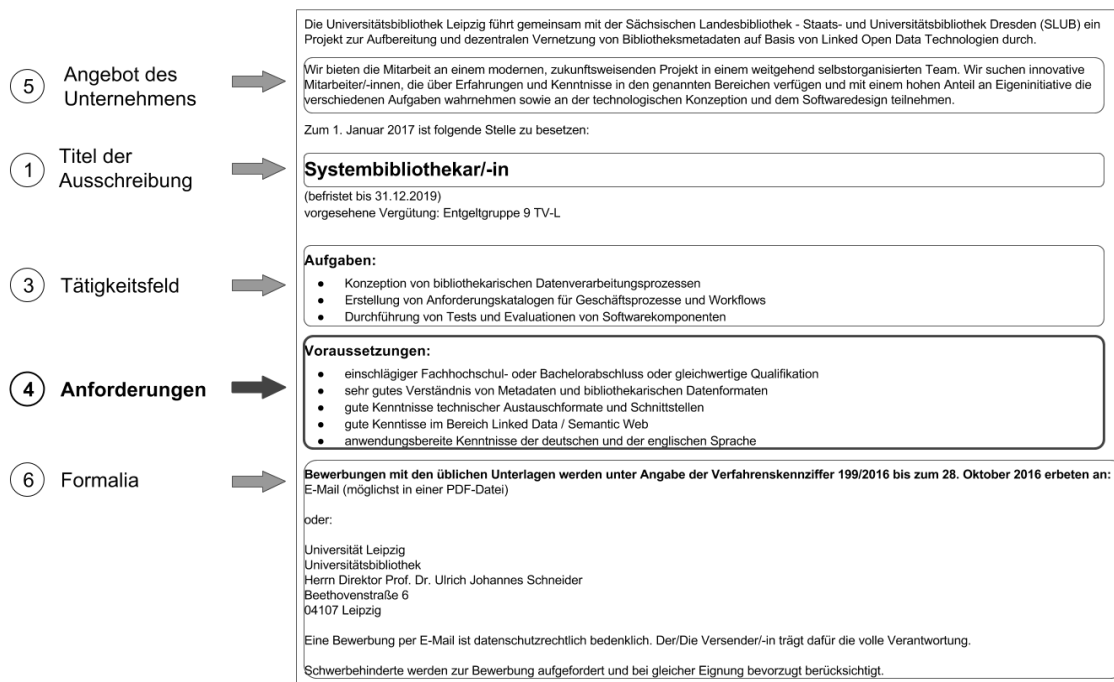


Abbildung 5: Aufbau einer Stellenausschreibung mit Strukturelementen²

Für das Extrahieren der Anforderungen ist die HTML-Struktur der Stellenausschreibung relevant. Listing 1 ist eine vereinfachte Darstellung der HTML-Struktur der Stellenbeschreibung in Abbildung 5. Für die späteren Phasen des Prozessablaufs ist es wichtig, sich diese Struktur vor Augen zu führen, da auf diese Art und Weise der eigentlich unstrukturierte Text eine Struktur erhält, die man für die Identifizierung der Kompetenzen nutzen kann. In diesem konkreten Beispiel ist das Angebot des Unternehmens in einem div-Container positioniert, der Titel in einem h2-Tag und die Tätigkeitsgebiete und Anforderungen in Listen strukturiert, bei denen sich jede Tätigkeit oder Anforderung in einem Listenelement befindet. Eingeleitet werden die Listen jeweils von einem h3-Tag. Durch CSS-Selektoren, wie zum Beispiel IDs oder Klassen, werden die einzelnen Bereiche, die durch einen div-Container abgegrenzt werden,

² Quelle der Stellenbeschreibung: stepstone.de [Zugriff am: 26.10.2016] Verfügbar unter: <http://www.stepstone.de/stellenangebote--Systembibliothekar-in-Leipzig-Universitaetsbibliothek-Leipzig--3994170-inline.html?suid=a12cb502-97f8-4db8-81c5-935ca6a69cff&ssaPOP=2&ssaPOR=2&pop=2&por=2&rpp=25>

identifizierbar. Die Selektoren werden in Phase III genutzt, um die Stellenbeschreibung von den übrigen Elementen der Webseite, wie Navigationsleisten, zu entfernen.

```
<div id="company-intro">
  Angebot des Unternehmens
  <h2>Titel der Stellenausschreibung</h2>
</div>
<div id="job-tasks">
  <h3>Tätigkeitsfeld:</h3>
  <ul>
    <li> Tätigkeit 1</li>
    <li> Tätigkeit 2</li>
    <li> Tätigkeit 3</li>
  </ul>
</div>
<div id="job-requim">
  <h3>Anforderungen:</h3>
  <ul>
    <li> Anforderung 1</li>
    <li> Anforderung 2</li>
    <li> Anforderung 3</li>
    <li> Anforderung 4</li>
    <li> Anforderung 5</li>
  </ul>
  <strong> Formalia</strong>
</div>
```

Listing 1: HTML-Struktur des Beispiels aus Abbildung 5 (vereinfachte Darstellung)

Würden sämtliche Stellenbeschreibungen der gleichen Struktur folgen, könnten die Anforderungen relativ einfach extrahiert werden. Die Herausforderung liegt jedoch darin, dass sich die Stellenbeschreibungen, auch wenn sämtliche genannten Elemente einbezogen werden, dennoch strukturell unterscheiden können. In vielen Stellenbeschreibungen werden der Tätigkeitsbereich und die Anforderungen, wie in dem Beispiel aus Abbildung 5, in Form einer Liste dargestellt. Es gibt allerdings auch einen großen Teil an Stellenbeschreibungen, in denen diese Bereiche in einem Fließtext genannt werden (Abbildung 6).

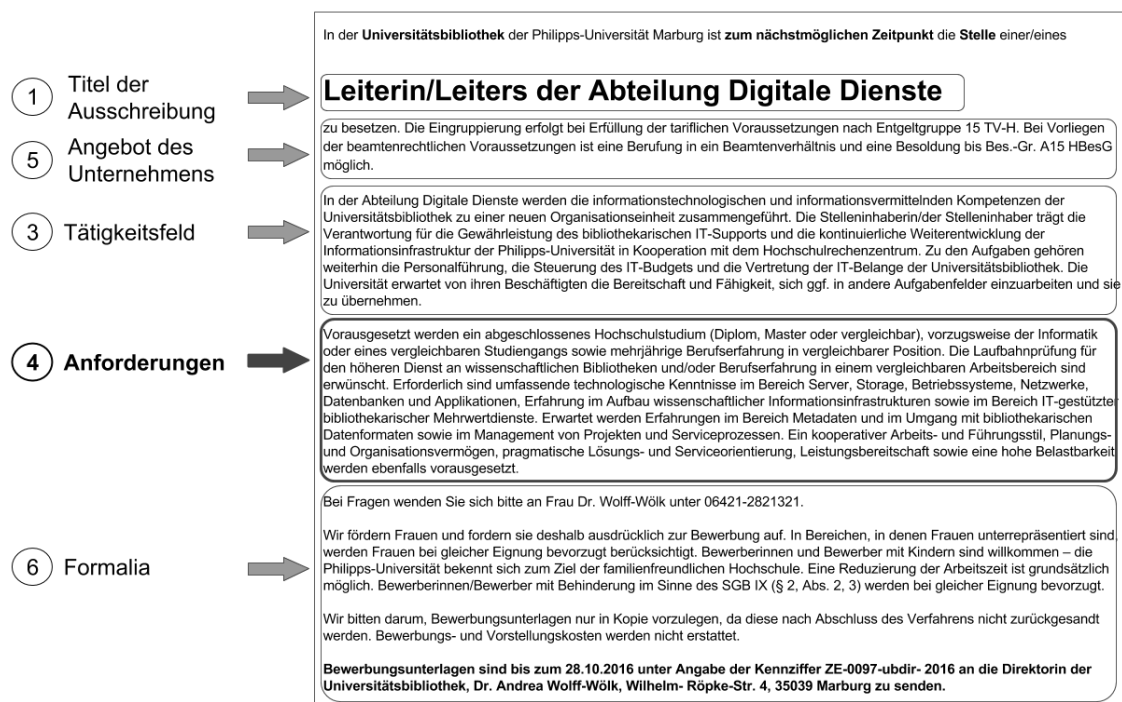


Abbildung 6: Aufbau einer Stellenanzeige ohne Strukturelemente³

Im Vergleich zu der Stellenbeschreibung in Abbildung 6 fällt auf, dass sich die Reihenfolge der Elemente zwar leicht unterscheidet, die Anforderungen jedoch wieder zwischen der Beschreibung des Tätigkeitsfeldes und der Formalia befindet.

Betrachtet man allerdings die HTML-Struktur (Listing 2) fällt auf, dass diese Stellenbeschreibung weit weniger Strukturelemente enthält. Die komplette Stellenbeschreibung befindet sich in einem einzelnen div-Container. Lediglich der Titel ist durch einen weiteren div-Container und dessen CSS-Selektor individuell selektierbar.

³ Quelle der Stellenausschreibung: Zeit Online. [Zugriff am: 27.10.2016] Verfügbar unter: http://jobs.zeit.de/jobs/marburg/leiter_m_w_der_abteilung_digitale_dienste_133465.html


```

<div class="jw-anzeige_div jw-text">
  <div class="jw-stellentitel">
    <h1>Titel der Ausschreibung</h1>
  </div>
  Angebot des Arbeitgebers<br>
  <br>
  Tätigkeitsfeld<br>
  <br>
  Anforderungen<br>
  <br>
  Formalia<br>
  <br>
</div>

```

Listing 2: HTML-Struktur des Beispiels aus Abbildung 6 (vereinfachte Darstellung)

4.1.4. Anwendungsbeispiel

Für die in dieser Arbeit beispielhaft durchgeführte Analyse wurde das Berufsfeld des Studiengangs *Bibliotheks- und Informationsmanagement* gewählt. Die einzelnen Berufsbezeichnungen in Tabelle 8 wurden zum einen aus klassischen Berufen des Berufsfeldes (Archivar, Bibliothekar und Dokumentar) zusammengesetzt, zum anderen aus Berufsbezeichnungen aus der Informationsbranche, die auf der Arbeit von Lamparter (2015, S. 8-12) beruhen. Zuletzt wurden noch drei neuere Berufsbezeichnungen aus der Bibliotheksbranche hinzugefügt, die sich durch den digitalen Wandel in den letzten Jahren gebildet haben. Diese Begriffe stammen aus einer Konferenzveröffentlichung des Deutschen Bibliothekartages 2016 (Seeliger et al. 2016).

Berufsbezeichnung
Archivar
Bibliothekar
Dokumentar
Informationsdesigner
Informationsmanager
Informationsspezialist
Informationswissenschaftler
Systembibliothekar
IT Bibliothekar
Technischer Bibliothekar

Tabelle 9: Berufsbezeichnungen Bibliotheks- und Informationsmanagement

Die Berufsbezeichnungen sind exemplarisch ausgewählt und decken nicht das komplette Berufsbild ab. Allerdings lassen sie einige potentielle Forschungsfragen zu. Man könnte zum Beispiel die Kompetenzanforderungen der traditionellen mit den neueren Berufen vergleichen oder die Bibliotheksbranche mit der Informationsbranche. Da die Anwendungsbeispiele in den folgenden Kapiteln allerdings möglichst allgemein gehalten werden sollen, um mit wenigen Anpassungen auf unterschiedliche Anwendungsfälle anwendbar zu sein, wurde die Forschungsfrage für das Beispiel ebenfalls sehr weit gefasst. Aus eben diesem Grund wurden auch keinerlei Filterkriterien zur Eingrenzung der potenziellen Datenmenge festgelegt. Mit der exemplarischen Analyse wird der Frage nachgegangen, welche Kompetenzen in dem durch die Berufsbezeichnungen abgegrenztem Berufsfeld gefordert werden.

Bei den Datenquellen wurden aufgrund des Vergleiches in Tabelle 8 die Stellenbörsen von Xing, Zeit Online und Stepstone ausgewählt. OpenBiblioJobs bietet zwar die präzisesten Stellenausschreibungen, jedoch würde es den Rahmen dieser Arbeit sprengen, eine Stellenbörse mit einzubeziehen, die lediglich auf externe Quellen verlinkt und zudem HTML und PDF Dokumente mischt.

4.1.5. Zusammenfassung

Viele dieser Schritte können auf dem ersten Blick trivial erscheinen, jedoch zahlt sich eine sorgfältige Planung im Vorfeld später aus, da die Ergebnisse eher der für das Erreichen der eigenen Zielsetzung nötigen Datenqualität entsprechen und somit zeit- und ressourcenaufwändige Anpassungen im Nachhinein vermieden werden. In Abbildung 7 werden die wichtigsten Schritte dieser Phase noch einmal zusammengefasst. Die Forschungsfrage steckt den Rahmen des Projektes ab und legt die Zielsetzung fest. Die Filtermöglichkeiten haben sowohl Einfluss auf das Konzept selbst als auch auf die Forschungsfrage. Teilweise werden durch die Forschungsfrage aber auch schon Filtermöglichkeiten vorgegeben. Das Datenverständnis, beziehungsweise die Kenntnis der Datenstrukturen, übt direkten Einfluss auf sämtliche Bereiche dieser Phase aus. Ohne dieses können weder geeignete Datenquellen ausgewählt, noch die Filtermöglichkeiten, die von den Quellen gegeben werden, identifiziert werden. Auch das Formulieren einer konkreten Forschungsfrage ist ohne Kenntnis der Datenstruktur nicht möglich.

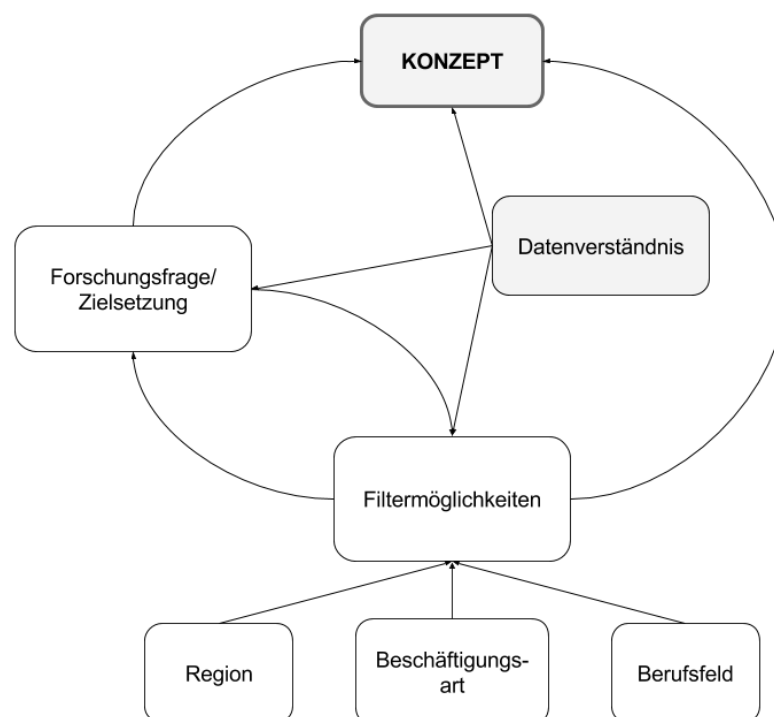


Abbildung 7: Konzeptualisierungsphase

Neben den in diesem Kapitel genannten Schritten zur Konzeptualisierung des Datenprojektes, sollte ebenfalls Organisatorisches, wie zum Beispiel die Ordnerstruktur zur Speicherung der Stellenanzeigen oder die einheitliche Benennung der Dateien, beschlossen werden. Vor allem wenn kollaborativ an dem Projekt gearbeitet wird, sollte jeder dahingehend der gleichen Struktur folgen, da es andernfalls zu Konflikten kommen kann.

4.2. Phase II: Datensammlung

Steht das Konzept fest und sind geeignete Stellenbörsen ausgewählt, werden im nächsten Schritt die Daten gesammelt und gespeichert (Abbildung 8). Dafür müssen die Datenquellen noch einmal genauer angeschaut werden, um die Suchanfrage zu formulieren, bevor die Daten gespeichert werden können. Dieses geschieht mit einem Verfahren namens Web Scraping.

Beim Speichern der Daten hat es sich als bewährt herausgestellt, zunächst einmal die kompletten HTML-Seiten der Suchübersicht sowie von den Stellenbeschreibungen zu speichern. Auf diese Art und Weise ist es möglich, sich den Code genauestens in einem Editor anzugucken und zu entscheiden, welche zusätzlichen Daten in den Metadaten erfasst werden sollen.

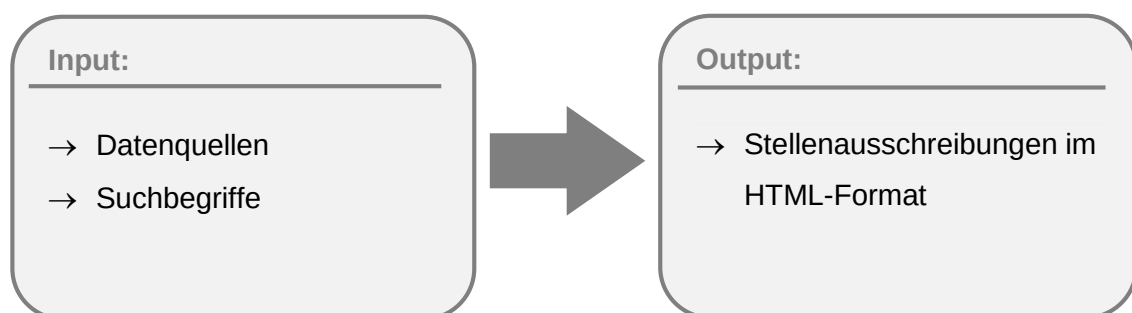


Abbildung 8: Input und Output Phase II – Datensammlung

4.2.1. Web Scraping

Als Web oder Screen Scraping wird die automatische Extraktion von Informationen auf einer Webseite bezeichnet, die meist durch das Senden eines GET-Requests an einen Server geschieht (Mitchell 2015, S. VIII). Diese Information wird anschließend

gespeichert, weiterverarbeitet und gegebenenfalls wieder veröffentlicht. Dadurch ist es möglich, große Datenmengen zu erhalten und zu analysieren.

Genutzt wird diese Technik zum Beispiel von Google zur Anreicherung von Einträgen auf GoogleMaps (onPage.org GmbH 2016).

Im Zusammenhang mit dem Web Scraping wird auch immer wieder die Frage der Legalität aufgegriffen. Es gibt kein Gesetz, das Web Scraping per se verbietet. Der Bundesgerichtshof (BGH) urteilte im Jahr 2014 im Fall der Klage gegen ein Flugpreisvergleichsportal, das Flugdaten mit Web Scraping bezogen und veröffentlicht hat, im Sinne des Angeklagten und wies die Klage ab (BGH. Urteil vom 30. April 2014. (Az. I ZR 224/12))⁴. Absatz 55 des oben genannten Urteils besagt, dass ein Betreiber einer öffentlich zugänglichen Webseite

„[...] im Allgemeininteresse an der Funktionsfähigkeit des Internets daran festhalten lassen muss, dass die von ihm eingestellten Informationen durch übliche Suchdienste in einem automatisierten Verfahren aufgefunden und dem Nutzer entsprechend seinen Suchbedürfnissen aufbereitet zur Verfügung gestellt werden.“

Die Einschränkung auf öffentlich zugängliche Daten ist dabei eines der Dinge, die man beim Web Scraping beachten sollte.

Besonders bei einer geplanten Veröffentlichung der Ergebnisse ist es zudem wichtig, das Urheber-, Leistungs- und Datenschutzrecht zu beachten.

Der Betreiber der Webseite kann in den allgemeinen Geschäftsbedingungen oder in den Nutzungsbedingungen festlegen, ob er Web Scraping gestattet oder nicht. Daher sollten diese vor Beginn des Scraping-Prozesses gelesen werden. Einen Aufschluss darüber, für welche Teile der Domain der Betreiber das Web Scraping nicht gestattet, bietet die robots.txt-Datei. In dieser kann der Betreiber festlegen, welche Verzeichnisse der Domain für welche User-Agents abrufbar sind (Abbildung 9). Diese ist einsehbar wenn man *robots.txt* als Pfad hinter dem Domain-Namen einfügt.

⁴ Aufgerufen am: 28.10.2016. Verfügbar unter: <http://openjur.de/u/691648.html>

```
User-agent: *
Disallow: /cgi-bin/
Disallow: /images/
Disallow: /images
Disallow: /aboutus/cv.cfm
Disallow: /document.all.id
Disallow: /admin/
Disallow: /controllers/
Disallow: /cfcomponents/
Disallow: /jobagent/
Disallow: /alteon/
Disallow: /error/
Disallow: /work/
Disallow: /offers/offer_detail.cfm?click=yes&nofooter=1&ID=76637
Disallow: /offers/viewTemplatefr.cfm?id=96827&Compid=4990&lang=DE&parentid=96827&admin=0
Disallow: /*listing_footer.cfm$
Disallow: /listingprint/*

User-agent: BingPreview
Crawl-Delay: 30
```

Abbildung 9: Robots.txt Datei von Stepstone

Um den Server nicht unnötig mit Traffic zu belasten, gehört es ebenfalls zu einem verantwortungsvollen Verhalten eine Pause einzufügen, nachdem ein Dokument gescraped wurde. Durch ein Timeout wird die Ausführung des Skriptes für ein paar Sekunden gestoppt, bevor das nächste Dokument gescraped wird.

Hat man geeignete Stellenbörsen ausgewählt, ist, wie auch beim manuellen Durchsuchen einer Stellenbörse, der nächste Schritt, um zur Stellenbeschreibung zu gelangen, das Formulieren einer Suchanfrage. Dazu wird, wie auch bei der manuellen Suche, ein Suchbegriff benötigt sowie die entsprechenden Filterkriterien.

Die automatische Suche funktioniert über die URL der jeweiligen Stellenbörse. Hierzu ist es nötig, genau zu untersuchen, wie sich die URL verändert, wenn ein Suchbegriff eingegeben oder das Ergebnis gefiltert wird.

Eine URL besteht aus mehreren Komponenten: dem Protokoll, das bei Online-Stellenbeschreibungen in der Regel das Hypertext Transfer Protocol (HTTP) ist, und der Domain. Weiterhin kann nach der Domain noch ein Verzeichnispfad und/oder ein Query-String folgen. Letzterer wird durch ein Fragezeichen eingeleitet, dem die Suchparameter folgen.

Schritt 1 - Formulieren einer Suchanfrage:

Bei jeder Stellenbörse können die Parameter des Query-Strings unterschiedlich aussehen. Daher muss auch jede Stellenbörse im Vorfeld individuell dahingehend betrachtet werden, wie sich der Query-String verhält, wenn eine Suche ausgeführt oder die Treffermenge durch Filterkriterien eingeschränkt wird.

Ein Beispiel dafür wäre der Suchbegriff, mit dem nach den Stellenausschreibungen gesucht wird. Einige Stellenbörsen, wie zum Beispiel Xing, haben für viele Berufe festgelegte Begriffe, mit denen sich das beste Suchergebnis erzielen lässt. Zusätzlich zu den feststehenden Begriffen sollten auf jeder Stellenbörse für jeden Beruf, verschiedene Suchbegriffe ausprobiert werden. Es empfiehlt sich die Suchbegriffe, mit denen die präzisesten Suchergebnisse und der beste Recall erreicht wurden, in einem Spreadsheet oder in einer Excel-Tabelle zu speichern, da diese Datenformate später einfach mit Python ausgelesen werden können und somit nicht noch einmal manuell in das entsprechende Skript eingetragen werden müssen. Um die Methode möglichst kollaborativ zu gestalten, sind die Skripte dieser Arbeit auf Google Spreadsheets ausgelegt.

Die weiteren Parameter ergeben sich aus den in Phase I festgelegten Filterkriterien. In dieser Arbeit wurde die Suche nicht weiter eingeschränkt, die Suchergebnisse lediglich nach Datum sortiert.

Schritt 2 - Scrapen der Suchübersicht:

In Python stehen für das Web Scraping verschiedene Bibliotheken zur Verfügung. In dieser Arbeit wurde mit der Requests-Bibliothek gearbeitet

Die Wahl begründet sich darin, dass Requests sowohl die in Kapitel 1.2. genannte Bedingung erfüllt und Teil der Bibliotheken in der Anaconda Distribution ist als auch eine einfach nachvollziehbare und lesbare Möglichkeit bietet, per HTTP-Request Daten von einer Webseite zu speichern.

Im Gegensatz zu der urllib2-Bibliothek, kann man in der Requests-Bibliothek die Parameter des Query-Strings in einem Dictionary der Basis-URL zuweisen und muss diese nicht vorher zu einer URL zusammenfügen.

Aufgrund der Möglichkeit kollaborativ zu arbeiten wurden die Suchbegriffe in einem Spreadsheet gesammelt und mit Pandas als Data Frame in das Skript geladen. Ein Data Frame ist eine Datentabelle mit gleichlangen Spalten und Zeilen und bietet eine

Möglichkeit Daten in strukturierter Form zu bearbeiten und zu analysieren. In die Funktion aus Listing 3 muss man lediglich die individuelle ID aus der URL als Parameter einfügen. Auf diese Weise können Suchbegriffe für alle Berufsbezeichnungen und Stellenbörsen, die in einem Spreadsheet gespeichert sind automatisch in das Skript geladen werden und müssen nicht manuell eingetragen werden.

```
def spreadsheet(file_id):  
    url = "https://docs.google.com/spreadsheets/d/{0}/export?format=  
csv".format(file_id)  
    df = pandas.read_csv(url)  
    return(df)
```

Listing 3: Funktion, um Google Spreadsheet als Data Frame zu importieren

Dadurch, dass die Suchbegriffe in der Requests-Bibliothek als Keywords der `get`-Funktion übertragen werden (Listing 4), müssen diese in der Schreibweise nicht weiter angepasst werden. Umlaute erzeugen keine Probleme bei der Enkodierung und Begriffe, die aus mehreren Wörtern bestehen, können auch mit Leerzeichen verwendet werden.

```
def search_results(base_url, searchterms, pages):  
    header = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; WOW64;  
rv:49.0) Gecko/20100101 Firefox/49.0'}  
    pages = (requests.get(base_url, headers = header, params={'key  
words': searchterms, 'sort': 'date', 'page': page+1}) for page in  
range(pages))  
    return(pages)
```

Listing 4: Funktion zum Scrapen der Suchübersicht

Mit der abschließenden Funktion (Listing 5) wird jede Suchübersichtsseite gespeichert. Alle drei Funktionen sind dabei so allgemein gehalten, dass sie, unabhängig von Stellenbörse und Suchbegriffen, genutzt werden können.

```
def save_result(content, path, filename, extension):  
    file = ''.join([path, filename, '.', extension])  
    with open(file, 'w', encoding = 'utf-8') as f:  
        f.write(content)
```

Listing 5: Funktion zum Speichern der HTML-Dateien

Schritt 3 - Extrahieren der Stellenausschreibungs-URL und Metadaten:

Aus jeder Suchübersichtsseite wird nun der Link zu den Stellenausschreibungen extrahiert. Dazu muss die entsprechende Position im HTML-Dokument lokalisiert werden. Diese sind in einem a-Tag mit dem Attribut *href* im DOM zu finden. Da sich auf einer Suchübersichtsseite eine Vielfalt an Verlinkungen finden lässt, muss zusätzlich der entsprechende CSS-Selektor für die eindeutige Zuordnung identifiziert werden. Ist dem a-Tag ein weiteres Attribut, wie zum Beispiel eine Klasse oder eine ID, zugewiesen, die sich ausschließlich in den a-Tags mit den URLs zu den Stellenausschreibungen befindet, kann man die URL wie in Listing 6 beschrieben mit den Funktionen *find_all* und *get* der BeautifulSoup-Bibliothek extrahieren, nachdem die HTML-Datei in ein BeautifulSoup-Objekt umgewandelt wurde.

```
job_urls = soup.find_all('a', attrs={'class' : 'jobTitle'})
for link in job_urls:
    links = link.get('href')
```

Listing 6: Extrahieren der URL zur Stellenausschreibung in der Suchübersicht ZeitOnline

Befindet sich kein CSS-Selektor im a-Tag, muss man den Bereich um den a-Tag etwas weiter fassen und den Selektor des div-Containers oder des section-Tags nutzen. Listing 7 zeigt den Unterschied auf. Anstatt mit der *get*-Funktion die URL zu extrahieren, wurde dort mit so genannten Regular Expressions (Regex) gearbeitet. Die *findall*-Funktion der *re*-Bibliothek findet in diesem Fall innerhalb des div-Containers alle href-Attribute und extrahiert alles, was zwischen den Anführungszeichen des Attributwertes steht. Da die *findall*-Funktion eine Liste kreiert, in der sich jede URL in Form einer weiteren Liste befindet, wird abschließend die inneren Listen mit der *join*-Funktion entfernt. Übrig bleibt eine einfache Liste mit allen URLs.

```
job_urls = soup.find_all('div', attrs = {'class' : 'jobitem__header'})
for link in job_urls:
    links = re.findall('href=\"(.*)\">', str(link))
    links = ''.join(links)
```

Listing 7: Extrahieren der URL zur Stellenausschreibung in der Suchübersicht von Stepstone

Schritt 4 - Scrapen und Speichern der Stellenausschreibung:

Die Liste aus Schritt 4 nutzt man nun, um einen erneuten GET-Request an die URL der Stellenausschreibung zu senden und anschließend die entsprechende Stellenausschreibung zu speichern. Sollten sich lediglich die relativen URLs, also nur

der Pfad-Teil der URL, in der Liste befinden, müssen diese zuvor noch mit der Domain zusammengefügt werden (*url*-Variable in Listing 8).

Um eine Sicherungskopie zu haben, falls im späteren Prozess festgestellt wird, dass noch weitere Daten extrahiert werden müssen, und um ausprobieren zu können, welche Daten wie extrahiert werden können, ohne den Server des Jobportals unnötig durch ständiges Scrapen zu belasten, werden die Stellenausschreibungen an dieser Stelle zunächst als HTML-Datei gespeichert. Dazu ist es wichtig, sich im Vorfeld eine Systematik überlegt zu haben, wie man die einzelnen Dateien benennt. Um schnell identifizieren zu können, welche Stellenbeschreibungen mit welchen Suchbegriffen auf welcher Plattform gefunden wurden, damit Berufsbezeichnungen oder Treffer von einzelnen Stellenbörsen individuell analysiert werden können, wurde beides in den Dateinamen integriert (*search*- und *filename*-Variable in Listing 8). Da jede Datei einen einzigartigen Dateinamen benötigt, wurde zudem ein Zeitstempel angefügt.

Um nicht zu viel Traffic zu verursachen, wurde am Ende des *for*-Loops eine Pause von drei Sekunden eingefügt, bevor die nächste Stellenausschreibung gescraped und gespeichert wird.

```
for rel_url in searchTerms_dict.keys():
    url = 'http://www.stepstone.de' + rel_url
    header = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0;
                           WOW64; rv:49.0) Gecko/20100101 Firefox/49.0'}
    response = requests.get(url, headers = header)
    webContent = response.text
    search = list(itertools.chain.from_iterable
                  (searchTerms_dict[rel_url]))
    search = ','.join(search)
    filename = search + '_Stepstone_' + str(time.time())
    filePath = 'G:/BA/Stellenbeschreibungen/Corpora/html/neu/'

    save_result(webContent, filePath, filename, 'html')

    time.sleep(3)
```

Listing 8: Scrapen und Speichern der Stellenbeschreibungen von Stepstone

4.2.2. Dublettenkontrolle

Möchte man Daten von unterschiedlichen Stellenbörsen beziehen oder arbeitet man mit verschiedenen Suchbegriffen, stößt man auf das Problem, dass Stellenanzeigen doppelt auftauchen. Sei es, weil die gleiche Stellenausschreibung auf verschiedenen

Plattformen veröffentlicht wurde, oder weil es unter den Suchbegriffen oder einzelnen Berufsbezeichnungen Überschneidungen in der Treffermenge auf einer einzelnen Plattform gibt. Diese Dubletten sollten aussortiert werden, damit die Analyse nicht verfälscht wird.

Dubletten, die innerhalb einer Stellenbörse auftauchen, sind in der Regel unkompliziert auszusortieren. Da die URL zu der jeweiligen Stellenausschreibung einzigartig ist, können diese verglichen und Dopplungen auf diese Weise identifiziert werden. Falls die Suchbegriffe für die Analyse relevant sind, können die Begriffe, mit denen die Dubletten gefunden wurden, in den Metadaten der einzelnen Stellenbeschreibungen gespeichert werden.

Führt man mehrere Datensammlungsvorgänge über einen längeren Zeitraum durch, sollte zusätzlich über das Publikationsdatum der Stellenausschreibung verhindert werden, dass Dubletten gespeichert werden. Dazu müssen die Suchergebnisse vorher nach Datum sortiert werden, was bei den meisten Stellenbörsen über den Query-String festgelegt werden kann, um anschließend mit einem if-else-Statement nur die Datensätze auszuwählen, die nach der zuletzt gespeicherten Stellenausschreibung erschienen sind. Die Kombination aus URL und Erscheinungsdatum verhindert innerhalb einer Plattform zuverlässig das Speichern von Dubletten.

Schwieriger wird es bei den Dubletten, die entstehen, wenn Stellenausschreibungen auf mehreren Plattformen veröffentlicht werden. URL und Publikationsdatum sind dabei keine große Hilfe. Eine Möglichkeit wäre es, die zu speichernde Stellenbeschreibung mit dem Volltext sämtlicher bereits gespeicherter Stellenausschreibungen zu vergleichen. Dazu muss sichergestellt sein, dass die Stellenausschreibungen jeweils sauber von den übrigen Elementen des HTML-Dokuments extrahiert wurden, so dass keine stellenbörsenspezifischen Elemente mehr vorhanden sind. Dieser Prozess kann, je nach Datenmenge, sehr viel Zeit in Anspruch nehmen.

Denkbar wäre es auch, aus Elementen, die in jeder Stellenbeschreibung vorhanden sind, für jede individuelle Stellenanzeige einen einzigartigen Schlüssel zu erstellen. Hierbei könnte man zum Beispiel den Titel der Stellenanzeige, den Arbeitgeber und die ersten drei Sätze der Stellenbeschreibung nehmen und diese miteinander abgleichen. Dabei wäre es sinnvoll den Schlüssel als CSV- oder Excel-Datei zu speichern, um nicht nach jedem erneuten Datensammelungsprozess sämtliche bereits gespeicherten Stellenbeschreibungen erneut laden und aus ihnen den Schlüssel extrahieren zu müssen.

4.2.3. Anwendungsbeispiel

Für das Anwendungsbeispiel dieser Arbeit wurden insgesamt 1036 Stellenausschreibungen im Zeitraum vom 31.07. bis zum 25.10.2016 von den Stellenbörsen der Zeit, Stepstone und Xing gesammelt. Exemplarisch werden in diesem Kapitel die Schritte aus Kapitel 4.2.1. mit Code-Elementen für die Stellenbörse Stepstone durchgeführt. Das komplette Skript sowie die Skripte für die anderen beiden Stellenbörsen befinden sich im Anhang.

Berufsbezeichnung	Xing	Stepstone	Zeit
Archivar	Archivar	Archivar	Archivar
Bibliothekar	Bibliothekar	Bibliothekar	Bibliothekar
Dokumentar	Dokumentar	Dokumentar	Dokumentar
Informations-designer	Informations-designer	Informations-designer	Informations-designer
Informations-manager	Informations-manager	Informations-manager	Informations-manager
Informations-spezialist	Informations-spezialist	Informations-spezialist	Informations-spezialist
Informations-wissenschaftler	Informations-wissenschaftler	Informations-wissenschaftler	Informations-wissenschaftler
Systembibliothekar	Systembibliothekar	Systembibliothekar	Systembibliothekar
IT Bibliothekar	IT Bibliothekar	-	IT Bibliothekar
Technischer Bibliothekar	Technischer Bibliothekar	-	Technischer Bibliothekar

Tabelle 10: Suchbegriffe aller genutzten Stellenbörsen

In Schritt 1 (*Formulieren einer Suchanfrage*) wird Tabelle 10 mit der in Kapitel 4.2.1. (Listing 3) beschriebenen Funktion von Google Drive als Data Frame in das Jupyter Notebook geladen. Beim Erstellen des Spreadsheets auf Google Drive ist darauf zu achten, dass alle Spalten gleich lang sind, da es ansonsten zu einer Fehlermeldung kommt, wenn die Tabelle in ein Data Frame umgewandelt wird.

```
df = spreadsheet("1Ip0tQ2rwpUWw-FZ4BX9_LwJe0HXtIzHRpdmKCITdaTs")
searchTerms = [term for term in df["Stepstone"]]
```

Listing 9: Erstellen einer Liste von Suchbegriffen aus einer Tabellenspalte

In der Variablen *searchTerms* wird zunächst eine leere Liste erstellt, die mit jedem Begriff der entsprechenden Spalte des Data Frames gefüllt wird. Die Data Frame Spalte wird dabei mit dem Spaltennamen ausgewählt (Listing 9).

Im Anschluss werden für jeden Suchbegriff die Anzahl der gefundenen Suchübersichtsseiten benötigt, die zusammen in einem Dictionary gespeichert werden. Daraufhin werden die URLs, die gescraped werden, und das Verzeichnis, in dem die Suchübersichtsseiten gespeichert werden sollen, festgelegt. Der for-Loop nimmt nun jeden Suchbegriff und übergibt diesen zusammen mit der dazugehörigen Seitenzahl und der URL der *search_results*-Funktion, die die Suchübersichtsseite scraped. Dadurch entsteht ein fließender Übergang von Schritt 1 zu Schritt 2 (*Scrapen der Suchübersicht*). Jede Suchübersichtsseite wird dann mit der *save_result*-Funktion in dem angegebenen Verzeichnis, Dateinamen und einem Zeitstempel als HTML-Datei gespeichert (Listing 10). Am Ende pausiert das Programm für drei Sekunden, bevor die Schleife mit der nächsten Suchübersichtsseite oder dem nächsten Suchbegriff wieder von vorne beginnt.

```
pages = {searchTerms[0]: 1, searchTerms[1]: 1, searchTerms[2]: 1,
         searchTerms[3]: 1, searchTerms[4]: 1, searchTerms[5]: 1,
         searchTerms[6]: 1, searchTerms[7]: 1}

url = 'https://www.stepstone.de/5/ergebnisliste.html?'
path = 'G:/BA/Stellenbeschreibungen/Corpora/Suchuebersicht/'

for term in searchTerms[:7]:
    for result in search_results(url, term, pages[term]):
        save_result(result.text, path, 'stepstone_search_result' +
                    str(time.time()), 'html')
    time.sleep(3)
```

Listing 10: Scrapen und Speichern der Suchübersichtsseiten

In diesem Beispiel sind die letzten beiden Suchbegriffe nicht für die gewählte Stellenbörse geeignet (näheres dazu in Kapitel 5.1. - *Qualität der Suchbegriffe*), daher werden in dem äußeren for-Loop lediglich die ersten sieben Suchbegriffe genutzt.

Schritt 3 (*Extrahieren der Stellenausschreibungs-URL und Metadaten*) besteht aus einem for-Loop, der je nach Information, die man neben der URL zur Stellenausschreibung noch benötigt, variieren kann (Listing 11).

Zunächst wird mit `os.listdir` zum Ordner der soeben gespeicherten Suchübersichtsseiten navigiert. Anschließend wird ein leeres Dictionary erstellt, das im folgenden for-Loop sukzessive gefüllt wird. Dieses wird dazu genutzt, plattforminterne Dubletten zu vermeiden und gleichzeitig sämtliche Suchbegriffe, mit denen die Stellenausschreibung gefunden wurde, zu erfassen.

Falls im Ordner der gespeicherten Suchübersichtsseiten eine Datei gefunden wird, die den Bestandteil `stepstone_search_result` enthält, wird die Datei mit UTF-8 Encodierung geöffnet und die HTML-Datei in ein BeautifulSoup-Objekt umgewandelt.

Aus dem BeautifulSoup-Objekt wird zunächst das Tag ausgelesen, in dem sich der Suchbegriff, mit dem die Stellenausschreibung gefunden wurde, befindet. In diesem Fall bekommt man einen kompletten Absatz ausgegeben, aus dem mit Regex der exakte Suchbegriff extrahiert wird und der `query`-Variable zugewiesen wird.

Als nächstes wird die URL zu der eigentlichen Stellenausschreibung extrahiert. Auch hier wird zunächst mit BeautifulSoup's `find_all`-Funktion ein kompletter div-Container identifiziert, bevor mit Regex die reine URL extrahiert wird. Da die URL zu einer Stellenausschreibung nur einmal vergeben werden kann, wird sich diese Eigenschaft für das Verhindern von plattforminternen Dubletten zu Nutze gemacht. Die URL wird am Ende der Schleife als Key, zusammen mit dem Suchbegriff als Wert, in das anfangs leere Dictionary hinzugefügt. Bei jeder Schleife wird abgeglichen, ob sich die URL bereits im Dictionary befindet. Falls nicht wird sie hinzugefügt, andernfalls wird lediglich der Suchbegriff der sich bereits im Dictionary befindenden URL als weiteren Wert hinzugefügt.

```

searchResults = os.listdir(path)
searchTerms_dict = {}

for files in searchResults:
    if files.find("stepstone_search_result") != -1:
        with open(path + files, 'r', encoding="utf8") as jobs:
            soup = BeautifulSoup(jobs, "lxml")

            query_sent = soup.find('p',
                                   attrs = {'class' : 'mb--xxs'})
            query = re.findall('<strong>(.*?)</strong> ergab',
                              str(query_sent))

            job_urls = soup.find_all('div',
                                     attrs = {'class' : 'jobitem__header'})

            for link in job_urls:
                links = re.findall('href=\\\"(.*)\\\">', str(link))
                links = ''.join(links)

                if links not in searchTerms_dict:
                    searchTerms_dict[links] = [query]

                else:
                    if query not in searchTerms_dict[links]:
                        searchTerms_dict[links].append(query)

```

Listing 11: Extraktion der URL und der Metadaten zu den Stellenausschreibungen

In Schritt 4 (*Scrapen und Speichern der Stellenausschreibung*) wird schlussendlich die im vorigen Schritt extrahierte URL genutzt, um die Stellenausschreibungen zu scrapen und die entsprechende HTML-Seite zu speichern (Listing 12).

Dazu muss zunächst die relative URL zu einer absoluten URL umgewandelt werden, bevor mit der *get*-Funktion der *Requests*-Bibliothek die Anfrage an den Server gestellt wird.

Für den Dateinamen der individuellen Stellenausschreibungen werden die Suchbegriffe aus dem vorigem Schritt genutzt. Diese befinden sich allerdings in Listen, so dass sie zuvor dort extrahiert werden müssen. Des Weiteren müssen sämtliche Leerzeichen entfernt oder durch Unterstriche ersetzt werden, um als Dateiname genutzt werden zu können. Für eine bessere Übersicht werden dem Dateinamen noch der Name der Stellenbörse sowie ein Zeitstempel hinzugefügt, der die Einzigartigkeit des Dateinamens zu gewährleistet.

```

for rel_url in searchTerms_dict.keys():
    url = 'http://www.stepstone.de' + rel_url
    header = {'User-Agent': 'Mozilla/5.0
              (Windows NT 10.0; WOW64; rv:49.0) Gecko/20100101
              Firefox/49.0'}
    response = requests.get(url, headers = header)

    webContent = response.text
    search = list(itertools.chain.from_iterable
                  (searchTerms_dict[rel_url]))
    search = ','.join(search)
    search = re.sub('\s|-', '_', search)

    filename = search + '_Stepstone_' + str(time.time())
    filePath = 'G:/BA/Stellenbeschreibungen/Corpora/html/'

    save_result(webContent, filePath, filename, 'html')

    time.sleep(3)

```

Listing 12: Scrapen und Speichern der Stellenausschreibung

4.2.4. Zusammenfassung

Wie in Abbildung 10 zu sehen, werden in der zweiten Phase die Suchbegriffe und Filterkriterien, die auf Grundlage von Phase I festgelegt wurden, mit den Datenquellen, die aufgrund der Evaluation in eben dieser Phase ausgewählt wurden, verbunden und eine automatische Suchanfrage an den Server gesendet. Die daraus resultierende Suchübersichtsseite wird im Anschluss dazu genutzt, die individuellen URLs zu den Stellenausschreibungen zu extrahieren, die in Zusammenhang mit dem Publikationsdatum als Filter dienen, um zu vermeiden Stellenausschreibungen doppelt zu speichern. Abschließend wird die Stellenausschreibung als HTML-Dokument gespeichert. Notebooks zum Scrapen der Stellenbörsen der Zeit, Xing und Stepstone befinden sich in Anhang.

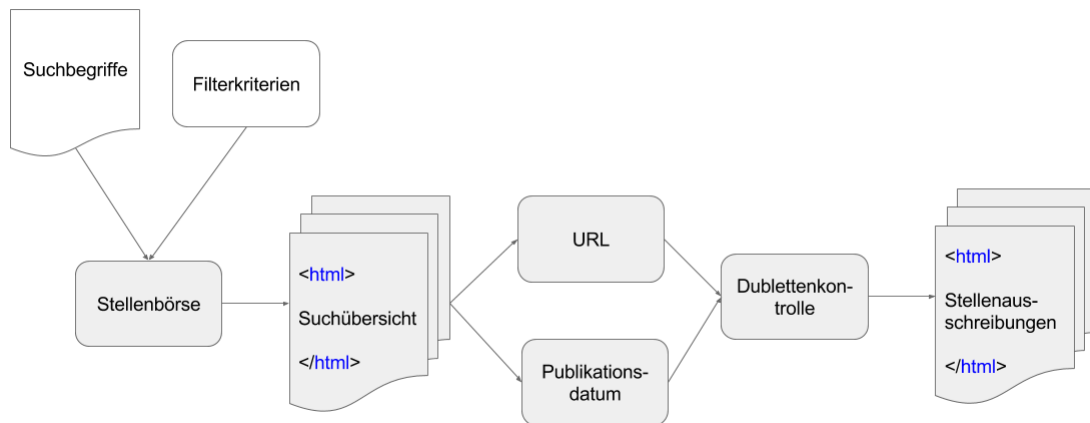


Abbildung 10: Zusammenfassung Phase II: Datensammlung

4.3. Phase III: Datenaufbereitung

Für die eigentliche Analyse der Daten ist es notwendig, diese aufzubereiten. Dieser Prozess ist meistens der aufwändigste Teil eines Text Mining Projektes. Dafür sind in der Regel verschiedene Schritte nötig, um die Daten für die jeweilige Zielsetzung zu bereinigen und zu strukturieren.

In diesem Kapitel werden die gängigen Schritte erklärt, um die Stellenausschreibungen in einzelnen HTML-Dokumenten in eine analysierbare Form zu bringen. Je nach Zielsetzung des Datenprojektes können noch weitere Schritte nötig sein, um die Daten aufzubereiten oder es kann sich eine andere Reihenfolge anbieten. Wie in Abbildung 11 zu sehen, werden die einzelnen Stellenausschreibungen im HTML-Format mit Hilfe einer Stoppwortliste bereinigt. Durch den weiteren Prozessablauf entsteht am Ende eine CSV-Datei mit den nach Wortart sortierten Wörtern und dem dazugehörigen Satz, in dem das Wort in der Stellenausschreibung vorkommt.

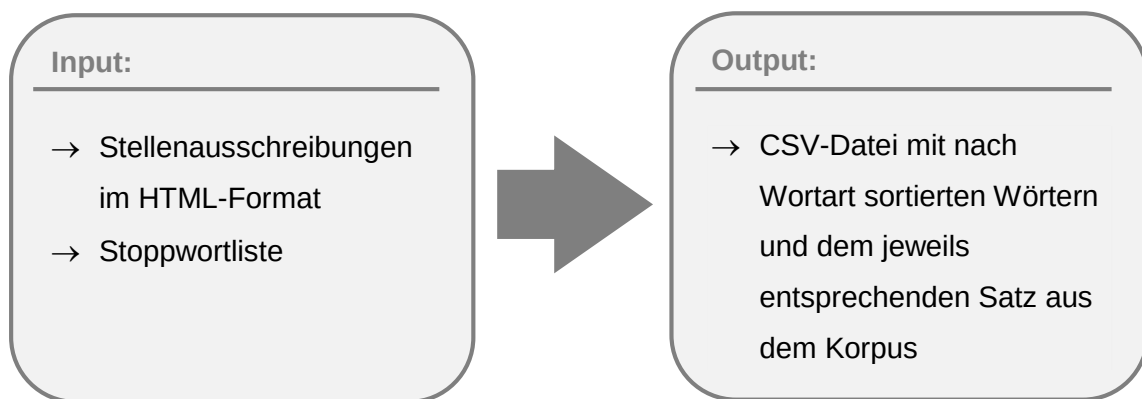


Abbildung 11: Input und Output Phase III - Datenaufbereitung

4.3.1. Extrahieren der Stellenbeschreibung und Erstellen eines Korpus

Bisher besteht der Datensatz aus einer Sammlung kompletter HTML-Seiten, inklusive Navigationsleisten, Formularfelder und Ähnlichem. Um sich den eigentlichen Stellenausschreibungen anzunähern, muss identifiziert werden, an welcher Stelle des DOM-Objektes sich diese befinden.

Weiterhin sollte bei diesem Schritt überlegt werden, welche Metadaten für das Erreichen des Analyseziels nötig sind. Für einen Überblick, wie häufig Stellenanzeigen im jeweiligen Berufsfeld publiziert werden, benötigt man zum Beispiel das Datum, an dem die Stellenanzeige veröffentlicht wurde. Es ist auch möglich dem Dokument, zusätzlich zu den Metadaten, die in dem HTML-Dokument vorhanden sind, weitere beschreibende Daten, wie zum Beispiel die Suchbegriffe, mit denen die Stellenanzeige gefunden wurde, hinzuzufügen.

Um die reine Stellenbeschreibung zu erhalten, sind zwei Schritte nötig.

Schritt 1 - Lokalisieren und Extrahieren der Stellenbeschreibung und der relevanten Metadaten:

Für diesen Schritt muss erneut der HTML-Code der Stellenausschreibung betrachtet werden. Wie bei der Lokalisierung der URL in Kapitel 4.2.1., kann sich die Stellenausschreibung bei jeder Domain an einer anderen Stelle befinden. Im Idealfall sind diese in einem div-Container oder einem article-Tag positioniert, dem eine eindeutige Klasse oder ID zugewiesen wurde. Im Einzelfall kann es allerdings auch

vorkommen, dass jede Stellenbeschreibung einer Domain eine unterschiedliche ID erhalten hat. In dem Fall kann es sich anbieten, den Bereich um die Stellenausschreibung etwas weiter zu fassen und einen anderen div-Container zu wählen, bevor man jede Stellenausschreibung einzeln nach der entsprechenden ID durchsucht. Der Nachteil dieser Methode ist, dass Navigationsleisten oder andere Elemente der HTML-Seite mit einbezogen werden, die später entfernt werden müssten, um die Analyse nicht zu beeinflussen.

Schritt 2 - Markup und Skriptelemente entfernen:

Auch wenn jetzt die Stellenausschreibung von der übrigen Webseite extrahiert wurde, befinden sich noch immer HTML-Tags und Skriptelemente in dem Dokument. Möchte man lediglich den Text der Stellenbeschreibung analysieren, kann sämtliches Markup mit BeautifulSoup's `getText`-Funktion entfernt werden.

Wenn man allerdings Listenelemente untersuchen möchte, sollten diese vorher extrahiert werden.

Je nach Zielsetzung kann dieser Schritt auch erst später in der Phase vervollständigt werden.

In einem weiteren Schritt wird nun mit den bereinigten Dokumenten ein Korpus gebildet.

Schritt 3 – Dokumente in Korpus laden:

Ein Korpus ist eine Sammlung von Textdokumenten, die die Grundlage für eine sprachwissenschaftliche Untersuchung bilden (Bibliographisches Institut GmbH 2016). Das NLTK bietet einem für die Erstellung eines individuellen Korpus, die *PlaintTextCorpusReader*-Funktion. Mit dieser ist es möglich, je nach Analyseziel aus den Dokumenten unterschiedliche Korpora zu erstellen. Für eine umfassende Analyse aller Dokumente bietet es sich an, diese zusammen in einen Korpus zu laden. Möchte man zunächst nur eine bestimmte Berufsbezeichnung analysieren, kann man einen Korpus mit ausschließlich den Dokumenten bilden, die mit dem entsprechenden Suchbegriff gefunden wurden.

4.3.2. Verarbeitungsverfahren

In wie weit der Korpus für die Analyse weiterverarbeitet werden muss, hängt wiederum von dem spezifischen Analyseziel ab. Um Kompetenzen zu identifizieren, sind folgende Schritte nötig.

Schritt 4 – Segmentierung:

Bei der Segmentierung wird der Textkorpus in einzelne Segmente aufgeteilt, die daraufhin analysiert werden.

Pythons NLTK-Paket bietet einem zwei verschiedene Möglichkeiten den Text zu segmentieren. Zum einen können Segmente, auch Tokens genannt, in Form von ganzen Sätzen oder Listenelementen bestehen, zum anderen aus einzelnen Worten, wobei bei letzterem auch Satzzeichen und Zahlenelemente als einzelner Token behandelt werden. Für die weiteren Bearbeitungsschritte wurde der Korpus zunächst in Satz-Tokens unterteilt, bevor jeder Satz noch einmal Wort für Wort segmentiert wurde. Dieses Verfahren hat den Vorteil, dass es möglich ist, jedem Wort den entsprechenden Satz, in dem das Wort vorkam, zuzuordnen.

Während der Wortsegmentierer die einzelnen Wörter am Leerzeichen trennt, nutzt NLTKs Satzsegmentierer die Interpunktion als Anhaltspunkt, wo ein Satz endet und ein anderer beginnt. Das funktioniert so lange, bis Listenelemente in Stellenbeschreibungen auftauchen, die nicht mit einem Satzzeichen beendet werden. Da die Funktion aufgrund des fehlenden Satzzeichens nicht erkennen kann, wo das Listenelement aufhört, wird die komplette Liste als ein Satz betrachtet. Dieses erscheint auf dem ersten Blick nicht unbedingt problematisch. Bedenkt man jedoch, dass Listen in Stellenausschreibungen oftmals genutzt werden, um das Anforderungsprofil darzustellen, wird klar, dass so von einigen Stellenbeschreibungen die kompletten Anforderungen in einem einzigen Satztoken zusammengefasst werden. Möchte man zum Beispiel zu einer bestimmten Kompetenz den dazugehörigen Satz finden, bekommt man einen umfangreichen Textblock, in dem alle Kompetenzen, die in der Stellenausschreibung genannt werden, vorkommen. Um dieses zu verhindern wurden im Anwendungsbeispiel die Listenelemente zuvor extrahiert und separat behandelt.

Schritt 5 - Part of Speech (POS) Tagging:

Beim POS-Tagging wird jedem Wort seine entsprechende grammatikalische Wortart zugeordnet und mit einem entsprechenden Kürzel annotiert (Bachs et al. 2014). Das NLTK-Paket liefert zwar einen POS-Tagger, der das Annotieren übernehmen kann, jedoch liefert dieser für deutschsprachige Texte kein zufriedenstellendes Resultat. Daher musste für diesen Schritt von der Zielsetzung, lediglich Bibliotheken und Funktionen aus der Anaconda Distribution zu nutzen, abgesehen und ein externer Tagger genutzt werden. Die Wahl fiel auf den Tagger, den Markus Konrad vom Wissenschaftszentrum Berlin für Sozialforschung am annotierten TIGER-Korpus des Instituts für Maschinelle Sprachverarbeitung der Universität Stuttgart trainiert hat. Dieser Tagger erreicht eine Präzision von 96% mit dem bereits erwähnten Korpus (Konrad 2016). Der trainierte Tagger kann als pickle-Datei von der im Literaturverzeichnis genannten Quelle heruntergeladen und anschließend mit dem pickle-Modul in das Skript geladen werden (Listing 13). Zuvor muss noch das NLTK-Contributions-Modul von GitHub⁵ heruntergeladen und in dem Wurzelverzeichnis der Anaconda-Distribution entpackt werden.

```
import pickle
with open ('input/nltk_german_classifier_data.pickle', 'rb') as f:
    tagger = pickle.load(f)
```

Listing 13: Tagger import

Der Tagging-Prozess kann, je nach Rechenleistung des PCs und Größe des Korpus, eine Weile dauern. Daher ist es sinnvoll den getaggten Korpus im Anschluss zu speichern. Das kann entweder als pickle-Datei mit der *dump*-Funktion geschehen oder die Wort-Tag-Paare werden in ein Data Frame übertragen, welches wiederum als CSV- oder Excel-Datei gespeichert wird.

Letztere Variante bietet sich an, da die Daten für den Analyseprozess ohnehin in einem Data Frame strukturiert sein müssen. Im konkreten Anwendungsfall wurde zusätzlich zu dem Wort und dem dazugehörigen Tag noch der Satz, in dem das Wort vorkommt, in einer dritten Data Frame-Spalte hinzugefügt, um zum einen in der Datenanalyse-Phase das Kategoriensystem zu verfeinern und zum anderen die Qualität des

⁵ Download unter: <https://github.com/ptnplanet/NLTK-Contributions>

Kategoriensystems zu überprüfen (näheres dazu in den entsprechenden Kapiteln 4.4.1. und 5.2.).

Tabelle 11 zeigt beispielhaft das Stuttgart-Tübingen Tagset für die deutsche Sprache. Mit dem Tag lassen sich anschließend die Wörter nach Wortart sortieren. Zusätzlich ist es möglich bestimmte Wortarten zu extrahieren oder von der weiteren Analyse auszuschließen.

Tag	Bedeutung
ADV	Adverb
APPR	Präposition
APPO	Postposition
ART	bestimmter oder unbestimmter Artikel
ITJ	Interjektion
KOKOM	Vergleichskonjunktion
NE	Eigennamen
NN	normales Nomen
PDS	substituierendes Demonstrativpronomen
PWS	Substituierendes Interrogativpronomen
PPER	irreflexibles Personalpronomen
PTKNEG	Negationspartikel
VVFIN	finites Verb, voll
VVINFINF	Infinitiv, voll
XY	Nichtwort, Sonderzeichen enthaltend
\$,	Komma
\$.	Satzbeendende Interpunktion

Tabelle 11: Beispiel POS-Tagset (nach Backs et al. 2014)

Schritt 6 - Stoppwort- und Satzzeichenentfernung:

Als Stoppwörter bezeichnet man die Wörter, die keine Bedeutung transportieren. Daher ist es sinnvoll, diese Wörter zu entfernen, um die Datenmenge zu reduzieren und sich so den Fähigkeiten, die in den Stellenausschreibungen genannt werden, anzunähern. Das NLTK-Paket bietet dafür eine Funktion, mit der man auf die Sprache spezifiziert Standardstoppwörter entfernen kann. Obwohl bei der beispielhaften Datensammlung lediglich deutschsprachige Stellenbörsen genutzt wurden und auch die Suchbegriffe auf den deutschsprachigen Arbeitsmarkt ausgelegt waren, erwies es sich dennoch als sinnvoll, sowohl deutsche als auch englische Stoppwörter zu entfernen. Zum einen wurde die Suche nicht geographisch eingegrenzt, so dass vereinzelt Stellenbeschreibungen aus anderen Ländern gefunden wurden, zum anderen verfassen einige internationale Unternehmen auch im deutschsprachigen Raum ihre Stellenausschreibungen in englischer Sprache. In vielen Branchen haben sich zudem englischsprachige Begriffe eingebürgert, die unter Umständen von der deutschen Stoppwortfunktion nicht erfasst werden.

Die Standardfunktion des NLTK-Pakets filtert allerdings nur gängige Stoppwörter heraus. Besonders im Hinblick darauf, sich durch das Aussortieren von Wörtern den Kompetenzen in der Stellenausschreibung zu nähern, ist es sinnvoll, eine eigene Stoppwortliste anzulegen und diese in mehreren Schritten anzupassen. Dafür bietet sich abermals ein Spreadsheet an, mit dem man eine individuelle Stoppwortliste erstellt und den Korpus weiter verkleinert.

Den Prozess der Stoppwortentfernung muss gegebenenfalls häufiger wiederholt werden. Ein Beispiel wäre, wenn nach einer Worthäufigkeitenanalyse festgestellt wird, dass die meist genannten Wörter im Korpus keine Bedeutung für die Analyse haben. Dann sollte die individuelle Stoppwortliste angepasst und der Entfernungsprozess wiederholt werden.

Für den weiteren Verlauf des Prozesses sind zudem die Satzzeichen irrelevant. Diese kann man entweder entfernen, indem man nur die Wörter auswählt, die zum Beispiel als Nomen oder Verb getaggt wurden, oder lediglich die Wortsegmente auswählt, die nicht alphanumerisch sind.

Schritt 7 – Stemming:

Unter Stemming versteht man das Zurückführen des Wortes auf seinen grammatikalischen Wortstamm (Cambridge University Press 2008). Dieser Schritt ist

nötig, weil ein Wort in mehreren grammatikalischen Formen im Korpus vorkommen kann. Diese werden durch das Stemming als ein Wort erkannt und können in einer Wortfrequenzanalyse zusammengefasst werden.

4.3.3. Anwendungsbeispiel

Zunächst ist es nötig, die Analyseeinheiten, also die Stellenbeschreibungen, von der restlichen HTML-Seite zu trennen und die nötigen Metadaten zu extrahieren. Ähnlich wie bei dem Scrapingprozess der Stellenausschreibungen werden die einzelnen Dateien in Schritt 1 (*Lokalisieren und Extrahieren der Stellenbeschreibung und der relevanten Metadaten*) geöffnet und anschließend in ein BeautifulSoup-Objekt umgewandelt. JavaScript- und Style-Elemente werden entfernt (Schritt 2 – *Markup und Skriptelemente entfernen*), bevor im Anschluss der div-Container, in dem sich die Stellenausschreibung befindet, mit der *find*-Funktion lokalisiert und der *jobData*-Variable zugewiesen wird (Listing 14). Die DOM-Struktur der Stellenausschreibung wird zunächst beibehalten.

```
path = 'G:/BA/Stellenbeschreibungen/Corpora/html/'
searchResults = os.listdir(path)

for files in searchResults:
    if files.find('Stepstone') != -1:
        with open(path + files, 'r', encoding="utf8") as jobs:
            soup = BeautifulSoup(jobs, "lxml")

            for script in soup(["script", "style"]):
                script.extract()

            jobData = soup.find('div', attrs = {'id' : 'Content'})
```

Listing 14: Extraktion der Analyseeinheiten

Soll lediglich der Inhalt der Stellenausschreibungen analysiert und keine zusätzlichen Daten in den Analyseprozess einbezogen werden, kann die Stellenbeschreibung, wie zuvor in Kapitel 4.2.3. beschrieben, gespeichert werden. Für den weiteren Prozess sollte für die Dateien das TXT-Format gewählt werden.

Ist es für die Beantwortung der Forschungsfrage nötig Metadaten in die Analyse mit einzubeziehen, können diese in der gleichen Schleife aus dem BeautifulSoup-Objekt extrahiert werden. Wie in Listing 15 ersichtlich, werden in diesem Fall der Titel der

Stellenausschreibung aus dem *title*-Tag sowie die für die Stellenausschreibung vergebenen Schlagwörter, die URL und das Datum, an dem die Stellenausschreibung veröffentlicht wurde, extrahiert. Zusätzlich werden aus dem Dateinamen mit Regex die Suchbegriffe herausgezogen.

Um diese als Meta-Tags der Stellenausschreibung hinzuzufügen wird BeautifulSoup's *new_tag*-Funktion genutzt. Auf diese Weise können dem Tag die einzeln extrahierten Daten als Attribut-Wert Kombination hinzugefügt werden, so dass die Metadaten, bei einheitlicher Benennung der Attribute, später unabhängig von der Stellenbörse ausgelesen und analysiert werden können. Zu beachten ist, dass die Attributwerte als String angegeben werden müssen. Datenformate, wie zum Beispiel Integer oder Float, verursachen Fehlermeldungen.

Um den individuell erstellten Metatag mit der Stellenausschreibung zusammen zu führen, kann man diesen mit der *write*-Funktion der geöffneten Stellenausschreibungsdatei hinzufügen. Wichtig dabei ist lediglich, dass das „w“ für write mit einem „a“ für append ausgetauscht wird, ansonsten wird die Stellenausschreibung mit dem Metatag überschrieben.

```
title = soup.find('title')
if title != None:
    title = title.get_text()

keywords = soup.find('meta', attrs = {'name' : 'keywords'})
website = soup.find('link', attrs = {'rel' : 'canonical'})
uploadDate = soup.find('div', attrs = {'itemprop' : 'datePosted'})

query = re.sub('_', ' ', files)
query = re.sub('Stepstone|_d+|.\d+|\.|html', '', query)

metaTag = soup.new_tag('meta')

metaTag.attrs['searchTerms'] = str(query)
metaTag.attrs['title'] = str(title)
metaTag.attrs['keywords'] = str(keywords)
metaTag.attrs['website'] = str(website)
metaTag.attrs['uploadDate'] = str(uploadDate)

with open(filePath, 'w', encoding='utf8') as f:
    f.write(str(jobData) + ' ')

with open(filePath, 'a', encoding='utf8') as f:
    f.write(str(metaTag))
```

Listing 15: Extrahieren und Hinzufügen von Metadaten

Nun können die einzelnen Textdateien in einen Korpus geladen werden (Schritt 3 – *Dokumente in Korpus laden*) und in ein Rohtext-Format umgewandelt werden (Listing 16). In diesem Fall werden alle Dateien im TXT-Format, die sich in dem angegebenen Ordner befinden, zu einem einzigen Rohstring zusammengefügt.

```
data = 'G:/BA/Stellenbeschreibungen/Corpora/txt/'
corpus = PlaintextCorpusReader(data, '.*txt')
all_text = corpus.raw()
```

Listing 16: Korpus erstellen

Für die verbleibende Analyse werden die Listenelemente in den Stellenausschreibungen getrennt behandelt. Dafür werden sämtliche HTML-Tags außer den Listenelementen entfernt. Diese werden schließlich der Variable *list_items* zugewiesen (Listing 17).

```
clean_text = re.sub(r'<[^>]*^<li>>|\\[.*\\]', '',
                    all_text)
list_items = re.findall(r'<li>(.*?)</li>', clean_text)
```

Listing 17: Entfernen von Tags und Extrahieren von Listenelementen

Da die Listenelemente bereits ein Segment darstellen, werden diese nur noch in Wortsegmente unterteilt (Schritt 4 – *Segmentierung*). Der gewünschte Output in dieser Phase ist eine CSV-Datei mit nach Wortart sortierten Wörtern, dem entsprechenden POS-Tag und dem Listenelement, in dem das Wort vorkommt. Daher wird die Wortsegmentierung im gleichen Schritt vorgenommen, wie Schritt 5 (*POS-Tagging*). Listing 18 zeigt das Verfahren am Beispiel für Adjektive. Zunächst wird eine leere Liste der Variablen *adjectives* zugewiesen. Nun wird jedes Listenelement in der Listenelementliste nach Wörtern segmentiert. Die einzelnen Wörter werden dann mit dem POS-Tagger annotiert, so dass ein Wort-Wortart-Paar entsteht. Jedes dieser Paare wird nun daraufhin geprüft, ob das Wortart-Kürzel mit *ADJ* beginnt und somit als Adjektiv gekennzeichnet wurde. Zudem wird aus dem Paar ein Trio, indem das dazugehörige Listenelement hinzugefügt wird. Abschließend wird das Trio der anfangs leeren Liste angefügt.

```

adjectives = []
for li_item in list_items:
    tok = nltk.word_tokenize(li_item)
    tagged = tagger.tag(tok)
    li = [(li_item, word, pos) for (word,pos) in tagged
          if pos.startswith('ADJ')]
    for (li_item, pos, word) in li:
        adjectives.append((word, pos, li_item))

```

Listing 18: Wortsegmentierung und POS-Tagging der Listenelemente

Da das POS-Tagging, je nach Datenmenge und Leistungsfähigkeit des Rechners, ein langwieriger Prozess sein kann, empfiehlt es sich, das Resultat zu speichern. Dazu muss die Liste zunächst in ein Data Frame umstrukturiert werden, bevor dieses als eine CSV- oder Excel-Datei gespeichert werden kann (Listing 19).

```

tagged_dataframe = pandas.DataFrame(adjectives,
                                    columns = ["word", "pos-tag", "sentence"])

path = 'output/'
filename = 'posTagged_adjectives.xlsx'

tagged_dataframe.to_excel(path + filename, index = False,
                          encoding = 'utf-8')

```

Listing 19: Speichern als Excel-Datei

Diese Tabelle enthält eine Vielzahl an Wörtern, die keine Bedeutung für die Analyse haben. Diese werden in Schritt 6 (*Stoppwortentfernung*) entfernt.

Listing 20 zeigt die Stoppwortentfernung mit der standard-Stoppwort-Funktion des NLTK-Pakets. Von 398 ursprünglichen Adjektiven wurde mit dieser Funktion lediglich eines entfernt.

```

no_stopwords = [(word, pos, li_item) for (word, pos, li_item) in
                 adjectives if word not in stopwords.words('german') and
                 word not in stopwords.words('english')]

```

Listing 20: Standard Stoppwortentfernung

Aus diesem Grunde ist es sinnvoll, eine individuelle Stoppwortliste anzulegen. Wenn kollaborativ an dem Projekt gearbeitet wird, bietet sich Google Spreadsheets an, um die

Liste zu erstellen. Dabei sollte darauf geachtet werden, dass der Titel der Spalte festgesetzt wird, andernfalls verrutscht dieser, wenn die eingetragenen Stoppwörter sortiert werden, was wiederum zu Fehlermeldungen führt, da im Skript mit dem Spaltentitel auf die Spalte zugegriffen wird. Listing 21 beschreibt das Vorgehen. Zunächst wird das Spreadsheet in das Skript geladen und eine Liste aus den darin enthaltenen Stoppwörtern erstellt. Anschließend wird die Liste mit der Adjektiv-Liste abgeglichen und nur die Adjektive ausgewählt, die sich nicht in der individuellen Stoppwortliste befinden.

```
df = spreadsheet('1lV3BF6_17cEbEBUAzvCo70ii1Rht2wG3tGitBtVWkzE')
custom_stopw = [word for word in df['Stopwords'].astype(str)]
no_custom_stopw = [(word, pos, li_item) for (word, pos, li_item)
                    in no_stopwords if word not in custom_stopw]
```

Listing 21: Individuelle Stoppwortentfernung

Als abschließenden Schritt in dieser Phase (Schritt 7 – *Stemming*), werden die übrigen Adjektive auf ihrem Wortstamm zurückgeführt. Dazu bietet das NLTK-Paket mehrere Stemmer. In diesem Beispiel kam der Snowball-Stemmer zum Einsatz, da bei diesem die Sprache wählbar ist (Listing 22). Das gestemmtte Wort wird als viertes Element der Matrix hinzugefügt, so dass als Abschluss dieser Phase das gestemmtte Wort, das Wort im Originalzustand, der POS-Tag und das entsprechende Listenelement zusammen in einer Zeile stehen. Bevor mit der Datenanalyse fortgefahren wird, empfiehlt es sich, das Resultat erneut als CSV- oder Excel-Datei zu speichern.

```
stemmer = SnowballStemmer('german')
stemmed = [(stemmer.stem(word), word, pos, li_item)
            for (word, pos, li_item) in no_custom_stopw]
```

Listing 22: Stemming

4.3.4. Zusammenfassung

In der Datenaufbereitungsphase werden die gesammelten und gespeicherten Stellenausschreibungen für den Analyseprozess vorbereitet. Dazu sind verschiedene Schritten nötig, die, je nach Analyseziel, unterschiedlich ausfallen können. In diesem konkreten Anwendungsfall wurden, nach der Extraktion und Bereinigung der

Stellenbeschreibungen und des Erstellens eines Korpus', die gängigen Verfahren Segmentierung, POS-Tagging und Stemming angewandt. Nach dem Stemming kann es nötig sein, erneut Stoppwörter zu entfernen, bevor die übrigen Wörter des Korpus' zusammen mit dem entsprechenden POS-Tag und dem Satz-Segment, in dem das Wort vorkommt, in einer CSV-Datei gespeichert wurde (Abbildung 12).

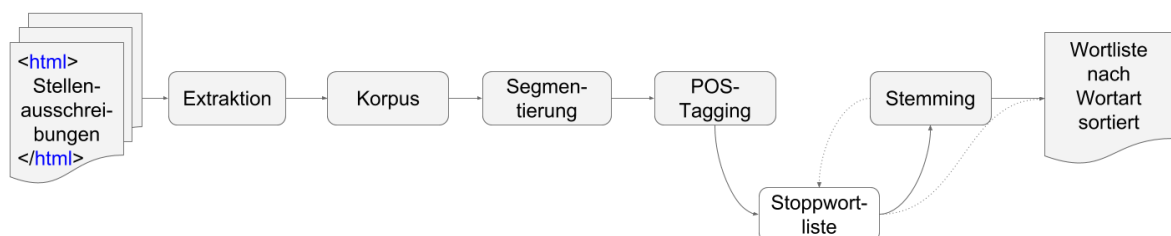


Abbildung 12: Zusammenfassung Phase III - Datenaufbereitung

4.4. Phase IV: Datenanalyse

Ist der Korpus soweit aufbereitet und sind die Wörter und Sätze strukturiert, folgt der Analyseprozess. Auch hier hängt das genaue Vorgehen von den im Vorfeld gesetzten Zielen ab. In dieser Arbeit werden die Kompetenzen anhand eines Kategoriensystems identifiziert und anschließend die Worthäufigkeiten der jeweiligen Fähigkeit beziehungsweise Kompetenz berechnet (Abbildung 13).

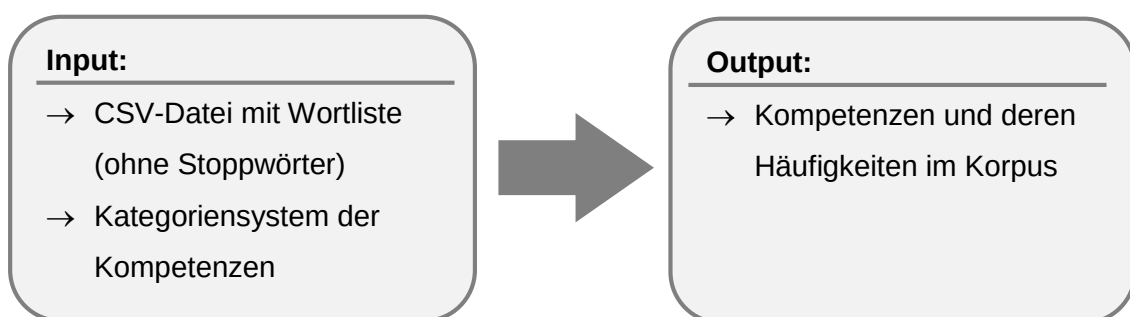


Abbildung 13: Input und Output Phase IV - Datenanalyse

4.4.1. Kategoriensystem

Das Erstellen eines Kategoriensystems ist ein aufwändiger Prozess, der aber für die Zielsetzung, eine möglichst verständliche Methode zu entwickeln, mit der kollaborativ an Datenprojekten gearbeitet werden kann, die beste Wahl erschien. Kategoriensysteme können sowohl induktiv, deduktiv oder in einer Mischform erstellt werden. Induktiv bedeutet, das Kategoriensystem wird anhand eines Teils der Datenmenge erstellt, bei einem deduktiven Vorgehen gibt es bereits eine grobe Vorstellung der Kategorien oder sie wurden anhand externer Quellen gebildet (Heindl 2015, S. 312).

In dieser Arbeit wurde das Kategoriensystem in einer Mischform entwickelt. Die Kompetenzkategorien waren deduktiv durch die Abgrenzung des Kompetenzbegriffes im beruflichen Kontext gegeben. Zusätzlich wurde der European Thesaurus for Skills and Competences herangezogen, um erste Fähigkeiten und Synonyme in das Kategoriensystem einzufügen. Im dritten Schritt wurde eine zufällig gewählte Stichprobe von 20 Stellenanzeigen genauer betrachtet und die dort genannten Fähigkeiten und Kompetenzen in das Kategoriensystem übernommen. Tabelle 12 zeigt einen Auszug des für diese Arbeit exemplarisch erstellten Kategoriensystems. Die vollständige Tabelle befindet sich im Anhang. Da der Schwerpunkt dieser Arbeit allerdings auf der Methode und nicht auf der Analyse liegt, dient das Kategoriensystem lediglich als Beispiel und erfüllt nicht den Anspruch der Vollständigkeit.

Kompetenzgruppe	Kompetenz	Synonym
fachlich	Regelwerk	RDA
fachlich	Regelwerk	RAK-WB
fachlich	Bibliothekssystem	PICA
fachlich	Bibliothekssystem	Aleph
fachlich	Bestandsaufbau	Bestandsaufbau
fachlich	Medienkompetenz	Medienkompetenz
fachlich	Medienkompetenz	Medienkompetenzvermittlung
fachlich	Benutzerstandards	Bibliotheksstandards
fachlich	Katalogisieren	Katalogisieren
fachlich	Recherche	Recherchekenntnisse
fachlich	Recherche	Literaturrecherche
fachlich	Recherche	Recherchestrategie

fachlich	Recherche	Suchmaschinen
fachlich	Bestandspflege	Bestandspflege
fachlich	Daten	Forschungsdaten
fachlich	Daten	Metadaten
fachlich	Daten	Metadatenstandards
fachlich	Regelwerk	AACR

Tabelle 12: Auszug des Kategoriensystems

Bei der Erstellung des Kategoriensystems ist es wichtig darauf zu achten, dass die Kategorien eindeutig bezeichnet werden und disjunkt sind, sich also nicht überschneiden. Da die einzelnen Fähigkeiten, je nach zu analysierender Berufsgruppe, zu unterschiedlichen Kompetenzgruppen zugeordnet werden können und die Trennung der Kompetenzgruppen teilweise nicht eindeutig ist, sollten eindeutige Regeln für die Zuordnung aufgestellt werden. Ist mehr als eine Person an der Erstellung des Kategoriensystems beteiligt, sind eindeutige Regeln unabdingbar, um eine inkonsistente Kategorienbildung zu vermeiden. Allerdings empfiehlt es sich im Sinne einer guten wissenschaftlichen Arbeitsweise auch bei nur einer Person klare Regeln festzuhalten, da dadurch der Entstehungsprozess transparent und nachvollziehbar wird.

4.4.2. Worthäufigkeiten

Mit einer Wortfrequenzanalyse kann man sich zunächst einen ersten Überblick über die im Korpus verwendeten Wörter verschaffen. Zum anderen kann mit dieser analysiert werden, welche Kompetenzen am häufigsten in den Stellenausschreibungen genannt werden. Hierbei wird ersichtlich, wie häufig jedes Wort im Korpus vorkommt. Ohne vorige Stoppwortentfernung, werden diese in der Regel am häufigsten vorkommen. So gibt eine Wortfrequenzanalyse auch über die Qualität der Stoppwortliste Aufschluss. Es empfiehlt sich nach diesem Schritt die Stoppwortliste gegebenenfalls noch einmal zu erweitern und die überflüssigen Wörter zu entfernen. Auch das Stemming ist ein notwendiger Schritt, der im Voraus durchgeführt werden muss, um ein zuverlässiges Ergebnis zu erhalten. Die *FreqDist*-Funktion des NLTK-Pakets fasst lediglich die Wörter zusammen,

die orthographisch exakt gleich sind. So würden *kreativ*, *kreatives* und *kreative* ohne Stemming als drei unterschiedliche Wörter gezählt werden.

4.4.3. Anwendungsbeispiel

Um einen Überblick über die im Korpus vorkommenden Adjektive zu bekommen, werden in Listing 23 zunächst die Wortfrequenzen aller Wortstämme gebildet und die 50 meist genannten ausgegeben. Dazu werden die Daten in ein Data Frame übertragen, bevor die Spalte mit den gestemmtten Adjektiven der *FreqDist*-Funktion übergeben wird. Mit dieser werden die Häufigkeiten eines jeden Wortstammes berechnet, bevor mit der *most_common*-Funktion die 50 häufigsten Wortstämme herausgefiltert werden (Listing 23). Den Output kann man nun entweder in die individuelle Stoppwortliste oder in das Kategoriensystem einsortieren.

```
prepped_dataframe = pandas.DataFrame(stemmed,
                                     columns = ["stemmed", "word", "pos-tag", "sentence"])
frequent = FreqDist(prepped_dataframe["stemmed"])
most_frequent = frequent.most_common(50)
```

Listing 23: Wortfrequenzanalyse Adjektive

Das in Kapitel 4.4.1. erstellte Kategoriensystem kann genauso wie die Stoppwortliste in das Skript geladen werden. Anschließend muss die Spalte, die mit den Adjektiven abgeglichen werden soll, ebenfalls gestemmt werden. Dazu wird eine vierte Spalte hinzugefügt, in der das jeweils gestemmtte Wort zu finden ist (Listing 24).

```
skill_df = spreadsheet("1cQiZhaRYAslG_erczJ979AIlJQRbn3muGX6HH7DN60c")
skill_df['stemmed'] = [stemmer.stem(s) for s in skill_df['Synonym']]
```

Listing 24: Laden des Kategoriensystems und Stemming der Synonym-Spalte

Abschließend wird die Wortstamm-Spalte des Kompetenz-Data-Frames mit der Wortstamm-Spalte des Adjektiv-Data-Frames abgeglichen und die Übereinstimmungen einer separaten Liste zugeordnet (Listing 25).


```

synonyms = skill_df['stemmed']
match_skills = []

for synonym in synonyms:
    for adjective in prepped_dataframe['stemmed']:
        match = re.findall(synonym, adjective, re.IGNORECASE)
        if match != []:
            low_match = [m.lower() for m in match]
            match_skills.append(low_match)

```

Listing 25: Matching von Synonymen

Mit den übereinstimmenden Adjektiven kann man jetzt die entsprechenden Kompetenzen aus dem Kompetenz-Data-Frame extrahieren (Listing 26).

```
skill_df.loc[skill_df['stemmed'].isin(match_skills)]
```

Listing 26: Anzeigen der Kompetenzgruppen zu den gefundenen Fähigkeiten

Tabelle 13 zeigt den Output zu Listing 26. Da dieses Beispiel nur mit einem sehr kleinen Kategoriensystem und einer limitierten Datenmenge durch die Begrenzung auf die Adjektive durchgeführt wurde, ist der Output entsprechend klein. Mit einem entsprechend ausgearbeiteten Kategoriensystem könnte man nun die Worthäufigkeiten der Kompetenzen bilden und die Ergebnisse visualisieren.

	Kompetenzgruppe	Kompetenz	Synonym	stemmed
25	fachlich	Statistik	Statistik	statist
53	methodisch	Methodik	strukturiert	strukturiert
61	persönlich	Selbstbewusstsein	sicher	sich
68	persönlich	Engagement	ergebnisorientiert	ergebnisorientiert

Tabelle 13: Finaler Output des Beispiels in Phase IV

4.4.4. Zusammenfassung

Für diese spezielle Forschungsfrage war es lediglich nötig, die Kompetenzen mit einem Kategoriensystem zu extrahieren und die entsprechenden Worthäufigkeiten zu bilden. Abbildung 14 fasst den Ablauf noch einmal zusammen. Das Kategoriensystem wurde in

einer Mischform aus induktivem und deduktivem Verfahren erstellt. Nach der Extraktion der Kompetenzen und der anschließenden Wortfrequenzanalyse kann es sinnvoll sein sowohl die Stoppwortliste als auch das Kategoriensystem noch einmal anzupassen.

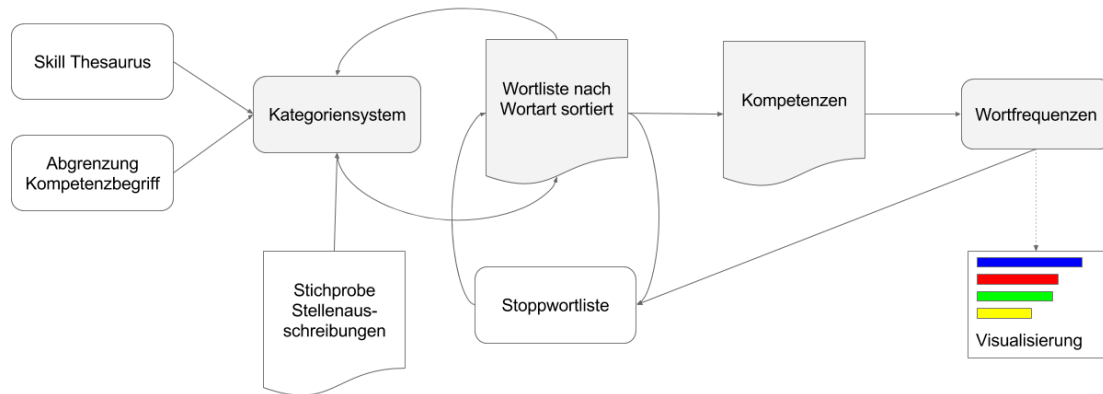


Abbildung 14: Zusammenfassung Phase IV – Datenanalyse

Nach der durchgeführten Wortfrequenzanalyse können die Ergebnisse in Grafiken visualisiert und anschließend interpretiert werden. Dieses und das weitere Vorgehen übersteigt allerdings den Umfang dieser Arbeit und wird lediglich kurz in Kapitel 6.2. angerissen.

Für andere Zielsetzungen kann es nötig sein, andere Elemente, wie zum Beispiel die Adressdaten, zu extrahieren und mit weiteren, externen Daten, wie etwa den dazugehörigen Geodaten anzureichern, um diese auf einer Karte darzustellen.

5. Qualitätskontrolle

Die Qualitätskontrolle ist keine eigenständige Phase, sondern sollte konstant in den gesamten Projektablauf mit einfließen. Je eher man bemerkt, dass die Ergebnisse nicht stimmen, desto weniger muss man anpassen. Wird zum Beispiel erst während der Analyse festgestellt, dass die gewählten Datenquellen nicht geeignet waren, muss fast der komplette Prozess erneut durchlaufen werden. Stellt man dieses jedoch gleich zu Beginn bei der Sichtung der Datenquellen oder bei der Datensammlung fest, hält sich der Aufwand in Grenzen.

Im Folgenden werden für die einzelnen Phasen Möglichkeiten aufgezeigt, mit denen die Qualität der Daten und der entwickelten Überlegungen überprüft werden können.

5.1. Qualität der Suchbegriffe

Um die Qualität der auf Grundlage des in Phase I definierten Berufsfeldes erstellten Suchbegriffe zu beurteilen, bietet es sich an, Recall und Precision für die einzelnen Begriffe anzuschauen.

Um den Recall zu berechnen, bräuchte man die Gesamtanzahl an Stellenausschreibungen zu der entsprechenden Berufsbezeichnung. Diese Daten liegen nicht vor, dennoch ist es sinnvoll, sich die Treffermenge anzuschauen und die in Frage kommenden Stellenbörsen dahingehend zu vergleichen.

Den Wert für Precision berechnet man, indem man die Zahl der gefundenen, relevanten Dokumente durch die Zahl der Gesamttreffermenge teilt. Je näher der Wert sich 1 nähert, desto präziser ist das Suchergebnis.

Tabelle 14 stellt die am 01.11.2016 erzielte Treffermenge und die Precision von zwei Stellenbörsen, Stepstone und die Stellenbörse der Zeit, für die ausgewählten Suchbegriffe gegenüber. Die Entscheidung, ob ein Treffer relevant ist oder nicht, wurde dabei anhand des Titels getroffen. Lediglich im Zweifelsfall wurde die Stellenausschreibung genauer angeschaut.

	ZeitOnline		StepStone	
Berufsbezeichnung	Treffermenge	Precision	Treffermenge	Precision
Archivar	1	0	8	0.63
Bibliothekar	1	1	10	0.8
Dokumentar	15	0.07	18	0.39
Informationsdesigner	0	-	7	0.57
Informationsmanager	2	0.5	45	0.07
Informationsspezialist	0	-	25	0.04
Informationswissenschaftler	0	-	7	0.29
Systembibliothekar	0	-	1	1
IT Bibliothekar	1	-	16246	0.0001
Technischer Bibliothekar	0	-	21966	0.0001

Tabelle 14: Vergleich Treffermenge - Precision Zeit und Stepstone (Suchergebnisse vom 01.11.2016)

Dieses Beispiel zeigt lediglich die Momentaufnahme eines Tages. Um ein aussagekräftiges Ergebnis zu erzielen, müssten Treffermenge und Precision mehrfach ausgewertet werden, um ausschließen zu können, dass diese Ergebnisse nur eine Ausnahme sind. Wäre diese Momentaufnahme repräsentativ, könnte man Aussagen über die einzelnen Suchbegriffe treffen. *IT Bibliothekar* und *Technischer Bibliothekar* erzielen bei beiden Stellenbörsen beispielsweise keine guten Ergebnisse, so dass auf diese verzichtet werden sollte. Die Treffermenge der Stellenbörse der Zeit hat zudem bei allen Suchbegriffen nur eine sehr geringe Treffermenge mit teilweise schlechter Precision erreicht, so dass zu überlegen wäre, die Zeit bei der Datensammlung auszuschließen.

5.2. Evaluation des Kategoriensystems

Für die Evaluation des Kategoriensystems empfiehlt es sich, die Inter-coder-Reliabilität zu testen. Dabei wird eine oder mehrere Personen hinzugezogen, die ebenfalls die gleichen Daten kategorisieren. Der Grad der Übereinstimmung trifft dabei eine Aussage über die Qualität der Kategorien (Mayring 2014, S. 107 ff.)

Des Weiteren kann man evaluieren, wie viele Kompetenzen von dem Kategoriensystem abgedeckt und gefunden werden. Da diese Methode darauf beruht, die Kompetenzen

zum einen durch das Entfernen irrelevanter Wörter und zum anderen durch das Auffinden relevanter Wörter zu identifizieren, kann die Menge der Wörter, die nach Stoppwortentfernung und Extraktion der Fähigkeiten mit dem Kategoriensystem noch übrig sind, Aufschluss über die Qualität von sowohl der Stoppwortliste als auch dem Kategoriensystem geben.

5.3. Feedbackschleifen

Bereits in Kapitel 4.3.2. wurde angedeutet, dass es wichtig ist, nach jedem Schritt die Qualität des Outputs zu evaluieren, um bei einer schlechten Qualität des Outputs den vorigen Schritt überprüfen und gegebenenfalls anpassen zu können. Dabei wird der IST-Zustand des Datenoutputs mit dem SOLL-Zustand verglichen. Weicht der IST-Zustand ab, sollte die Durchführung der letzten Schritte noch einmal begutachtet und gegebenenfalls optimiert werden.

Bei der Konzeptualisierung sollten von vorne herein Feedbackschleifen eingeplant werden. So sollten zum Beispiel nach Erstellung der Worthäufigkeiten geschaut werden, ob die Stoppwortliste noch erweitert und dementsprechend auch die Stoppwortentfernung wiederholt werden muss.

6. Zusammenfassung und Ausblick

In dieser Arbeit wurde aufgezeigt, wie mit der Programmiersprache Python und eines Kategoriensystems Kompetenzen aus Online-Stellenausschreibungen herausgelesen und für die Analyse aufbereitet werden können.

Fasst man die einzelnen Phasen der Methode zusammen (Abbildung 15), lassen sich Ähnlichkeiten mit den in Kapitel 3 besprochenen Vorgehensmodellen feststellen. Auch hier lässt sich ein Kreislauf bilden. Nach der Identifizierung der Kompetenzen folgt aus den weiteren Schritten ein Erkenntnisgewinn, der wiederum zu neuen Forschungsfragen führen kann.

Die entsprechenden Skripte zu den einzelnen Phasen befinden sich im Anhang.

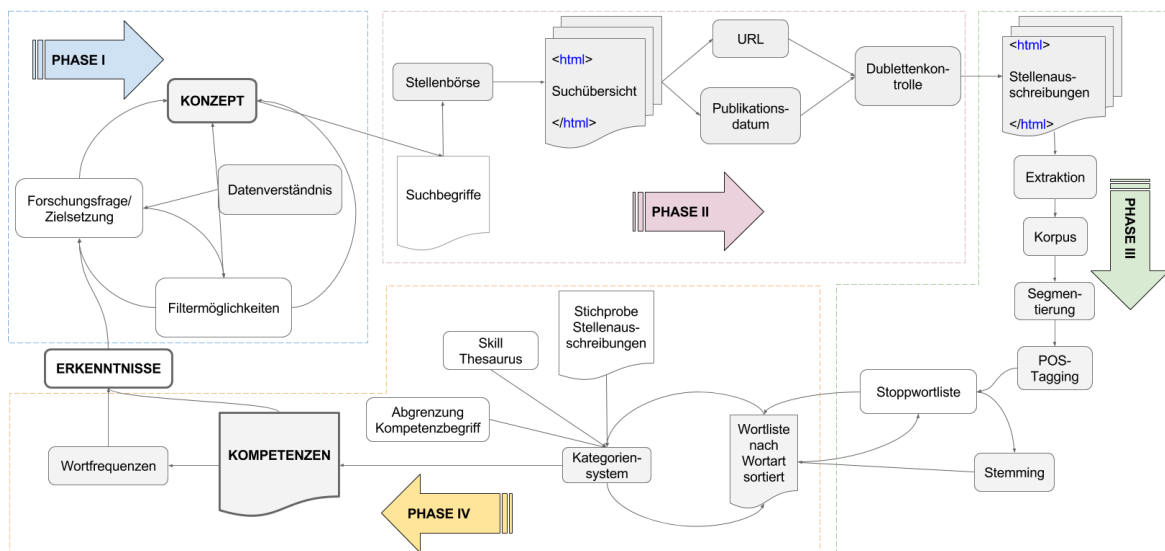


Abbildung 15: Gesamtdarstellung der entwickelten Methode

Mit der Identifizierung der Kompetenzen ist das eigentliche Vorhaben der Arbeit abgeschlossen. Im Anschluss müssten diese Ergebnisse noch visualisiert und angewendet werden.

Dazu eignen sich, je nach Zielsetzung und Zielgruppe, unterschiedliche Möglichkeiten, deren Erörterung den Umfang dieser Arbeit sprengen würden. Daher gibt es an dieser Stelle lediglich einen kurzen Ausblick auf mögliche Visualisierungs- und Anwendungsmöglichkeiten.

Für die Visualisierung bietet Python verschiedene Bibliotheken. Bokeh wäre eine davon, die zudem in der Anaconda Distribution enthalten ist. Mit der ist es möglich interaktive Grafiken zu erstellen. Man könnte zum Beispiel die unterschiedlichen Kompetenzschwerpunkte in den einzelnen Berufen visualisieren.

Die Anwendung der Ergebnisse ist in der Regel schon in der Zielsetzung definiert, beispielsweise das Abgleichen und Anpassen des Curriculums eines Studiengangs.

Durch die gewonnenen Erkenntnisse können sich zudem auch neue Sichtweisen auf die Problematik, das Themengebiet oder das Berufsfeld erschließen, was wiederum zu neuen Forschungsfragen führt.

Das Studierendenprojekt *Digitale Profile* wird mit dieser hier entwickelten Methode und den daraus entstandenen Skripten, eine prototypische Applikation entwickeln, die Berufsprofile für Studierende des Masterstudiengangs *Digitale Kommunikation* erstellt. Einzelne Elemente der in dieser Arbeit entstandenen Methode wurden dazu bereits verwendet. Das unterschiedliche Berufsfeld bereitete dabei keine Probleme, lediglich das Kategoriensystem musste individuell erstellt werden.

Eine umfassende Evaluation dieser Methode ist allerdings noch nicht möglich, da die Studierenden des Projektes zum Zeitpunkt der Abgabe dieser Bachelorarbeit noch nicht alle Phasen durchlaufen hatten. Erste Beobachtungen lassen aber darauf schließen, dass die Methode gut angenommen wird und die Arbeit mit den Jupyter Notebooks das Erlernen der Programmiersprache erleichtert.

Literaturverzeichnis

3S UNTERNEHMENSBERATUNG, 2016. *DISCO-Projekt-Informationen* [online]. Wien: 3s Unternehmensberatung. [Zugriff am: 08.10.2016]. Verfügbar unter: http://disco-tools.eu/disco2_portal/projectInformation.php

ABDALLA, Safia, 2015. *How will the children of the future learn about science?* [online]. Raleigh (North Carolina): Red Hat, Inc. [Zugriff am: 07.10.2016]. Verfügbar unter: <https://opensource.com/education/15/11/project-jupyter-science-notebooks>

AHMED, Shamima, 2005. Desired competencies and job duties of non-profit CEOs in relation to the current challenges. In: *Journal of Management Development* [online]. 24(10), S. 913-928 [Zugriff am: 06.10.2016]. ISSN 0262-1711. Verfügbar unter: DOI: [10.1108/02621710510627055](https://doi.org/10.1108/02621710510627055)

BACKS, Anna, Alexander KÜSTERS, Michael PONGRATZ und Michel SCHMIDT, 2014. *Analyseverfahren im Text Mining* [online]. Düsseldorf: Hochschule für Ökonomie und Management. [Zugriff am: 17.10.2016]. Verfügbar unter: [http://winfwiki.wi-fom.de/index.php/Analyseverfahren im Text Mining](http://winfwiki.wi-fom.de/index.php/Analyseverfahren_im_Text_Mining)

BENSBERG, Frank und Gandalf BUSCHER, 2016. Job Mining als Analyseinstrument für das Human-Resource-Management. In: *HMD Praxis der Wirtschaftsinformatik* [online]. Keiner Ausgabe zugewiesen. [Zugriff am: 17.10.2016]. ISSN 2198-2775. Verfügbar unter: DOI: [10.1365/s40702-016-0256-3](https://doi.org/10.1365/s40702-016-0256-3)

BERKELEY INSTITUTE FOR DATA SCIENCE, 2016. *Project Jupyter* [online]. Berkeley (California): Berkeley Institute for Data Science. [Zugriff am: 07.10.2016]. Verfügbar unter: <https://bids.berkeley.edu/research/project-jupyter>

BIBLIOGRAPHISCHES INSTITUT GMBH, 2016. *Korpus, das* [online]. Berlin: Bibliographisches Institut GmbH. [Zugriff am: 30.10.2016]. Verfügbar unter: http://www.duden.de/rechtschreibung/Korpus_Sammlung

BIRD, Steven, Ewan KLEIN und Edward LOPER, 2009. *Natural Language Processing with Python*. Sebastopol (California): O'Reilly. ISBN 978-0-596-51649-9

BUNDESAGENTUR FÜR ARBEIT, 2014. *Klassifikation der Berufe* [online]. Nürnberg: Bundesagentur für Arbeit. [Zugriff am: 13.10.2016]. Verfügbar unter: <https://statistik.arbeitsagentur.de/Navigation/Statistik/Grundlagen/Klassifikation-der-Berufe/Klassifikation-der-Berufe-Nav.html>

BUNDESAGENTUR FÜR ARBEIT, 2016. *Systematik und Verzeichnisse der KldB 2010* [online]. Nürnberg: Bundesagentur für Arbeit. [Zugriff am: 13.10.2016]. Verfügbar unter: <https://statistik.arbeitsagentur.de/Navigation/Statistik/Grundlagen/Klassifikation-der-Berufe/KldB2010/Systematik-Verzeichnisse/Systematik-Verzeichnisse-Nav.html>

BUNDESINSTITUT FÜR BERUFSBILDUNG, 2016. *Definition und Kontextualisierung des Kompetenzbegriffes* [online]. Bonn: Bundesinstitut für Berufsbildung. [Zugriff am: 08.10.2016]. Verfügbar unter: <https://www.bibb.de/de/8570.php>

CAMBRIDGE UNIVERSITY PRESS, 2008. *Stemming and lemmatization* [online]. Cambridge: Cambridge University Press [Zugriff am: 06.11.2016]. Verfügbar unter: nlp.stanford.edu/IR-book/html/htmledition/stemming-and-lemmatization-1.html

CARNEVALE, Anthony P., Tamara JAYASUNDERA und Dmitri REPNIKOV, 2014. *Understanding online job ads data* [online]. A technical report. Washington D.C.: Georgetown University [Zugriff am: 05.11.2016]. Verfügbar unter: https://cew.georgetown.edu/wp-content/uploads/2014/11/OCLM.Tech_Web_.pdf

CHAPMAN, Pete et al., 2000. *CRISP-DM 1.0* [online]. *Step-by-step data mining guide*. [Zugriff am: 09.10.2016]. Verfügbar unter: <ftp://ftp.software.ibm.com/software/analytics/spss/support/Modeler/Documentation/14/UserManual/CRISP-DM.pdf>

CONTINUUM ANALYTICS INC., 2016. *Anaconda documentation* [online]. *Anaconda distribution*. Austin (Texas): Continuum Analytics Inc. [Zugriff am: 06.10.2016]. Verfügbar unter: <https://docs.continuum.io/anaconda/>

DEBORTOLI, Stefan, Oliver MÜLLER und Jan vom BROCKE, 2014. Vergleich von Kompetenzanforderungen an Business-Intelligence- und Big-Data-Spezialisten: eine Text-Mining-Studie auf Basis von Stellenausschreibungen. In: *Wirtschaftsinformatik* [online]. **56**(5), S. 315-328 [Zugriff am: 03.11.2016]. ISSN 1861-8936. Verfügbar unter: DOI: [10.1007/s11576-014-0432-4](https://doi.org/10.1007/s11576-014-0432-4)

EUROPÄISCHE KOMMISSION BILDUNG UND KULTUR, 2008. *Der europäische Qualifikationsrahmen für lebenslanges Lernen* [online]. Luxemburg: Europäische Gemeinschaften. [Zugriff am: 08.10.2016]. ISBN 978-92-79-08472-0. Verfügbar unter: https://ec.europa.eu/ploteus/sites/eac-eqf/files/broch_de.pdf

FELDMAN, Ronen und James SANGER, 2007. *The text mining handbook* [online]. *Advanced approaches in analyzing unstructured data*. Cambridge: Cambridge University Press. ISBN 978-0-511-33507-5. Verfügbar unter: <http://www.roelsbeestenboel.nl/text.pdf>

FILß, Christian, 2005. *Vergleichsmethoden für Vorgehensmodelle* [Diplomarbeit]. Dresden: Technische Universität. Verfügbar unter: <ftp://ftp.uni-kl.de/pub/v-modell-xt/Release-1.2/Infomaterial/Diplomarbeit-von-Christian-Filss.pdf>

GAO, Lu und Neil ELDIN, 2014. Employer's expectations: a probabilistic text mining model. In: *Procedia Engineering* [online]. **85**(90), S. 175-182 [Zugriff am: 17.10.2016]. ISSN 1877-7058. Verfügbar unter: DOI: [10.1016/j.proeng.2014.10.542](https://doi.org/10.1016/j.proeng.2014.10.542)

HARPER, Ray, 2012. The collection and analysis of job advertisements: a review of research methodology. In: *Library and Information Research* [online]. **36**(112), S. 29-54 [Zugriff am: 02.10.2016]. ISSN 1756-1086. Verfügbar unter: <http://www.lirjournal.org.uk/lir/ojs/index.php/lir/article/view/499>

HEINDL, Andreas, 2015. Inhaltsanalyse. In: Achim HILDEBRANDT et al., Hrsg. *Methodologie, Methoden, Forschungsdesign*. Wiesbaden: Springer Fachmedien, S. 299-333. ISBN 978-3-531-18256-8

HIPPNER, Hajo und René RENTZMANN, 2006. Text Mining. In: *Informatik Spektrum* [online]. **29**(4), S. 287-290 [Zugriff am: 02.10.2016]. ISSN 1432-122X. Verfügbar unter: DOI: [10.1007/s00287-006-0091-y](https://doi.org/10.1007/s00287-006-0091-y)

HOCHSCHULE FÜR TELEKOMMUNIKATION LEIPZIG, 2016. *JobMining@HfTL* [online]. Ein innovatives Projekt zur Bildungsbedarfsanalyse von Unternehmen der ICT-Branche. Leipzig: Hochschule für Telekommunikation [Zugriff am: 01.10.2016]. Verfügbar unter: <https://www.hft-leipzig.de/de/forschung/akkordeon-forschungsprojekte/job-mininghftl.html>

JENSEN, Kenneth, 2012. *CRISP-DM process diagram* [online]. Wikimedia Commons [Zugriff am: 06.11.2016]. Verfügbar unter: https://commons.wikimedia.org/wiki/File:CRISP-DM_Process_Diagram.png

KONRAD, Markus, 2016. *Accurate Part-of-Speech tagging of German texts with NLTK* [online]. Berlin: Wissenschaftszentrum Berlin für Sozialforschung [Zugriff am: 30.10.2016]. Verfügbar unter: <https://datascience.blog.wzb.eu/2016/07/13/accurate-part-of-speech-tagging-of-german-texts-with-nltk/>

LACKES, Richard, 2016. *Data Mining* [online]. Wiesbaden: Springer Gabler Verlag [Zugriff am: 22.10.2016]. Verfügbar unter: <http://wirtschaftslexikon.gabler.de/Archiv/57691/data-mining-v8.html>

LAMPARTER, Anna, 2015. *Kompetenzprofil von Information Professionals in Unternehmen* [Masterarbeit]. Hannover: Hochschule. Verfügbar unter: <https://serwiss.bib.hs-hannover.de/frontdoor/index/index/docId/528>

MASSACHUSETTS INSTITUTE OF TECHNOLOGY, 2016. *MIT Subjects* [online]. *Electrical Engineering and Computer Science (Course 6)*. Cambridge (Massachusetts): Massachusetts Institute of Technology [Zugriff am: 06.10.2016]. Verfügbar unter: <http://catalog.mit.edu/subjects/6/>

MAYRING, Phillip, 2015. *Qualitative content analysis: Theoretical foundation, basic procedures and software solution*. Klagenfurt: Beltz.

McKinney, Wes, 2016. *Pandas* [online]. *Powerful Python data analysis toolkit*. [Zugriff am: 07.10.2016]. Verfügbar unter: <http://pandas.pydata.org/pandas-docs/stable/>

MEHRA, Shew-Ram und Kathrin DIETZ, 2015. Stellenanzeigenanalyse zur Ermittlung von zu vermittelnden Kompetenzen im Rahmen des neuen berufsbegleitenden Studiengangs „Master Online Akustik“ [online]. Oldenburg: Mint.Online [Zugriff am: 03.10.2016]. Verfügbar unter: https://de.mintonline.de/projekt/files/publikationen/MOA_Stellenanzeigen.pdf

MERRIAM-WEBSTER, INCORPORATED, 2015. *Natural language* [online]. Springfield (Massachusetts): Merriam-Webster, Incorporated [Zugriff am: 06.10.2016]. Verfügbar unter: <http://www.merriam-webster.com/dictionary/natural%20language>

MITCHELL, Ryan, 2015. *Web scraping with Python: Collecting data from the modern web*. Sebastopol (California): O'Reilly. ISBN 978-1-491-91027-6

ONPAGE.ORG GmbH, 2016. *Scraping* [online]. München: onPage.org GmbH [Zugriff am: 06.11.2016]. Verfügbar unter: <https://de.onpage.org/wiki/Scraping>

PIATETSKY, Gregory, 2014. *CRISP-DM, still the top methodology for analytics, data mining, or data science projects* [online]. [o.A.]: KDnuggets. [Zugriff am: 05.11.2016]. Verfügbar unter: <http://www.kdnuggets.com/2014/10/crisp-dm-top-methodology-analytics-data-mining-data-science-projects.html>

PROJECT JUPYTER, 2015. *The Jupyter Notebook* [online]. [o.A.]: Project Jupyter [Zugriff am: 07.10.2016]. Verfügbar unter: <http://jupyter-notebook.readthedocs.io/en/latest/notebook.html>

PYTHON SOFTWARE FOUNDATION, 2016. *General Python FAQ* [online]. Beaverton (Oregon): Python Software Foundation [Zugriff am: 06.10.2016]. Verfügbar unter: <https://docs.python.org/3/faq/general.html>

REITZ, Kenneth, 2016. *Requests* [online]. *HTTP for humans*. [Zugriff am: 05.11.2016]. Verfügbar unter: <http://docs.python-requests.org/en/latest/>

RICHARDSON, Leonard, 2015. *Beautiful Soup Documentation* [online]. [Zugriff am: 07.10.2016]. Verfügbar unter: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

SAILER, Maximilian, 2009. Anforderungsprofile und akademischer Arbeitsmarkt: Die Stellenanzeigenanalyse als Methode der empirischen Bildungs- und Qualifikationsforschung. Münster [u.a.]: Waxmann. Erziehung & Bildung. 3. ISBN 978-3-8309-2162-2

SEELIGER, Frank, Tracy HOFFMANN und Andrea Kiefer, 2016. Systembibliothekar, Bibliotheksinformatiker, IT-Bibliothekar - lässt sich dieses Anforderungsprofil akademisieren für eine Klientel im Berufsstand [online]. Reutlingen: Berufsverband Information Bibliothek [Zugriff am: 03.11.2016]. Verfügbar unter: <https://opus4.kobv.de/opus4-bib-info/frontdoor/index/index/docId/2603>

SCHIEBER, Andreas und Andreas HILBERT, 2014. Entwicklung eines generischen Vorgehensmodells für Text Mining. In: *Dresdner Beiträge zur Wirtschaftsinformatik* [online]. Nr. 69. [Zugriff am: 17.10.2016] ISSN 0945-4837. Verfügbar unter: [http://www.qucosa.de/fileadmin/data/qucosa/documents/14137/Schieber Hilbert Entwicklung%20eines%20generischen%20Vorgehensmodells%20f%C3%BCr%20Text%20Mining.pdf](http://www.qucosa.de/fileadmin/data/qucosa/documents/14137/Schieber_Hilbert_Entwicklung%20eines%20generischen%20Vorgehensmodells%20f%C3%BCr%20Text%20Mining.pdf)

SPRINGER GABLER VERLAG (Hrsg.), 2016a. *Fachkompetenz* [online]. Wiesbaden: Springer Gabler Verlag. [Zugriff am: 08.10.2016]. Verfügbar unter: <http://wirtschaftslexikon.gabler.de/Archiv/85641/fachkompetenz-v7.html>

SPRINGER GABLER VERLAG (Hrsg.), 2016b. *Sozialkompetenz* [online]. Wiesbaden: Springer Gabler Verlag. [Zugriff am: 08.10.2016]. Verfügbar unter: <http://wirtschaftslexikon.gabler.de/Archiv/85643/sozialkompetenz-v7.html>

SPRINGER GABLER VERLAG (Hrsg.), 2016c. *Methodenkompetenz* [online]. Wiesbaden: Springer Gabler Verlag. [Zugriff am: 08.10.2016]. Verfügbar unter: <http://wirtschaftslexikon.gabler.de/Archiv/85642/methodenkompetenz-v9.html>

SPRINGER GABLER VERLAG (Hrsg.), 2016d. *Handlungskompetenz* [online]. Wiesbaden: Springer Gabler Verlag. [Zugriff am: 08.10.2016]. Verfügbar unter: <http://wirtschaftslexikon.gabler.de/Archiv/85640/handlungskompetenz-v7.html>

SWEIGART, Al, 2015. *Automate the boring stuff with Python* [online]. 2. Auflage. San Francisco (California): No Starch Press. [Zugriff am: 05.11.2016]. Verfügbar unter: <https://automatetheboringstuff.com/>

WAGNER, Fred (Hrsg.), 2016. *Definition Handlungskompetenz* [online]. Wiesbaden: Springer Fachmedien. [Zugriff am: 08.10.2016]. Verfügbar unter: <http://www.versicherungsmagazin.de/Definition/35166/handlungskompetenz.html>

WAGNER, Fred (Hrsg.), 2016a. *Definition Persönlichkeitskompetenz* [online]. Wiesbaden: Springer Fachmedien. [Zugriff am: 08.10.2016]. Verfügbar unter: <http://www.versicherungsmagazin.de/Definition/35170/persoendlichkeitskompetenz.html>

YANG, Qinghong et al., 2016. Current market demand for core competencies of librarianship: a text mining study of American Library Association's advertisements from 2009 to 2014. In: *Applied Sciences* [online]. **6**(2), S. 1-16 [Zugriff am: 03.11.2016]. ISSN 2076-3417. Verfügbar unter: DOI: [10.3390/app6020048](https://doi.org/10.3390/app6020048)

Anhang

Anhang 1: Beigabe (CD)

Inhalt der CD:

1. Die Bachelorarbeit als PDF-Version
2. Excel-Dateien der POS-getaggten Adjektive, Verben und Nomen
3. Die Skripte als ipynb-Datei

Anhang 2: Skripte zur Phase II – Datensammlung

Anhang 2.1.: Data Collection - Stepstone

In this script, you will search Stepstone for job advertisements, get the urls to the ads and save the job description as html-file.

Overview

#	Steps	Input Adjustable	Required	Input: Format & Variables	Output: Format & Variables	Next Steps
0.	Setup					
0.1.	Import Libraries	---	yes	---	---	0.2
0.2.	Define Functions	---	yes	---	---	1
1.	Load Searchterms	yes	yes	Spreadsheed File-ID	List of Searchterms	2
2.	Scrape and Save Searchresults	yes	yes	List of Searchterms, URL to Job Board, Filepath	Searchresults as HTML-File	3
3.	Extract URLs to Job Ads from Searchresult-Page	yes	---	Path to Searchresult-Files	Dictionary of URLs and Searchterms	4
4.	Scrape Job Advertisement and Save Page as HTML	yes	yes	path, dictionary	Job Ads as HTML-File	---

0. Setup

0.1. Import Libraries

The **os** library is used to navigate in your directory and select the folder containing the documents needed. With **time** you can create a timestamp. In this script the timestamp is used as part of the filename to make it unique. **Requests** is a library to scrape websites, for parsing **BeautifulSoup** is used. With the **re** library you can detect patterns using regular expressions. In this script **pandas** is used to create a data frame from a Google spreadsheet. **Itertools** is used here to flatten nested lists.

```
import os, time, requests, re, pandas, itertools
from bs4 import BeautifulSoup
```

0.2. Define Functions

Before you start scraping Stepstone, you need to define some functions that are used in this script. All function-definitions need to be run before continuing, so you can call them when needed by passing the required parameters.

The `spreadsheet()`-function takes a file-id as parameter, exports the spreadsheet as csv-file and turns the csv into a data frame.

```
def spreadsheet(file_id):
    url = "https://docs.google.com/spreadsheets/d/{0}/export?format=csv"
    ".format(file_id)
    df = pandas.read_csv(url)
    return(df)
```

The `search_results()`-function takes searchterms and the amount of searchresult-pages as parameter. With these parameters, the function creates and scrapes the url for each searchresult-page.

```
def search_results(base_url, searchterms, pages):
    header = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; WOW64; rv:49.0) Gecko/20100101 Firefox/49.0'}
    pages = (requests.get(base_url, headers = header, params={'keywords': searchterms, 'sort': 'date', 'page': page+1}) for page in range(pages))
    return(pages)
```

The `save_result()`-function takes the content you want to save and saves it at the path where you want your files to be located under the given filename and its extension (e.g. html or txt).

```
def save_result(content, path, filename, extension):
    file = ''.join([path, filename, '.', extension])
    with open(file, 'w', encoding = 'utf-8') as f:
        f.write(content)
```

2. Load Searchterms

In this section, you call the previously defined `spreadsheet()`-function by passing the file-id as string in the brackets. As you have seen when defining the function, the output is data frame. As second step, you select the data frame column with the respective search terms for Stepstone and convert the search terms to strings by passing each of them to a list.

```
df = spreadsheet("1Ip0tQ2rwpUWw-FZ4BX9_LwJe0HXtIzHRpdmKCITdaTs")
searchTerms = [term for term in df["Stepstone"]]
```

2. Scrape and Save Searchresults

First you need to pass the amount of searchresult-pages, that is found with every searchterm, to the dictionary with the variable **pages**.

Then the for-loop takes each searchterm, passes it with the corresponding pagenumber to the **search_results**-function before saving the result as html-file.

```
### USER INPUT ###
pages = {searchTerms[0]: 1, searchTerms[1]: 1, searchTerms[2]: 1, searchTerms[3]: 1, searchTerms[4]: 1, searchTerms[5]: 1, searchTerms[6]: 1, searchTerms[7]: 1}
url = 'https://www.stepstone.de/5/ergebnisliste.html?'
path = 'G:/BA/Stellenbeschreibungen/Corpora/Suchuebersicht/'

### STANDARD ###
for term in searchTerms[:7]:
    for result in search_results(url, term, pages[term]):
        save_result(result.text, path, 'stepstone_search_result' + str(
            time.time()), 'html')
        time.sleep(3)
```

3. Extract URLs to Job Ads from Searchresult-Page

To extract the URLs to get to the actual job advertisements, you first take the path where you saved the html-files at, and pass it to the **searchResults**-variable. Second you create an empty dictionary in the **searchTerms_dict**-variable. This is to make sure, that you do not save job ads twice that are found with two different searchterms.

Every file in the folder whose filename contains **stepstone_**, will then be opened and turned into a BeautifulSoup-object. The **query**-variable contains the searchterm which was used to find the results. They will be later used to create the filename and they will also be appended to the metadata of each job description.

Then you extract the URL to each job advertisement using BeautifulSoup's **find_all**-function. You simply pass the tag and the attributes to the function and then extract the **href**-part of each result. As URLs are unique, you then take each URL and check whether it already is in the **searchTerms_dict**-dictionary. If this is not the case, you append the URL plus the searchterm. Else, you only append the searchterm to the respective URL.

```

searchResults = os.listdir(path)
searchTerms_dict = {}

for files in searchResults:
    if files.find("stepstone_") != -1:
        with open(path + files, 'r', encoding="utf8") as jobs:
            soup = BeautifulSoup(jobs, "lxml")
            query_sent = soup.find('p', attrs = {'class' : 'mb--xxs'})
            query = re.findall('<strong>(.*?)</strong> ergab', str(query
            _sent))

            job_urls = soup.find_all('div', attrs = {'class' : 'jobitem
            __header'})

            dates = soup.find_all('time')

            for d in dates:
                date = d.get('datetime')

            for link in job_urls:
                links = re.findall('href=\"(.*?)\">', str(link))
                links = ''.join(links)

                if links not in searchTerms_dict:
                    searchTerms_dict[links] = [query]

                else:
                    if query not in searchTerms_dict[links]:
                        searchTerms_dict[links].append(query)

```

4. Scrape Job Advertisement and Save Page as HTML

For this last step, you take each URL from the dictionary and join it with its domain. This needs to be done, because the URL in the dictionary is a relative URL. Afterwards, you take the URL and send another get-request to the server. If any problems arise, e.g. the job ad is offline already, the script will print an error message.

To create the filename, you take the values (searchterms) from the dictionary and turn it into a list, before replacing whitespace with an underscore. Then you add the name of the job board and a timestamp to the filename and determine the path where you want to save the files.

Last, but not least, you save the content using the **save_result**-function again.

```

### USER INPUT ###
finalPath = 'G:/BA/Stellenbeschreibungen/Corpora/html/'

### STANDARD ###
for rel_url in searchTerms_dict.keys():
    url = 'http://www.stepstone.de' + rel_url

```

```

print(url)
header = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; WOW64; rv:49
.0) Gecko/20100101 Firefox/49.0'}
response = requests.get(url, headers = header)

try:
    response.raise_for_status()

except Exception as exc:
    print('There was a problem: %s' % (exc))

webContent = response.text
search = list(itertools.chain.from_iterable(searchTerms_dict[rel_ur
l]))
search = ','.join(search)
search = re.sub('\s|-', '_', search)
print(search)

filename = search + '_Stepstone_' + str(time.time())
print(filename)

save_result(webContent, finalPath, filename, 'html')

time.sleep(3)

```

This code is for the notebook-design only, feel free to ignore it.

```

%%html
<style>
table {float:left}
</style>

```

Anhang 2.2.: Data Collection – Zeit Online

This notebook guides you through scraping the jobboard of Zeit Online and saving the search-result pages as well as the job advertisements.

Overview

#	Steps	Input Adjustable	Required	Input: Format & Variables	Output: Format & Variables	Next Steps
0.	Setup					
0.1.	Import Libraries	---	yes	---	---	0.2
0.2.	Define Functions	---	yes	---	---	1
1.	Load Searchterms	yes	yes	Spreadsheet File-ID	List of Searchterms	2
2.	Scrape and Save Searchresults	yes	yes	List of Searchterms, URL to Job Board, Filepath	Searchresults as HTML-File	3
3.	Extract URLs to Job Ads from Searchresult-Page	yes	---	Path to Searchresult-Files	Dictionary of URLs and Searchterms	4
4.	Scrape Job Advertisement and Save Page as HTML	yes	yes	path, dictionary	Job Ads as HTML-File	---

0. Setup

0.1. Import Libraries

The **os** library is used to navigate in your directory and select the folder containing the documents needed. With **time** you can create a timestamp. In this script the timestamp is used as part of the filename to make it unique. **Requests** is a library to scrape websites, for parsing **BeautifulSoup** is used. With the **re** library you can detect patterns using regular

expressions. In this script **pandas** is used to create a data frame from a Google spreadsheet. **Itertools** is used here to flatten nested lists.

```
import os, time, requests, re, pandas, itertools
from bs4 import BeautifulSoup
```

0.2. Define Functions

In this section we will define three functions. The first one is used to load a spreadsheet as data frame into the script. The second one is used to send a query to the server of a job board and the third function saves the result you get back from the server, e.g. the search-result pages.

Load Google Spreadsheet

This function takes the file-id of a Google Spreadsheet as parameter and turns it into a data frame.

```
def spreadsheet(file_id):
    url = "https://docs.google.com/spreadsheets/d/{0}/export?format=csv"
    url = url.format(file_id)
    df = pandas.read_csv(url)
    return df
```

Scrape Job Board

This function takes the searchterms (keywords) and the amount of pages found with the respective searchterm as parameter and sends a request to the server of the job board using the parameters.

```
def search_results(base_url, keywords, pages):
    header = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; WOW64; rv:49.0) Gecko/20100101 Firefox/49.0'}
    pages = (requests.get(base_url, headers=header, params={'keywords': keywords, 'pageOffset': page+10}) for page in range(pages))
    return pages
```

Save Results

This function can be used to save the results from scraping the job board. It takes the content (what you want to save), the path to the directory where you want to save the files, the filename and the extension that determines the format your data will be saved in, as parameters. The function then joins path, filename and extension and saves the content with utf-8 encoding.

```
def save_result(content, path, filename, extension):
    file = ''.join([path, filename, '.', extension])
    with open(file, 'w', encoding='UTF-8') as f:
        f.write(content)
```

1. Load Searchterms

To load the searchterms from a Google Spreadsheet, we take the **spreadsheet**-function and pass the file-id of the respective spreadsheet as parameter. You can find the file-id in the URL right after the **/dl**-part. Make sure to use the URL of the share link, otherwise you might get an error message.

Second, we select the column of the job board we want to scrape by simply passing the column-title in squared brackets. This will result in a pandas-series-object, so we need to pass every term in that column to a list called **terms** to be able to work with the searchterms.

```
### USER INPUT ###
fileID = "1Ip0tQ2rwpUWw-FZ4BX9_LwJe0HXtIzHRpdmKCITdaTs"

### STANDARD ###
df = spreadsheet(fileID)
terms = [term for term in df["Zeit"]]
```

2. Scrape and Save Searchresults

First you need to pass the amount of searchresult-pages, that is found with every searchterm, to the dictionary with the variable **pages**.

Then the for-loop takes each searchterm, passes it with the corresponding pagenumber to the **search_results**-function before saving the result as html-file.

```
### USER INPUT ###
pages = {terms[0]: 1, terms[1]: 1, terms[2]: 1, terms[3]: 1, terms[4]:
1, terms[5]: 1, terms[6]:1, terms[7]:1, terms[8]:1, terms[9]:1}
url = 'http://jobs.zeit.de/action/av/search/withAgents?searchString='
path = 'G:/BA/Stellenbeschreibungen/Corpora/Suchuebersicht/'

### STANDARD ###
for term in terms:
    for result in search_results(url, term, pages[term]):
        urls = result.url
        save_result(result.text, path, 'Zeit_' + 'search_result_' + term +
str(time.time()), 'html')
        time.sleep(3)
```

3. Extract URLs to Job Ads from Searchresult-Page

To extract the URLs to get to the actual job advertisements, you first take the path where you saved the html-files at, and pass it to the **searchResults**-variable. Second you create an empty dictionary in the **searchTerms_dict**-variable. This is to make sure, that you do not save job ads twice that are found with two different searchterms.

Every file in the folder whose filename contains **Zeit_**, will then be opened and turned into a BeautifulSoup-object. The **query**-variable contains the searchterm which was used to find the results. They will be later used to create the filename and they will also be appended to the metadata of each job description.

Then you extract the URL to each job advertisement using BeautifulSoup's **find_all**-function. You simply pass the tag and the attributes to the function and then extract the **href**-part of each result. As URLs are unique, you then take each URL and check whether it already is in the **searchTerms_dict**-dictionary. If this is not the case, you append the URL plus the searchterm. Else, you only append the searchterm to the respective URL.

```
searchResults = os.listdir(path)

#empty dictionaries to check and avoid duplicates
searchTerms_dict = {}

for files in searchResults:
    if files.find("Zeit_") != -1:
        with open(path + files, 'r', encoding="utf8") as jobs:
            soup = BeautifulSoup(jobs, "lxml")

            #extracts searchterm to later include it in filename
            query = re.findall('Zeit_search_result_([A-Za-z]*\s?[A-Za-z]*\s?[A-Za-z]*)\d+', str(files))

            #extracts urls
            job_urls = soup.find_all('a', attrs={'class' : 'jobTitle'})

            for link in job_urls:
                links = link.get('href')

                if links not in searchTerms_dict:
                    searchTerms_dict[links] = [query]

                else:
                    if query not in searchTerms_dict[links]:
                        searchTerms_dict[links].append(query)
```

4. Scrape Job Advertisement and Save Page as HTML

For this last step, you take each URL from the dictionary and join it with its domain. This needs to be done, because the URL in the dictionary is a relative URL. Afterwards, you take the URL and send another get-request to the server. If any problems arise, e.g. the job ad is offline already, the script will print an error message.

To create the filename, you take the values (searchterms) from the dictionary and turn it into a list, before replacing whitespace with an underscore. Then you add the name of the job board and a timestamp to the filename and determine the path where you want to save the files.

Last, but not least, you save the content using the **save_result**-function again.

```
### USER INPUT ###
finalPath = 'G:/BA/Stellenbeschreibungen/Corpora/html/'

### STANDARD ###
for items in searchTerms_dict.keys():
    url = ''.join(items)
    url = 'http://jobs.zeit.de' + url

    header = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; WOW64; rv:49.0) Gecko/20100101 Firefox/49.0'}
    response = requests.get(url, headers = header)

    try:
        response.raise_for_status()

    except Exception as exc:
        print('There was a problem: %s' % (exc))

    webContent = response.text

    search = list(itertools.chain.from_iterable(searchTerms_dict[items]
))
    search = '_'.join(search)
    search = re.sub('\s', '_', search)
    #creates the filename and places it in the new directory
    filename = search + '_Zeit_' + str(time.time())
    #saves the file
    save_result(webContent, finalPath, filename, 'html')
    #pauses for 3 second
    time.sleep(3)
```

This code is for the notebook-design only, feel free to ignore it.

```
%%html
<style>
table {float:left}
</style>
```

Anhang 2.3.: Data Collection – Xing

In this script, you will search Xing for job advertisements, get the urls to the ads and save the job description as html-file.

Overview

#	Steps	Input Adjustable	Required	Input: Format & Variables	Output: Format & Variables	Next Steps
0.	Setup					
0.1.	Import Libraries	---	yes	---	---	0.2
0.2.	Define Functions	---	yes	---	---	1
1.	Load Searchterms	yes	yes	Spreadsheet File-ID	List of Searchterms	2
2.	Scrape and Save Searchresults	yes	yes	List of Searchterms, URL to Job Board, Filepath	Searchresults as HTML-File	3
3.	Extract URLs to Job Ads from Searchresult-Page	yes	---	Path to Searchresult-Files	Dictionary of URLs and Searchterms	4
4.	Scrape Job Advertisement and Save Page as HTML	yes	yes	path, dictionary	Job Ads as HTML-File	---

0. Setup

0.1. Import Libraries

The **os** library is used to navigate in your directory and select the folder containing the documents needed. With **time** you can create a timestamp. In this script the timestamp is used as part of the filename to make it unique. **Requests** is a library to scrape websites, for parsing **BeautifulSoup** is used. With the **re** library you can detect patterns using regular

expressions. In this script **pandas** is used to create a data frame from a Google spreadsheet. **Itertools** is used here to flatten nested lists.

```
import os, time, requests, re, pandas, itertools
from bs4 import BeautifulSoup
```

0.2. Define Functions

In this section we will define three functions. The first one is used to load a spreadsheet as data frame into the script. The second one is used to send a query to the server of a job board and the third function saves the result you get back from the server, e.g. the search-result pages.

Load Google Spreadsheet

This function takes the file-id of a Google Spreadsheet as parameter and turns it into a data frame.

```
def spreadsheet(file_id):
    url = "https://docs.google.com/spreadsheets/d/{0}/export?format=csv"
    url = url.format(file_id)
    df = pandas.read_csv(url)
    return df
```

Scrape Job Board

This function takes the searchterms (keywords) and the amount of pages found with the respective searchterm as parameter and sends a request to the server of the job board using the parameters.

```
def search_results(base_url, keywords, pages):
    header = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; WOW64; rv:49.0) Gecko/20100101 Firefox/49.0'}
    pages = (requests.get(base_url, params={'keywords': keywords, 'sort': 'date', 'page': page+1}) for page in range(pages))
    return pages
```

Save Results

This function can be used to save the results from scraping the job board. It takes the content (what you want to save), the path to the directory where you want to save the files, the filename and the extension that determines the format your data will be saved in, as parameters. The function then joins path, filename and extension and saves the content with utf-8 encoding.

```
def save_result(content, path, filename, extension):
    file = ''.join([path, filename, '.', extension])
    with open(file, 'w', encoding='UTF-8') as f:
```

```
f.write(content)
```

1. Load Searchterms

To load the searchterms from a Google Spreadsheet, we take the **spreadsheet**-function and pass the file-id of the respective spreadsheet as parameter. You can find the file-id in the URL right after the **/d/**-part. Make sure to use the URL of the share link, otherwise you might get an error message.

Second, we select the column of the job board we want to scrape by simply passing the column-title in squared brackets. This will result in a pandas-series-object, so we need to pass every term in that column to a list called **terms** to be able to work with the searchterms.

```
### USER INPUT###
fileID = "1Ip0tQ2rwpUWw-FZ4BX9_LwJe0HXtIzHRpdmKCITdaTs"

### STANDARD ###
df = spreadsheet()
terms = [term for term in df["Xing"]]
```

2. Scrape and Save Searchresults

First you need to pass the amount of searchresult-pages, that is found with every searchterm, to the dictionary with the variable **pages**.

Then the for-loop takes each searchterm, passes it with the corresponding pagenumber to the **search_results**-function before saving the result as html-file.

```
### USER INPUT ###
pages = {terms[0]: 5, terms[1]: 5, terms[2]: 10, terms[3]: 1, terms[4]:
2, terms[5]: 1, terms[6]: 10, terms[7]: 1, terms[8]: 1, terms[9]: 4}
base_url = 'https://www.xing.com/search/in/jobs/'
path = 'G:/BA/Stellenbeschreibungen/Corpora/Suchuebersicht/neu/'

### STANDARD ###
for term in terms:
    for result in search_results(term, pages[term]):
        urls = result.url
        save_result(result.text, path, 'xing_search_result' + str(time.
time()), 'html')
        time.sleep(3)
```

3. Extract URLs to Job Ads from Searchresult-Page

To extract the URLs to get to the actual job advertisements, you first take the path where you saved the html-files at, and pass it to the **searchResults**-variable. Second you create an empty dictionary in the **searchTerms_dict**-variable. This is to make sure, that you do not save job ads twice that are found with two different searchterms.

Every file in the folder whose filename contains **xing_**, will then be opened and turned into a BeautifulSoup-object. The **query**-variable contains the searchterm which was used to find the results. They will be later used to create the filename and they will also be appended to the metadata of each job description.

Then you extract the URL to each job advertisement using BeautifulSoup's **find_all**-function. You simply pass the tag and the attributes to the function and then extract the **href**-part of each result. As URLs are unique, you then take each URL and check whether it already is in the **searchTerms_dict**-dictionary. If this is not the case, you append the URL plus the searchterm. Else, you only append the searchterm to the respective URL. The second measure to make sure not to save duplicates is that you pass the date when you last saved job ads from Xing to the **lastSaved**-variable. Then the program extracts the upload date from the job ad and checks whether the job ad was uploaded after you last saved an ad from Xing.

```
### USER INPUT ###
lastSaved = 20160923232100 #yyyymmddhhmmss

### STANDARD ###
searchResults = os.listdir(path)

#empty dictionaries for urls + searchterms and duplicate urls + searchterms
searchTerms_dict = {}

for files in searchResults:
    if files.find("xing_") != -1:
        with open(path + files, 'r', encoding="utf8") as jobs:
            soup = BeautifulSoup(jobs, "lxml")

            #extracts searchterm and pagenumber of searchresult-page to
            later include it in filename
            title_tag = soup.find('title')
            query = re.findall('<title>Jobs mit diesen Schlagwörtern:\s
            (.*)', str(title_tag))

            #extracts urls to job ads
            job_urls = soup.find_all('a', attrs = {'class' : 'title job
            -posting-link name-page-link'})

            #extracts date
            dates = soup.find_all('time', attrs = {'class' : 'date'})
            for d in dates:
                date = d.get('content')
                date = re.sub('-|T|:|Z', '', date)

                if int(date) > lastSaved:
                    for link in job_urls:
                        links = link.get('href')
                        links = re.sub('\?.*', '', links)
```

```

if links not in searchTerms_dict:
    searchTerms_dict[links] = [query]
    print(links + ' ' + date)

else:
    if query not in searchTerms_dict[links]:
        searchTerms_dict[links].append(query)

```

4. Scrape Job Advertisement and Save Page as HTML

For this last step, you take each URL from the dictionary and send another get-request to the server. If any problems arise, e.g. the job ad is offline already, the script will print an error message.

To create the filename, you take the values (searchterms) from the dictionary and turn it into a list, before replacing whitespace with an underscore. Then you add the name of the job board and a timestamp to the filename and determine the path where you want to save the files.

Last, but not least, you save the content using the **save_result**-function again.

```

### USER INPUT ###
finalPath = 'G:/BA/Stellenbeschreibungen/Corpora/html/'

for url in searchTerms_dict.keys():
    header = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; WOW64; rv:49.0) Gecko/20100101 Firefox/49.0'}
    response = requests.get(url, headers = header)

    try:
        response.raise_for_status()

    except Exception as exc:
        print('There was a problem: %s' % (exc))

    webContent = response.text
    search = list(itertools.chain.from_iterable(searchTerms_dict[url]))
    search = ','.join(search)
    search = re.sub('\s|-', '_', search)

    filename = search + '_Xing_' + str(time.time())

    save_result(webContent, finalPath, filename, 'html')

    time.sleep(3)

```

This code is for the notebook-design only, feel free to ignore it.

```
%%html
<style>
table {float:left}
</style>
```


Anhang 3: Skripte zur Phase III – Datenaufbereitung

Anhang 3.1. Job Ad Extraction - Stepstone

In this script, you will take the html files that you have saved before, extract meta data from it, narrow down the area around the actual jobdescription and save the document as txt file.

Overview

#	Steps	Required	Input	Output	Next Steps
1	Import Libraries	yes	---	---	2
2	Extract Meta Data and Jobdescription and Save File	yes	path	txt-file	---

1. Import Libraries

Os, **re** and **BeautifulSoup** have been used in the XingJobs script already, check the description to get more information on the libraries. **datetime** is another library to create a timestamp of the current date and time.

```
import os, datetime, re
from bs4 import BeautifulSoup
```

2. Extract Meta Data and Jobdescription and Save File

In this for loop, each html-file in the folder is opened, parsed with the lxml-parser and turned into a Beautiful Soup object. Using Beautiful Soup's find() and find_all()-functions, you then extract the given meta data and additional data that will be appended as meta data.

Afterwards, script- and style-tags are removed and the actual job description is being assigned to the variable *jobData*. You take the filename and extract the searchterms from it using regex and assign it to the variable *query*.

Before saving the file, you stitch together a filename consisting of the filename of the html-file, a new timestamp and a .txt extension and determine the directory where you want the files to be saved.

With the first *with open* part, you save the jobdescription to file. The second one opens the file again and appends the extracted meta data to the end of the file.

```
path = 'G:/BA/Stellenbeschreibungen/Corpora/html/'
searchResults = os.listdir(path)
```

```

for files in searchResults:
    if files.find('Stepstone') != -1:
        with open(path + files, 'r', encoding="utf8") as jobs:
            soup = BeautifulSoup(jobs, "lxml")

            title = soup.find('title')
            if title != None:
                title = title.get_text()

            keywords = soup.find('meta', attrs = {'name' : 'keywords'})
            website = soup.find('link', attrs = {'rel' : 'canonical'})
            uploadDate = soup.find('div', attrs = {'itemprop' : 'datePo
sted'})

            for script in soup(["script", "style"]):
                script.extract()

            jobData = soup.find('div', attrs = {'id' : 'Content'})

            if jobData != None:
                metaData = soup.find_all('meta')

                query = re.sub('_', ' ', files)
                query = re.sub('Stepstone|_d+|\d+|\.|html', '', query
)

                metaTag = soup.new_tag('meta')

                metaTag.attrs['downloadDate'] = datetime.datetime.today
().strftime("%Y%m%d_%H%M%S")
                metaTag.attrs['searchTerms'] = str(query)
                metaTag.attrs['title'] = str(title)
                metaTag.attrs['keywords'] = str(keywords)
                metaTag.attrs['website'] = str(website)
                metaTag.attrs['uploadDate'] = str(uploadDate)

                filenames = re.sub('\d+\.html', '', files)

                filename = filenames + '_' + str(datetime.datetime.toda
y().strftime("%Y%m%d_%H%M%S")) + '.txt'
                filePath = 'G:/BA/Stellenbeschreibungen/Corpora/txt/' +
filename

                with open(filePath, 'w', encoding='utf8') as f:
                    f.write(str(jobData) + ' ')

```

```
with open(filePath, 'a', encoding='utf8') as f:  
    f.write(str(metaData) + ' ' + str(metaTag))
```

```
%%html  
<style>  
table {float:left}  
</style>
```

Anhang 3.2. Job Ad Extraction – Zeit

In this script, you will take the html files that you have saved before, extract meta data from it, narrow down the area around the actual jobdescription and save the document as txt file.

Overview

#	Steps	Required	Input	Output	Next Steps
1	Import Libraries	required	---	---	2
2	Extract Meta Data and Jobdescription and Save File	required	path	txt-file	---

1. Import Libraries

Os, **re** and **BeautifulSoup** have been used in the XingJobs script already, check the description to get more information on the libraries. **datetime** is another library to create a timestamp of the current date and time.

```
import os, time, datetime, re
from bs4 import BeautifulSoup
```

2. Extract Meta Data and Jobdescription and Save File

In this for loop, each html-file in the folder is opened, parsed with the lxml-parser and turned into a BeautifulSoup object. Using BeautifulSoup's find() and find_all()-functions, you then extract the given meta data and additionaly data that will be appended as meta data.

Afterwards, script- and style-tags are removed and the actual job description is being assigned to the variable *jobData*. You take the filename and extract the searchterms from it using regex and assign it to the variable *query*.

Before saving the file, you stitch together a filename consisting of the filename of the html-file, a new timestamp and a .txt extension and determine the directory where you want the files to be saved.

With the first *with open* part, you save the jobdescription to file. The second one opens the file again and appends the extracted meta data to the end of the file.

```
### USER INPUT ###
path = 'G:/BA/Stellenbeschreibungen/Corpora/html/neu/'

### STANDARD ###
```

```

searchResults = os.listdir(path)

for files in searchResults:
    if files.find('Zeit') != -1:
        with open(path + files, 'rb') as jobs:
            soup = BeautifulSoup(jobs, "lxml")

            #Extracts title, keywords, URL and Uploaddate
            title = soup.find('title')

            if title != None:
                title = title.get_text()

            keywords = soup.find('meta', attrs = {'name' : 'keywords'})
            website = soup.find('link', attrs = {'rel' : 'canonical'})
            uploadDate = soup.find('span', attrs = {'class' : 'job_publication_date'})

            if uploadDate != None:
                uploadDate = uploadDate.get_text()

            #removes script- and style-tags
            for script in soup(["script", "style"]):
                script.extract()

            #Extracts job description
            jobData = soup.find('div', attrs = {'id' : 'jobsDetails'})

            if jobData != None:
                #find meta tags and creates new meta tags for download date, searchterms and title
                metaData = soup.find_all('meta')

                metaTag = soup.new_tag('meta')
                metaTag.attrs['downloadDate'] = datetime.datetime.today().strftime("%Y%m%d-%H%M%S")

                query = re.sub('_', ' ', files)
                query = re.sub('Zeit|_d+|_d+|_html', '', query)

                metaTag.attrs['searchTerms'] = str(query)
                metaTag.attrs['title'] = str(title)
                metaTag.attrs['keywords'] = str(keywords)
                metaTag.attrs['website'] = str(website)
                metaTag.attrs['uploadDate'] = str(uploadDate)

                filenames = re.sub('_d+_html', '', files)

```

```

#create the filename and place it in the new directory
filename = filenames + '_' + str(time.time()) + '.txt'
filePath = 'G:/BA/Stellenbeschreibungen/Corpora/txt/' +

filename

#save the file
with open(filePath, 'w', encoding='utf8') as f:
    f.write(str(jobData) + ' ')

with open(filePath, 'a', encoding='utf8') as f:
    f.write(str(metaData) + ' ')

with open(filePath, 'a', encoding='utf8') as f:
    f.write(str(metaTag) + ' ')

%%html
<style>
table {float:left}
</style>

```

Anhang 3.3. Job Ad Extraction – Xing

In this script, you will take the html files that you have saved before, extract meta data from it, narrow down the area around the actual jobdescription and save the document as txt file.

Overview

#	Steps	Required	Input	Output	Next Steps
1	Import Libraries	required	---	---	2
2	Extract Meta Data and Jobdescription and Save File	required	path	txt-file	---

1. Import Libraries

Os, **re** and **BeautifulSoup** have been used in the XingJobs script already, check the description to get more information on the libraries. **datetime** is another library to create a timestamp of the current date and time.

```
import os, datetime, re
from bs4 import BeautifulSoup
```

2. Extract Meta Data and Jobdescription and Save File

In this for loop, each html-file in the folder is opened, parsed with the lxml-parser and turned into a BeautifulSoup object. Using BeautifulSoup's find() and find_all()-functions, you then extract the given meta data and additional data that will be appended as meta data.

Afterwards, script- and style-tags are removed and the actual job description is being assigned to the variable *jobData*. You take the filename and extract the searchterms from it using regex and assign it to the variable *query*.

Before saving the file, you stitch together a filename consisting of the filename of the html-file, a new timestamp and a .txt extension and determine the directory where you want the files to be saved.

With the first *with open* part, you save the jobdescription to file. The second one opens the file again and appends the extracted meta data to the end of the file.

```

### USER INPUT ###
path = 'G:/BA/Stellenbeschreibungen/Corpora/html/neu/'

### STANDARD ###
searchResults = os.listdir(path)

for files in searchResults:
    if files.find('Xing') != -1:
        with open(path + files, 'r', encoding="utf8") as jobs:
            soup = BeautifulSoup(jobs, "lxml")

            title = soup.find('title')
            if title != None:
                title = title.get_text()

            keywords = soup.find('meta', attrs = {'name' : 'keywords'})
            website = soup.find('link', attrs = {'rel' : 'canonical'})
            uploadDate = soup.find('time', attrs = {'itemprop' : 'dateP
osted'})

            for script in soup(["script", "style"]):
                script.extract()

            jobData = soup.find('div', attrs = {'id' : 'maincontent'})

            if jobData != None:
                metaData = soup.find_all('meta')

                metaTag = soup.new_tag('meta')
                metaTag.attrs['downloadDate'] = datetime.datetime.today
().strftime("%Y%m%d_%H%M%S")

                query = re.sub('__', '- ', files)
                query = re.sub('_', ' ', query)
                query = re.sub('Xing|_d+|\.d+|\.html', '', query)

                metaTag.attrs['searchTerms'] = str(query)
                metaTag.attrs['title'] = str(title)
                metaTag.attrs['keywords'] = str(keywords)
                metaTag.attrs['website'] = str(website)
                metaTag.attrs['uploadDate'] = str(uploadDate)

                filenames = re.sub('\d+\.html', '', files)

                filename = filenames + '_' + str(datetime.datetime.toda
y().strftime("%Y%m%d_%H%M%S")) + '.txt'

```



```
filename

filePath = 'G:/BA/Stellenbeschreibungen/Corpora/txt/' +

with open(filePath, 'w', encoding='utf8') as f:
    f.write(str(jobData) + ' ')

with open(filePath, 'a', encoding='utf8') as f:
    f.write(str(metadata) + ' ' + str(metaTag))
```

Style code for the markdown parts in this script. Feel free to ignore it.

```
%%html
<style>
table {float:left}
</style>
```

Anhang 3.4. Data Preparation

In this script, we will prepare the data for analysis. That means, we will tag them according to their part of speech, remove stopwords and reduce them to their wordstem.

Overview

#	Steps	Input or Output Adjustable	Required	Input: Format & Variables	Output: Format & Variables	Next Steps
0.	Setup					
0.1	Install German Classifier	no	yes	---	---	0.2
0.2	Import Libraries and Functions	no	yes	---	---	0.3
0.3	Load Tagger	no	yes	---	---	0.4
0.4	Load Stopword-Spreadsheet	yes	yes	file-id	list of stopwords	1
1.	Data Collection					
1.1	Load Job Descriptions into Document Corpus	yes	yes	path	raw text	1.2
2.	Data Preparation					
2.1	Remove HTML-Tags	yes	yes	raw text	clean text	2.2
2.2	Separate List Items from Raw Text	yes	yes	clean text	list of list items	2.3
2.3	Tokenize List Items and Sort	yes	yes	list items, specification of pos-tags	annotated words	2.4

	them by POS-Tag					
2.4	Turn Tagged Words into Data Frame	yes	yes	list of annotated words	data frame	2.5, 2.6
2.5	Save Tagged Words as Excel-File	yes	no	data frame	excel file	2.6
2.6	Remove Stopwords	no	yes	list of annotated words	list of annotated words without stopwords	2.7
2.7	Stemming	no	yes	list of annotated words without stopwords	list of annotated words without stopwords, but with word stem	2.8
2.8	Data Frame of Prepared Data	no	yes	list of annotated words without stopwords, but with word stem	data frame	2.9
2.9	Saving	yes	yes	data frame	excel files	---

0. Setup

0.1 Install ClassifierBasedGermanTagger

Download the NLTK Contributions zip-folder from here: <https://github.com/ptnplanet/NLTK-Contributions>

Unpack it in the same location where Anaconda is located.

0.2. Import Libraries and Functions

```
import pandas, re, nltk, pickle
from ClassifierBasedGermanTagger.ClassifierBasedGermanTagger import Cla
ssifierBasedGermanTagger
from nltk.corpus import PlaintextCorpusReader, stopwords
from nltk.stem import SnowballStemmer
from nltk.probability import FreqDist
```

0.3. Load Tagger

For the POS-Tagging, we need to load the pickle-file into this script. The file is in the output folder.

```
with open ('input/nltk_german_classifier_data.pickle', 'rb') as f:
    tagger = pickle.load(f)
```

0.4. Load Stopwords-Spreadsheet

The following code loads the stopwords-spreadsheet from Google Drive and turns it into a list of stopwords.

```
fileID = '1srhT4VVawVJaMcoxt2ptzekDQ8m_ks4OnSJU77LTB0U'
url = "https://docs.google.com/spreadsheets/d/{0}/export?format=csv".fo
rmat(file_id)
df = pandas.read_csv(url)
custom_stopw = [word for word in df['Stopwords'].astype(str)]
```

1. Data Collection

1.1. Load Job Descriptions into Document Corpus

Now we use the **function PlaintextCorpusReader()** to load our job ads into a **new variable corpus**, which can be used for document collections. For this we pass as argument to the function the path to the folder in which each job ad is currently saved as individual txt file (see variable `data_folder`).

We then turn every document in the corpus variable into a raw text string.

```
###    USER INPUT    ###
data = 'G:/BA/Stellenbeschreibungen/Corpora/txt/'

###    STANDARD    ###
corpus = PlaintextCorpusReader(data, '.*txt')
all_text = corpus.raw()
```

2. Data Preparation

2.1. Remove HTML-Tags

This time, we do not want to remove all html-tags at once. We want to keep the list item tags that are in the job description that might contain skills. As that also leaves li-tags that are used for the page format, we separately remove li-tags that contain a class.

```
clean_text = re.sub(r'^<[^>]*^<li>>|<li class.*>|\\[.*\\]', '', all_text
)
```

2.2. Separate List Items from Raw Text

As the sentence-tokenizer splits the sentences by punctuation, it struggles with lists, because often a list item does not end with a punctuation mark. Therefore, we will treat list items separately from the rest of the text. After extracting the list items, we also remove unnecessary linebreaks

```
### STANDARD ###
list_items = re.findall('<li>(.*?)</li>', clean_text)
```

2.3. Tokenize List Items and Sort them by POS-Tag

First, we create empty lists for all grammatical types of words we want to work with. Then, depending on what list we want to fill with the loop first, we change the tag in the if-statement of the loop.

Part of Speech Tagging

To determine which word is a noun and which one a verb, etc., we use a technique called **Part-of-Speech Tagging**. As the default part of speech tagger from the nltk-package is not useful for German language text, we have to import a different one. This one was developed by the Wissenschaftszentrum Berlin für Sozialforschung. You can read about how it was trained and get some more background information here: <https://datascience.blog.wzb.eu/2016/07/13/accurate-part-of-speech-tagging-of-german-texts-with-nltk/>

NOTE: The POS-Tagging takes quite a while (roughly 20-30 mins), so you better run this section and then do something else until it is done.

```
nouns = []
verbs = []
adjectives = []
```

```

### USER INPUT ###
### Change the tag in the if-statement ###
adj_tag = 'ADJ'
verb_tag = 'V'
noun_tag = 'NN'

### Determine the list you want to fill ###
list_specification = adjectives

### STANDARD ###
for li_item in list_items:
    tok = nltk.word_tokenize(li_item)
    tagged = tagger.tag(tok)
    li = [(li_item, word, pos) for (word, pos) in tagged if pos.startswith(adj_tag)] #if you prefer to chose all words, remove the if-statement
    for (li_item, word, pos) in li:
        list_specification.append((word, pos, li_item))

```

2.4. Turn Tagged Words into Data Frame

To structure our data, we turn the word-tag pairs into a data frame and add the respective list item the word was found in. Again, specify what list you want to turn into a data frame.

```

### USER INPUT ###
specification = adjectives

### STANDARD ###
tagged_dataFrame = pandas.DataFrame(specification, columns = ["word", "pos-tag", "sentence"])

```

2.5. Save Tagged Words as Excel-File

As the process might take a while, depending on your data size, you might want to save it as an excel-file now. That way you can try out analysing the data without having to go through the entire process again.

```

### USER INPUT ###
#determine file name and path to save excel-file and
#select what you want to save (verbs, nouns or adjectives)
path = 'output/'
filename = 'posTagged_verbs.xlsx'

### STANDARD ###
tagged_dataFrame.to_excel(path + filename, index = False, encoding = 'utf-8')

```

2.6. Remove Stopwords

At first we let the standard stopwords function run through our data, before using our individual one from Google Drive.

```
### USER INPUT ###
specification = adjectives

### STANDARD ###
no_stopwords = [(word, pos, li_item) for (word, pos, li_item) in specification
if word not in stopwords.words('german') and word not in stopwords.words('english')]

no_custom_stopw = [(word, pos, li_item) for (word, pos, li_item) in no_stopwords
if word not in custom_stopw]
```

2.7. Stemming

In the process of stemming the word is reduced to its stem. This needs to be done so we can group the same words in the data analysis phase.

```
stemmer = SnowballStemmer('german')
stemmed = [(stemmer.stem(word), word, pos, li_item) for (word, pos, li_item) in no_custom_stopw]
```

2.8. Data Frame of Prepared Data

To get more structure into our data, we construct another data frame. This time with the stemmed word at the very beginning.

```
prepped_dataFrame = pandas.DataFrame(stemmed, columns = ["stemmed", "word", "pos-tag", "sentence"])
```

2.9. Saving

The final step of the data preparation process is to save the prepared data in another excel file.

```
### USER INPUT ###
path = 'output/'
filename = 'prepped_adjectives.xlsx'

### STANDARD ###
prepped_dataFrame.to_excel(path + filename, index = False, encoding = 'utf-8')
```

Anhang 4: Skript zur Phase IV – Datenanalyse

Identify Competences

In this skript, you will use the skills categories to identify the competences in our prepared job ad data.

Overview

#	Steps	Input Adjustable	Required	Input: Format & Variables	Output: Format & Variables	Next Steps
0.	Setup					
0.1	Import Libraries and Functions	no	yes	---	---	1
1.	Data Collection					
1.1	Load Prepared Words	yes	yes	path	data frame	1.2
1.2	Load Competency Thesaurus from Google Spreadsheet	yes	yes	spreadsheet	data frame	2
2.	Data Analysis					
2.1	Add Stemmed Column to Data Frame	no	yes	list of words	data frame with additional column	2.2, 2.3
2.2	Frequency Distribution of Words in Job Ads	no	no	list of words	word frequencies	2.3
2.3	Match Skill Synonyms with Job Ad Data	no	yes	data frame columns	list of matches	2.4
1.2	Find Competences and Competence Groups to Matches	no	yes	list of matches	data frame column	---

0. Setup

0.1. Import Libraries and Functions

Itertools is used in this script to flatten nested lists (lists within a list). For a description of the other libraries, see notebook **Basic Data Preparation & Word Analysis**

```
import pandas, nltk
from nltk.probability import FreqDist
from nltk.stem import SnowballStemmer
```

1. Data Collection

1.1. Load Prepared Words

In the last script, we prepared our data in a way that we ended up with three excel files. Now we are loading them back into the script, so we do not need to repeat the entire data preparation process.

```
###    USER INPUT    ###
# choose which file you want to work with
verbs = 'input/prepped_verbs.xlsx'
nouns = 'input/prepped_nouns.xlsx'
adjectives = 'input/prepped_adjectives.xlsx'

### STANDARD ###

prepped_dataframe = pandas.read_excel(adjectives)
```

1.2. Load Competency Thesaurus from Google Spreadsheet

Now we will read in a **competency thesaurus** we created before in **Google Spreadsheet** with the format below. Alternatively, with some small changes to the standard code also a local csv file with the same format could be used for the thesaurus. In this case we are using a Google spreadsheet that can be **accessed via URL**. Executing the code will read all data from the spreadsheet and save it under a **dataframe**, which we will call **df**. A dataframe is a datatype in Python that can be compared to a table. Our dataframe df will be used throughout this notebook.

So that we can **compare the synonyms in the last column to words in our documents and this way create a link to the competencies and competency groups**, we need to clean the synonyms by changing all synonyms to lowercase.

```
###    USER INPUT    ###
file_id = "1vhHNFzWiQorvut2OLE7VarMtbT1_agd5_K9mrPCkj_A"

### STANDARD ###
#load spreadsheet
url = "https://docs.google.com/spreadsheets/d/{0}/export?format=csv".format(file_id)
skill_df = pandas.read_csv(url, index_col = False,)
```

2. Data Analysis

2.1. Add Stemmed Column to Skill Data Frame

Before we properly start with the analysis, we need to prepare the skill data frame for matching by stemming the synonyms.

```
stemmer = SnowballStemmer('german')
skill_df['stemmed'] = [stemmer.stem(s) for s in skill_df['Synonym']]
```

2.2. Frequency Distribution of Words in Job Ads

The frequency distribution gives us an overview on how many words there are in our dataset. Depending on the outcome, it might make sense to add some words to the stopwords list and repeat the stopword-extraction.

```
frequent = FreqDist(prepped_dataframe["stemmed"])
most_frequent = frequent.most_common(50)
print(most_frequent)
```

2.3. Match Skill Synonyms With Job Ad Data

To find the matches, we go through every synonym and see whether there is a match in the job ad data set. If this is the case, the word will be added to the **match_skills** list.

```
match_skills = []

for synonym in skill_df['stemmed']:
    for adjective in prepped_dataframe['stemmed']:
        if synonym == adjective:
            match_skills.append(synonym)
```

2.4. Find Competences and Competence Groups to Matches

To do so, we simply get the entire data frame column for each match. This can then be used to further process the analysis.

```
skill_df.loc[skill_df['stemmed'].isin(match_skills)]
```

Anhang 5: Kategoriensystem

Kompetenzgruppe	Kompetenz	Synonym
fachlich	Regelwerk	RDA
fachlich	Regelwerk	RAK-WB
fachlich	Bibliothekssystem	PICA
fachlich	Bibliothekssystem	Aleph
fachlich	Bestandsaufbau	Bestandsaufbau
fachlich	Medienkompetenz	Medienkompetenz
fachlich	Medienkompetenz	Medienkompetenzvermittlung
fachlich	Benutzerstandards	Bibliotheksstandards
fachlich	Katalogisieren	Katalogisieren
fachlich	Recherche	Recherchekenntnisse
fachlich	Recherche	Literaturrecherche
fachlich	Recherche	Recherchestrategie
fachlich	Recherche	Suchmaschinen
fachlich	Bestandspflege	Bestandspflege
fachlich	Daten	Forschungsdaten
fachlich	Daten	Metadaten
fachlich	Daten	Metadatenstandards
fachlich	Regelwerk	AACR
fachlich	Wissensorganisationskenntnisse	Dewey Decimal Classification
fachlich	Wissensorganisationskenntnisse	Thesaurus
fachlich	Wissensorganisationskenntnisse	Systematiken
fachlich	Wissensorganisationskenntnisse	Wissensmanagement
fachlich	Wissensorganisationskenntnisse	inhaltliche Erschließung
fachlich	Wissensorganisationskenntnisse	Aufstellungssysteme
fachlich	Rechtskenntnisse	Lizenzrecht
fachlich	Statistik	Statistik
fachlich	IT	MS-Office
fachlich	IT	SPSS
fachlich	IT	Projektmanagementsoftware
fachlich	IT	SQL
fachlich	IT	Scrum
fachlich	IT	Python

fachlich	IT	R
fachlich	IT	Datenbanken
fachlich	IT	SAS
fachlich	IT	Java
fachlich	IT	Softwareentwicklung
fachlich	IT	Webapplikation
fachlich	IT	Programmiererfahrung
fachlich	IT	XML
fachlich	Finanzplanung	Budgetverwaltung
fachlich	bibliothekarische Kenntnisse	systembibliothekarische Kenntnisse
methodisch	Problemlösung	Problemlösung
methodisch	Management	Management
methodisch	Führungsqualitäten	Führungsqualitäten
methodisch	Führungsqualitäten	Personalführung
methodisch	Organisation	Organisation
methodisch	Zielorientierung	zielorientiert
methodisch	Dienstleistungsorientierung	dienstleistungsorientiert
methodisch	konfliktfähig	konfliktfähig
methodisch	Präsentationsfähigkeiten	Präsentation
methodisch	Projektmanagement	Projektmanagement
methodisch	Methodik	methodisch
methodisch	Methodik	strukturiert
methodisch	Methodik	Abstraktionsfähigkeit
persönlich	analytisches Denken	analytisches Denken
persönlich	Beobachtungsgabe	Beobachtungsgabe
persönlich	Einfallsreichtum	Einfallsreichtum
persönlich	Konzentrationsfähigkeit	Konzentrationsfähigkeit
persönlich	Kritikfähigkeit	Kritikfähigkeit
persönlich	Selbstbewusstsein	selbstbewusst
persönlich	Selbstbewusstsein	sicher
persönlich	Belastbarkeit	Einsatzbereitschaft
persönlich	Einsatzfähigkeit	flexibel
persönlich	Einsatzfähigkeit	Reisebereitschaft
persönlich	Einsatzfähigkeit	Mobilität
persönlich	Engagement	Engagement

persönlich	Engagement	enthusiastisch
persönlich	Engagement	ergebnisorientiert
persönlich	Lernbereitschaft	Lernbereitschaft
persönlich	logisches Denken	logisch
persönlich	Durchsetzungsvermögen	Durchsetzungsvermögen
persönlich	Kreativität	Kreativität
persönlich	Arbeitsweise	systematisch
persönlich	Arbeitsweise	Detail
persönlich	Belastbarkeit	Belastbarkeit
persönlich	Verantwortung	Verantwortungsbewusstsein
persönlich	Lernbereitschaft	Lernfähigkeit
sozial	Teamfähigkeit	teamfähig
sozial	Empathie	empathisch
sozial	Kundenorientation	kundenorientiert
sozial	Auftreten	freundliches
sozial	soziales Engagement	soziales Engagement
sozial	Hilfsbereitschaft	hilfsbereit
sozial	Kommunikationsfähigkeit	Kommunikationsfähigkeit
sozial	Kommunikationsfähigkeit	Ausdrucksvermögen
sozial	Kommunikationsfähigkeit	kommunikativ
sozial	Kommunikationsfähigkeit	Englischkenntnisse
sozial	interkulturelle Kompetenz	interkulturelle kompetenz
sozial	Verhandlungsgeschick	Verhandlungsgeschickt
sozial	Beratung	Beratung

Anhang 6: Stoppwortliste

Stopwords
bewerben
Bewerbung
Lebenslauf
einreichen
zusenden
sind
wären
ist
bieten
können
übernehmen
gut
gleichwertig
wünschenswert
Position
Hauptbahnhof
Studium
Vorteil
Bereich
Erfahrung
Anfang
Arbeitszeiten
arbeiten
verfügen
können
passen
ernten
high
gute
hohe
wünschenswert
alternativ

gängig
eigene

Eidesstattliche Erklärung

Ich versichere, die vorliegende Arbeit selbstständig ohne fremde Hilfe verfasst und keine anderen Quellen und Hilfsmittel als die angegebenen benutzt zu haben. Die aus anderen Werken wörtlich entnommenen Stellen oder dem Sinn nach entlehnten Passagen sind durch Quellenangabe kenntlich gemacht.

Hamburg, d. 07.11.2016

Kathrin Wardatzky